

AI for Risks

David L. Olson

Project Management Tools

 Springer

AI for Risks

Series Editors

Desheng Wu, School of Economics and Management, University of Chinese Academy of Sciences, Beijing, China

Jim Lambert, University of Virginia, Charlottesville, VA, USA

David L. Olson, Department of Management, University of Nebraska–Lincoln, Lincoln, NE, USA

Risks are widespread in human society, such as in the financial field (various investment risks), in technical fields (risks brought by emerging technologies), in social fields (political risks), and in all aspects of our lives (health risks, natural environment risks, etc.) At the same time, the inherent links between various risks generate systemic risks. The major challenge today is not to deal with new types of risks, but to focus more on those risks that are difficult to distinguish effectively and timely, or risks that have emerged in a different way from the past.

This series of books aims to strengthen discussions on frontier hot topics across disciplines, that is, using AI technologies to solve risks problems that exist in the environment, healthcare, technology, and financial fields. The scope of it focuses on the implementation of artificial intelligence technology in dealing with risks, such as risk prediction, assessment, and mitigation. It includes monographs, edited volumes, textbooks and proceedings etc. on the application of artificial intelligence in risk management in the fields of social media, healthcare, the public sector, financial technology, and regulatory technology.

David L. Olson

Project Management Tools

David L. Olson 
Supply Chain Management and Analytics
University of Nebraska-Lincoln
Lincoln, NE, USA

ISSN 2731-6327

ISSN 2731-6335 (electronic)

AI for Risks

ISBN 978-981-97-1719-4

ISBN 978-981-97-1720-0 (eBook)

<https://doi.org/10.1007/978-981-97-1720-0>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2024

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Singapore Pte Ltd. The registered company address is: 152 Beach Road, #21-01/04 Gateway East, Singapore 189721, Singapore

Paper in this product is recyclable.

Preface

Projects are found everywhere. The fast pace of change in our world calls for constant development of new products and processes. Building new facilities creates the need for engineering and/or construction projects. In order to keep close to the cutting edge of computer power, many information systems projects are required. Large accounting firms have enlarged their information systems consulting operations, and almost all of this type of consulting involves an information systems project. Governments get involved in conflicts and require new and better munitions systems. The COVID epidemic that began in 2020 created the need for new drugs, another specialty type of project involving pharmaceuticals and regulatory agencies. There are more and more projects of varying complexity. They involve unique activities that draw people together from diverse locations and diverse organizations with diverse, specialized skills.

This book follows up two books published with Business Expert Press in 2020: *Core Concepts of Project Management* and *Quantitative Tools of Project Management*, to which the author has retained rights. Those books focused on information systems projects. This book takes a more general approach, unifies and updates those publications.

Chapter 1 gives an overview of projects and discusses the difficulties in completing projects on time, within budget and at designed functionality. While the successful completion of a project is a challenge, there are some things that can be done to improve the probability of a project's success. The book reviews a number of project management concepts. These include developing organizational ability to work on projects, as discussed in Chapters 2 and 3. Sponsor expectations can be based on better information if a good job of project development, estimation and selection is conducted, as discussed in Chapters 4, 5 and 6. Project planning tools involving the critical path method are covered in Chapter 7. Tools to evaluate project risk are covered in Chapter 8. The critical chain method is covered in Chapter 9. Chapter 10

discusses means to rush projects when circumstances demand. Chapter 11 covers project implementation and control, including assessment of delay responsibility.

Lincoln, NE, USA

David L. Olson

Contents

1	The Importance of Project Management	1
1.1	What is a Project?	1
1.1.1	Projects	2
1.1.2	Project Characteristics	2
1.1.3	Types of Projects	3
1.1.4	Dimensions of Complexity	4
1.2	Project Size by Group Size	4
1.2.1	Modern Business	5
1.3	Project Management Systems	6
1.3.1	Project Entities	7
1.4	The Project Environment	7
1.4.1	Project Success	8
1.4.2	Project Failure	8
1.4.3	Primary Reasons for Project Failure	9
1.5	Quality Control in Project Implementation	10
1.6	Summary	11
2	Human Factors	13
2.1	Information Systems Project Features	13
2.2	Conflict and Performance	16
2.3	Project Managers	17
2.3.1	Comparison: Functional and Project Managers	18
2.4	Summary	19
3	Project Organization	21
3.1	Alternate Organization Structures	21
3.1.1	Functional Organization	22
3.1.2	Project Organization	23
3.1.3	Matrix Organization	24
3.2	Comparison of Organization Structures in Projects	25
3.2.1	Task Forces	25
3.2.2	Hybrids	26

3.2.3	Levels of Project Organization	26
3.2.4	Criteria for Selection	27
3.3	Summary	27
4	Project Estimation	29
4.1	Software Project Estimation	29
4.2	Overview of Analysis and Design Methods	31
4.2.1	The Waterfall Model	31
4.2.2	Prototyping	32
4.2.3	Agile Software Development	33
4.2.4	Other Options for Systems Development	34
4.3	Software Estimation Methods	35
4.3.1	Lines of Code	35
4.3.2	Function Point Analysis	36
4.4	Demonstration of Count-Total Calculation	38
4.4.1	Constructive Cost Model	39
4.5	Capability Maturity Model	41
4.6	Systems Development Approach	43
4.6.1	Specification	44
4.6.2	Design	44
4.6.3	Code (or Acquisition)	46
4.6.4	Testing	47
4.6.5	Implementation	47
4.7	Summary	47
5	Planning	49
5.1	Planning Process	49
5.1.1	Set Objectives and Requirements	50
5.1.2	Specify Work Activities	51
5.1.3	Plan Project Organization	53
5.1.4	Develop the Schedule	55
5.1.5	Control Points	55
5.2	Planning for Change	56
5.2.1	Scheduling	56
5.3	Summary	57
6	Selection from Project Options	59
6.1	Measurement of Project Impact	59
6.2	Selection Practice	60
6.3	Project Evaluation Techniques	61
6.3.1	General Project Selection	61
6.3.2	Screening	61
6.4	Traditional Analysis	64
6.4.1	Cost–Benefit Analysis	64
6.4.2	Payback	65
6.4.3	Net Present Value	65

6.4.4	Other Factors	66
6.5	Multiple Objectives	68
6.5.1	Conflicts	68
6.6	Summary	70
7	Project Scheduling	73
7.1	Early Start Schedule	74
7.2	Networks	75
7.3	Late Start Schedule	75
7.4	Slack	76
7.5	Resource Leveling	78
7.6	Multiproject Environments	81
7.6.1	Resource Smoothing	82
7.7	Critical Path Criticisms	85
7.8	Summary	86
8	Probabilistic Scheduling	87
8.1	PERT	87
8.2	Criticisms of PERT	90
8.3	Simulation for Project Scheduling	91
8.3.1	Generating Multiple Simulation Runs	93
8.4	Summary	95
9	Critical Chain Scheduling	97
9.1	Buffers	97
9.2	Critical Chain Process	98
9.3	Fever Charts	102
9.4	Summary	103
	References	108
10	Agile Project Development	109
10.1	PMI View of Agile	109
10.2	Agile Definition	110
10.3	Scrum	113
10.3.1	Artifacts	114
10.3.2	Processes	114
10.3.3	Kanban	114
10.4	Project Crashing	116
10.5	Summary	118
	References	118
11	Implementation and Control	119
11.1	Risk Management	119
11.1.1	Personnel Shortfalls	119
11.1.2	Controlling Dynamic Requirements	120
11.1.3	Controlling Externally Provided Project Components	120

11.1.4	Real-Time Performance Risk	121
11.1.5	Contingent Development Methods	121
11.1.6	Unrealistic Estimates	121
11.2	The Control Process	122
11.2.1	Responsibility Assignment	123
11.2.2	Budget	123
11.2.3	Work Authorization	127
11.2.4	Project Evaluation	128
11.3	Schedule Delay	129
11.3.1	Demonstration Schedule	130
11.4	Summary	133
	Reference	135
	Glossary	137
	Index	145

Chapter 1

The Importance of Project Management



Construction projects have been around since before the pyramids. Construction projects are stable compared to software projects, due to less variability in durations. Information systems project management involves some characteristics that are different from those found in construction, but many of the same tools can be applied. The information technology environment involves high turnover of personnel, turbulent work environments, and rapidly changing technology. This results in high levels of uncertainty with respect to time and cost. Despite this more volatile environment, project management principles applicable to operations management can often be transferred to the information systems environment. Massive government projects and pharmaceutical development projects involve a much greater level of complexity due to heavy regulation and the complexity involved due to the participation of multiple organizations.

1.1 What is a Project?

A project involves getting a new, complex activity accomplished. Many activities qualify as projects. Building the Chunnel, attempting to elect a candidate to public office, and producing a movie or television show are all major projects. So were the development of the atomic bomb and sending men to the moon.

Each political campaign is a marketing project, just like other marketing projects to sell new products. You have each written a paper, which was assigned as a “project.” These projects involved researching some topic and organizing ideas into a cohesive, rational whole.

1.1.1 Projects

- Involve a definable purpose
- Cut across organizational lines
- Are unique activities

Projects are purposeful in that they are designed to accomplish something for the organization undertaking them. Projects usually cut across organizational lines, drawing people from a variety of functional specialties. Constructing automobiles on an assembly line is no longer a project once the assembly line is developed, because it becomes a closed, repetitive activity that continues as long as anyone can foresee. Making a series of sales calls is not a project, because it is not a unique activity. However, just like the first assembly line, the first round of sales calls is a project, until a desired level of competence is attained. Projects include:

- Constructing something
 - a road, a dam, a building, an information system
- Organizing something
 - a meeting, an election campaign, a symphony, a movie
- Doing anything the first time
- Accomplishing a new, complex activity

1.1.2 Project Characteristics

Because projects involve new activities, they typically involve high levels of uncertainty and risk. One of the reasons assembly line operations are efficient is that everyone does the same thing over and over, hour after hour, day after day, year after year. This repetitiveness allows high degrees of specialization, which in turn enables greater productivity. The activities of many different people and machines can be balanced for maximum efficiency in an assembly line operation. **Projects involve lower degrees of efficiency** than are obtained in assembly line operations.

Because of this higher degree of uncertainty, it is much more difficult to estimate the level of resources required to accomplish a project than it is for other forms of productive organizations. It is also more difficult to estimate the time required (which amounts to another resource). Many projects are late, but not all projects take longer than estimated. The Russian atomic bomb project was completed ahead of schedule. The U.S. U-2 airplane project was finished in about one-tenth of the estimated time. Yet, projects finished ahead of schedule are still rare. **Projects are collections of activities.** If one activity is late, other activities often have to wait for it to finish. If an activity is ahead of schedule, those doing the work tend to be more careful, or slow down for other reasons. The activities that follow often cannot start early anyway, as the people and materials for those activities may not be available until the originally scheduled starting time. For these and other reasons, it is far more common for projects to be late than to be finished early.

Because of their temporary nature, projects inevitably involve gathering a diverse group of specialists to accomplish a variety of tasks. Project team members often will not know each other very well, at least in the beginning of the project. They will tend to be quite different people, with different skill sets and interests. The primary feature of a project is that it is a **set of temporary activities conducted by ad hoc organizations**.

Information systems projects have many similarities to generic projects. They consist of activities, each with durations, predecessor relationships, and resource requirements. They involve high levels of uncertainty and often suffer from time and cost overruns, while rarely experiencing time and cost underruns. However, information systems projects are different from other types of projects in some respects. While each project is unique, there are usually many, many replications of the types of information systems projects. Most are served by a standard methodology, with the need to identify user requirements, followed by design of a system, production of the system, testing of the system, training users, and implementation, and ultimately maintenance of the system. These steps are not always serial, with many loops back to prior stages. They involve the need for specialists in different areas of the information systems field, but these specialties are not as distinctly different as carpentry and electrical work. Systems analysts usually know how to program, and testers know all of the other functions involved in a project. Project team members from the development side usually understand each other well. Information systems projects of course involve computers, which is a distinct characteristic that has more impact than what might have been apparent initially.

1.1.3 Types of Projects

Projects in the engineering world tend to involve a lot of uncertainty (especially with respect to how long they will take). But information systems projects have added levels of uncertainty. To demonstrate these differences, let us consider four types of projects: engineering (construction), political, movies, and information systems.

First, engineering projects involve more physical activities, while the other three types of projects involve people creating something. It can be argued that engineering projects are thus much easier to manage, because scheduling is a matter of calculating physical quantities and using past production rates to determine how long activities should take.

Second, information systems projects require specialists with different skill sets to work together to create a software product. Movies are similar in this respect. There may be a variety of skills needed for political projects, but for the most part, it is public relations—working with the press to put your candidate in the best light and getting maximum positive exposure. (Some specialists may be needed to cover up negative exposures.) Information systems and movies involve lots of different specialties. Movies need actors, a staff to make actors happy, camera people, directors, grips, and lots of other things. Information systems projects need system analysts, programmers

and/or software developers, testers, system installers, trainers, and other specialty skills.

Third, projects involving humans creating things are much more difficult to estimate, because it is more difficult to estimate how long a creative activity (like writing a bug-free code) will take. Political campaigns are more predictable because there is an end point—voting day. Effectiveness might be hard to estimate, but duration is pretty much given. Movies also have planned schedules, but directors may feel that artistic creativity was lacking in scheduled shots and insist on redoing them. The fourth type of project, involving information systems, clearly involve less certainty as to duration than the other three kinds of projects considered here.

1.1.4 Dimensions of Complexity

Projects can differ on several aspects. These include the number of people involved and the diversity of skills involved. Some projects are individual efforts to accomplish something. Others, like a major military campaign, can involve hundreds of thousands of people. The more people that are involved, the greater the need to organize into subunits, requiring a higher proportion of managers and thus a lower proportion of productive people. In general, the more complex the project, the more time and resources required.

Group size dimensions can vary over extremes. A few examples of projects for different sized groups, ranging from individual effort through three general group levels, are given for comparison.

1.2 Project Size by Group Size

- **Individual:** A term paper is usually supposed to be an individual effort
A landscape painting is also usually an individual project.
- **Group:** Organizing a wedding can be a major small group project
Implementing a computer system may involve a small group.
Accounting audits are projects involving a group of specialists.
- **Organization:** Construction projects involve companies-alone or with others
Election campaigns involve political party organizations.
- **Multio rganization:** Space shuttles or satellite launches involve multiple organizations.
Military campaigns involve complex logistical supply chains.

Projects also differ on the dimensions of uncertainty. It is much more difficult to predict how much time is to be required the first time you do something. Since projects are usually things done for the first time, they usually take longer than expected when they were estimated. Information systems currently are in very high

demand, outstripping their supply. Another possible bias is introduced by the practice of making initial estimates intentionally low to get work. This bias improves the probability of getting work, which is often negotiated on a cost-plus basis. This practice is not at all recommended, as it leads to a bad reputation when initial promises are not kept. Furthermore, it has ethical ramifications with respect to truth in advertising. An additional factor in project lateness is that large government projects are the most reported. These projects tend to be very complex and often overrun time and budget. How many times have you read about a government project of significant magnitude taking less time than estimated? Since there is a strong correlation between time and money, late projects almost always cost more than expected. When was the last time you heard of a government project having a cost underrun?

General project management is a field that has developed primarily since World War II. With more complex undertakings, many project management principles have been developed. They typically involve a cost/time/quality trade-off, found in almost any project. Specifically in the information systems field, this trade-off can be stated as follows:

You can have any two of three aspects of a project: You can get it done on time, you can get it done within budget, or you can attain designed quality.

This adage is not presented as a recommended way to treat all projects. We all like to think that we can do better than anyone else and accomplish all three tasks. But over and over, in the fields of construction, government projects, and information systems, problems in completing projects on time, within budget, and meeting specifications have been encountered. Project management cannot be blamed for all these reported failures. The point is that we should understand the difficulties involved in a project environment, seeking to understand the project as a system so that we keep it on target with respect to accomplishing what it is intended to do, in the most timely and efficient manner possible. Bringing in a project on time, within budget, and meeting specifications is tough. Project managers need to expect difficult challenges.

1.2.1 Modern Business

Business has grown much more complex, with interrelated currencies and stock markets. The pace of business is at the speed of light, as stock trading is conducted electronically, oftentimes by artificial intelligence systems. Information technology markets are less predictable. The outputs of many companies are tied together through just-in-time systems with dedicated suppliers. At the output end of production, producers and retailers are often connected through electronic data interchange. The international aspect of business is typified by arrangements such as GATT, NAFTA, and the European Economic Community. The rapid pace of change has resulted in the disappearance of many companies, age-old organizations like the Southwest and Pac12 Conferences, and entire countries like the Soviet Union.

In the rapidly changing world of business today, there is a growing need to manage projects intelligently. Project management advanced a great deal in the defense, aerospace, and construction industries. Techniques developed for controlling the interrelated activities of many different organizations and crews can be applied to the field of information systems, which includes many projects to install new applications or to tie old applications together.

1.3 Project Management Systems

Projects are systems. Subsystems found in project management systems include a technical core, a control subsystem, and a project information subsystem. The **technical core** includes the technical expertise and equipment that gives the system the ability to accomplish what it needs to do. Expertise can include systems analysis, program development, testing, installation, and user training skills. Equipment in a broad sense can include software, such as CASE tools and subroutines, that improve productivity. The **control subsystem** is the means management has available to control operations. Within an organization, this control subsystem coordinates the technical core with the outside environment. In an institution, an example of a control subsystem is the board of directors, which approves goals and strategies for the organization (which are usually generated by top management). In a project management system, control includes procedures specified for specific tasks, milestones to mark the completion of project phases, and the expertise available within the project team to solve problems when they are encountered. The **project information subsystem** gives management measures of how the system is accomplishing its objectives. Project information systems need to record the current status of activities, list responsibilities, planned and actual durations of activities, and cost expenditures.

The value of viewing projects as systems is that the total view of the project considering its intended purpose is clearer. Projects consist of many interrelated tasks, done by different people with different skills. If each task was accomplished in isolation, many suboptimalities would occur. Possibly specific tasks would be done faster or at less expense if the rest of the project was disregarded, but the focus of each member of the project team should be to accomplish project objectives and not to optimize production of specific tasks. If trade-offs exist between task accomplishment and project accomplishment, the systems view makes it clear that overall project considerations come first.

Systems provide a useful framework within which to view projects. To make projects work, project managers need to be able to anticipate the consequences of planned actions. They need to develop an organization system, through hiring and training appropriate, qualified people, within budget. They need to be able to know who they must deal with outside of the system, for supplies, materials, regulation compliance, etc. They need to understand how to measure how the project is progressing and what controls are available if the project does not progress as

planned. Understanding the concept of systems makes it much easier to see the impact of the principles of project management.

1.3.1 Project Entities

Several people are needed to make projects work. We have stated that user involvement is important. One reason is that they are the client (**stakeholders**). The client paying for or controlling the project is the **sponsor** (sometimes appearing in the form of a project board), causing the project to be undertaken. Related to the sponsor is the **project champion** (sometimes another synonym for sponsor). A project champion may not have authority, but has influence at the budgetary authorization level, and often serves as a cheerleader in keeping top management support for the project high.

The **project manager** coordinates the efforts of people coming from a variety of functional areas. Project managers also need to integrate planning and control costs, by assigning tasks and schedules to the members of the project team.

The **project team** is a group of people with the required skills to accomplish the project. They will often come from different places with radically different skills and backgrounds. Oftentimes these project team members will enter the project (and leave) at different times, making for an even greater degree of turbulence. People who work on projects need to be very flexible and to learn to work with various people.

The **project management system** is the organizational structure used by the project manager to get things done. The project management system includes the information system to provide project team members with the necessary information, as coordination between groups is critical to integrate activities. Organizational structure involves procedures to ensure accurate communication and completeness of activities.

1.4 The Project Environment

Successful implementation has been found to require mastery of the technical aspects of systems along with an understanding of key organizational and behavioral dynamics. There has been a great deal of study of information systems **project failure**. Failure can arise due to failing to meet design objectives. Projects also can fail with respect to time and budget constraints. Seemingly successful projects may fail because their intended users do not use them. And finally, systems may not meet the expectations of stakeholders.

In this book, we began with a discussion of the risks involved in accomplishing a project. We then discussed what is involved in information systems project development. We now return to an overall perspective of what is involved in the final

implementation of projects, with the intent of examining things that can be done to improve a project's prospects of success.

1.4.1 Project Success

Not all projects fail. Care to involve users early in the requirements analysis has proven to be very effective in information system projects. Although there was inevitable variance in the computer literacy of users, the comprehensive and carefully planned training program for the implementation of the new corporate-wide information system led to the successful adoption of a more efficient and flexible system.

1.4.2 Project Failure

There are many cases of projects failing, for a variety of reasons. The implementation stage involves putting the system developed by the project into operation and turning the system over to the user. For information systems projects, the major elements of implementation are system installation and conversion, user acceptance testing, and user training. If testing procedures are cut short as a means of recovering lost time, catastrophic failure often results.

Many projects fail to some degree in terms of meeting design specifications or meeting time and cost estimates. Most project problems are not apparent until late in the project.

Technical validity refers to the system accurately doing the job it was designed to do. Organizational validity has to do with people using the system. Systems can be technically valid, but if there are reasons why users will not use them (such as difficulty in using them), the system will have been a waste. Organizational effectiveness refers to how the system contributes to better performance by the organization. Information system elements are meant to lead to better decision making, but the quality of decision making is impossible to measure except in very narrow contexts. Profit is often used as a measure of effectiveness, but rarely is only one factor responsible for changes in profitability. Probably the best measure of information system effectiveness is its use. Those systems that are used probably are successful, but even those may involve problems.

Four major categories of project system failure are:

- Corresponding failure
- Process failure
- Interaction failure
- Expectation failure

Corresponding failure alludes to the failure of the system to meet design objectives. This is a technical failure in that a computer code did not do what it was intended to do.

Process failure is a failure to bring in a project system on time and within budget. The system may technically work, but it is no longer economically justifiable, or at least not within current business plans.

Interaction failure occurs when a system is not used as much as it was planned to be. This can arise when a system is built to technical specifications within budget and on time, but the intended users do not use it. This can be because of some bias on the users' part to continue to operate the old way, or because the planned system design really did not effectively deal with the problem.

Expectation failure occurs when the system does not quite match up with the expectations of project stakeholders. The system may perform technically, and may be on time and within budget, and may be used, but may not do the job as management was led to expect.

1.4.3 Primary Reasons for Project Failure

Surveys of project failure consistently identify user involvement as the primary reason, followed by lack of top executive support, and lack of clear statement of business objectives. While these factors are specific to information systems projects, they have a lot of similarity to failure reasons in any project. These traditional critical success factors continue to be cited as the most important.

1.4.3.1 Lack of Client Involvement

Heavy involvement on the part of users is needed for project success for two important reasons. First, systems analysts need user input to accurately identify the business system, and what business problems are. Second, the project will not work if users do not use the computer system. If the users have been involved in identifying computer system problems, and have been consulted about proposed solutions, there is a much greater likelihood that they will accept the project.

1.4.3.2 Lack of Top Management Support

Projects have a very difficult time succeeding if they are not “fostered” by top management. Project leadership is important too, but favorable views at the top, with control over purse strings, are probably more necessary for projects to succeed.

That is not to say that support within the project team is not important as well. The implementation team needs to develop team building and cooperation to succeed. Successful team characteristics include a clear sense of mission, understanding of

Table 1.1 Comparison between project managers and champions

Project manager	Project champion
Technical understanding	Cheerleader
Leadership and team building	Visionary
Coordination and control	Politician
Obtain and provide support	Risk taker
Administrative	Ambassador

interdependencies of system elements, cohesiveness of the system, and shared enthusiasm and trust within the project team. Within project teams, it is necessary to be flexible. It is also necessary to keep everyone informed.

Although there is no one best leadership style, project managers should have the ability to motivate and inspire. This in great part is because they have little coercive authority, especially within matrix organizations.

In addition to the project manager, another key role is the project champion. Project champions often have no authority but are crucial to the success of projects. A comparison between project managers and champions is given in Table 1.1.

Project champions have been found to be crucial at maintaining interest in information systems projects. This can be very positive, but it can also be negative if counterproductive projects are kept alive after their need has passed. This also has occurred when strong project champions have been present.

1.4.3.3 Lack of Project Definition

A detailed project plan needs to be developed, defining the system, user requirements, and system requirements. Part of this plan is a clear statement of the business objectives of the proposed system.

The system definition should identify the project team, including its leaders. Team members should be drawn from functional areas. Networks of the flow of information should be developed, along with identification of tasks and their relationship. Budgets need to be established and policies need to be stated.

User requirements include performance measures. These measures are needed to determine the acceptability of the final product. For legal reasons, as well as for operational continuity, it is wise to have good documentation. This documentation becomes the ultimate reference for resolving issues that arise during the project.

1.5 Quality Control in Project Implementation

Implementation planning involves developing plans and gathering resources to install the system and train users. **Quality control** is an important element in implementing the project.

Table 1.2 Relative defect impact

Stage	Relative cost of defect removal
Requirements definition	1
Design stage	3.5
Coding stage	10
Testing stage	50
After delivery	170

One reason that reported losses on information systems projects are so high is that many projects are canceled late in the project’s life. This is because most problems are not identified until the testing stage of a project, after much of the cost of the project has been expended. An initial study of project failure found that for large projects, the ratio of removing defects by phase was estimated to be as shown in Table 1.2:

It is obvious that correcting errors at an early stage is far better than later. To identify errors at early stages, early testing is required. High levels of quality in any operation are obtained by quality inputs, careful production, followed by testing in realistic environments, and user feedback.

Actual installation of information systems can be accomplished in a variety of ways. Alternative strategies include parallel installation, pilot operation, and cold turkey approaches. **Parallel installation** involves running the new system in parallel with the old system. This is the most expensive approach because the resources required to operate both systems are used. However, it is far safer than other approaches. The **pilot operation approach** involves running the new system on a limited basis. This does not expose the system to a full load. Obvious problems can be identified by this pilot approach, but problems due to the workload will not be detected. The **cold turkey approach** is to place a great deal of faith in the new project, pull the plug on the old system, and turn on the new system. This approach is not recommended if it can be avoided.

1.6 Summary

Project management has many features that are different from those of repetitive operations. These include:

- Lower degrees of efficiency
- Operating in a much less predictable market with more rapidly changing technology
- The need to coordinate more parties and organizations
- A highly dynamic environment involving temporary tasks

Projects are systems consisting of interrelated parts working together to accomplish project objectives. There are several important roles within information systems

projects. Project managers must balance technical understanding with the ability to motivate diverse groups of people (the project team) brought together on a temporary basis. Managing this team requires organizing in a way that groups can coordinate their diverse activities. Project champions play an important role in obtaining organizational commitment to projects.

All projects involve uncertainty. But information systems projects have higher degrees of variability. While there are many valuable information systems projects that have been completed, the development environment is very difficult. Rarely do information systems projects get completed on time, stay within budget, and fulfill specifications simultaneously. Top management support to projects has repeatedly been found to be critical to information systems project success. User groups need to be consulted to find out just what systems will be required to do. Systems designers need to be involved to make sure that new systems fit in with the overall organizational information system. Programmers need to be involved to ensure realistic production rates. End-users need to be involved to ensure the quality of systems by making sure that they are usable and useful. After planning, many meetings need to be held to coordinate the project through acceptance and completion.

Most information systems projects have been reported to be much less successful, reflecting in part a very turbulent environment where many changes are needed. Quite often, management gives up and changes direction. This is not always possible to do.

Four levels of project failure were presented. Studies of both information system project failure and success were reviewed, intending to identify the circumstances that led to failure, and using these bad experiences to identify procedures that can reduce the likelihood of failure. Studies have found that the primary causes of information system failure are lack of user involvement, lack of top management support, and lack of clear system objectives. Similar factors apply to projects in any field. Critical success factors in information system implementation were found to change with project stages, but to primarily influence the motivation of the project team as well as the three primary factors of user involvement, top management support, and clear system objectives.

Three factors have consistently appeared as success factors in project failure. These factors, which are also found in general kinds of projects, comprise the following:

- Client involvement
- Top management support
- Clear statement of project objectives

Chapter 2

Human Factors



A major success factor in the management of any project is combining the efforts of a diverse set of skills. Project participants tend to join the project on call as needed, meaning that most of the project team do not work together throughout the life of the project. This makes communication and coordination paramount in project success.

A great deal of the effort involved in making a computer software project work is to get the project development team to work together. This is often difficult because many computer software developers view life much like engineers and scientists. Their emphasis focuses on getting computers to work. Computers have no feelings and do exactly what they are told. This environment leads to those people sometimes using a rather abrupt and curt mode of communication. Their ability to communicate effectively with humans is occasionally curtailed. Effective management of teams of software developers requires understanding both their personal tendencies and those of users, usually a quite different personality type.

Information systems projects involve the need to coordinate the efforts of a diverse group of people. One of the most difficult (but also one of the most lucrative) jobs in the world right now is project management. These people have very high-pressure jobs, with a lot of demands upon them, and they tend to have a lot of turnover. However, project managers with experience are paid very well.

2.1 Information Systems Project Features

Information systems projects promise a great deal of benefit to organizations. There are many kinds of information systems projects. Enterprise resource planning (ERP) systems are very large scale, possibly centralizing all information systems support for an organization. ERP systems can be built in-house, but usually are purchased software systems. Installation of an ERP is a very large project, often involving interorganizational elements (including vendor and consultant personnel), but little

systems development and programming. Web systems are at the other extreme. They are a very useful means of implementing e-commerce and e-business (and other e-whatever), but usually are small scale, and can be turned over to one specialist who does everything. (There may be good reasons to have a team specializing in developing many websites, and there is a definite need to work with users.) Conventional information systems projects often consist of a series of development activities, referred to as a **waterfall model** (see Table 2.1) because if all goes well, results of prior stages spill down to the next stage. (In reality, a great deal of looping back is encountered.)

Systems analysts have the job of identifying what the needs of the system are, and then designing an appropriate software product. This involves the need to understand business problems and to talk to business users, as well as the ability to understand the technical domain and to talk to computer personnel. **Programmers** are usually specialists who talk to computers and make them function. They can do a more complete job if they talk to humans as well, but some specialize. Testers are very important personnel who try their best to find flaws in systems. Sometimes testers obtain character traits reflecting this critical attitude, but to do their job well, they need to be very meticulous. Systems administrators have the job of implementing software on server systems. This tends to be even more computer-centric than programming. Maintenance personnel are responsible for fixing bugs that are encountered, as well as keeping systems up to date with hardware and software system changes.

There are other information systems job titles as well. For instance, there are workers in database management, in providing help to users, in linking networks together, and many other specialties. Some of these may well be involved in specific projects. But generically, Table 2.1 lists the primary information system specialists found in conventional information systems projects.

Working in this environment, regardless of job title, is very interesting and can be very rewarding. However, it can also be very frustrating. There is a need to not only be technically competent, but also to be able to work with a variety of types

Table 2.1 Waterfall model of software development with systems personnel

Stage	Personnel
System feasibility analysis	Systems analysts , users, finance
Software plans and requirements	Systems analysts , users
Product design	Systems analysts
Detailed design	Systems analysts
Coding	Programmers, testing
Integration	Systems analysts, programmers, testers, systems administrators
Implementation	Systems administrators, testers, users
Operations and maintenance	Maintenance

Information systems personnel titles in bold.

of people. There is a need to work with users who have problems that computer systems hopefully will solve. There is also a need to work with technical people, from systems administrators to coders and testers.

Individual characteristics are the result of different personalities, of individuals with different demographic backgrounds as well as education and experience. These different people find themselves in different organizational roles, with different status levels. Individuals also have different psychological make-up, in the form of needs, interests, and goals. Within information systems project teams, a specific type of personality tends to predominate, focused on technical matters. But members of these project teams need to work with users, who tend to have psychological characteristics more common to the public, focused on dealing with people, with a broader organizational focus.

Being part of a **project team** can lead to conflict. The degree of conflict is expected to depend upon team size, heterogeneity, and leadership. The larger the team, the more likely for some conflicts to arise, although it can be possible for antagonists to avoid each other in larger teams. The more diverse the team, the more likely conflicts are to arise. Within information systems teams, there tends to be a common technical focus, except for those whose specialty is training. However, sometimes having too many shared interests can lead to conflict, as there can be different opinions about how to proceed technically, and individuals may feel that other people are not competent, whereas they may simply have a different technical approach. Team processes can play a role in conflict, through the ability to communicate efficiently, and through the relative degree of participation encouraged. Team history may well play a major role in conflict. Working together in the past is usually viewed as a positive factor in conflict but can be the worst case if personalities clashed in previous project efforts.

Characteristics of projects can also lead to greater levels of conflict. A major factor is time pressure. Some people thrive under pressure, others react adversely. Projects usually involve pressure. They also often involve constraints, in that the ideal level of resources may not be available. Some people find that unacceptable and refuse to proceed with project efforts. Others cope somehow. The more flexible approach is usually more productive. Other factors include success criteria (expectations) and top management support. Without top support, projects tend to be starved for resources, making the work environment difficult.

Organizational characteristics can also be important. Matrix organizations (to be discussed in Chapter 3) involve a very hectic kind of environment, highly suitable for information systems projects. However, it may not be suitable for specific individuals. In matrix organizations, there is a need for actors to quickly adapt to new circumstances, and to deal with new people, new responsibilities, and in general, be able to prosper in a highly dynamic environment. If this is not the type of working climate desired by a specific individual, that individual should find one of the many other professional fields to work in.

Interpersonal conflict can manifest itself in at least four levels. **Interdependence** is the process of people working together, realizing that their success depends on the cooperation of others. This is the form of team unity most desirable in an information systems project team. There will always be disagreements that arise. This

is healthy, and reflects thought and effort applied by team members. (If there were no disagreements, this is an indication that some of the team members do not care.) The process of interpersonal conflict becomes disruptive if interference results, from disagreements escalating into actions taken to force one position at the expense of another. A higher level of conflict arises when negative emotions arise, as in heated arguments.

There are several different management styles available for team leaders to cope with interpersonal conflict. These styles include problem solving, compromising, asserting, accommodating, and avoiding. The appropriate style depends upon circumstances. Only by understanding human behavior can team leaders learn which style is needed. This selection can be an important factor in project success, system success, individual performance, and effective organizational performance.

2.2 Conflict and Performance

Conflict management is essential in obtaining positive information systems project outcomes. Conflict can be useful in generating better solutions, but harmful behavior is counterproductive in any environment. The bottom line of this result is that it is better to avoid interpersonal conflict in the first place rather than rely upon project leadership ability to mitigate it. Features that might improve information systems project success include:

- 1. Project scope and objectives communicated to all of the project team**

Management often feels that once a project is officially adopted, everyone should automatically rally behind it. They therefore omit a detailed explanation of what the project is designed to do for the organization. This often leads to some resistance to the project by those who were not consulted in the decision, especially those who feel that their input would have been important.

A related issue is the scope of the project. If the project scope is not clear to everyone, those who make decisions related to the project will not be able to make the best decisions. Additionally, user requirements need to have assurance that they are feasible. This is easier to assess if clearly stated acceptance criteria are present. Every requirement should include a definition of what makes system performance acceptable.

- 2. Disseminate Business Rationale for Undertaking a Project**

The business reasons for adopting an information systems project usually are not disseminated. This is often the case because there are no sound business reasons for adopting such projects, especially those in time-constrained environments. (We will argue that business cases are very difficult, because they require estimation of impact that is nearly impossible accurately.)

- 3. Accurate Project Budgets**

In rushed projects, budgets may not even be created. Even if they are, it is very common for such budgets to overlook, or grossly underestimate, some key factors. Budgets, however, are important tools for project control.

4. **Project Support Present**

Rushed projects never have unqualified support. Some organizational members may continue to oppose such a project long after its adoption by top management. This opposition can manifest itself through open opposition, or simply by small means of uncooperative behavior to delay the project or to reduce its likelihood of succeeding.

5. **Firmly Establish Project Control**

Within organizations, there are always issues concerning control. Good project team unity is the best way to deal with such issues. In rush projects, personnel should be selected considering their ability to work under pressure. In information systems projects involving vendors, related issues often arise. Vendors have their own agenda, with the client organization's welfare rarely taking top priority. Control of vendors (and consultants) is key to project success where vendors (or consultants) are present.

6. **Rules not Changed During Execution of the Project**

A common feature of project environments, involving specialists assigned on short-term bases, is vanishing resources. Vanishing resources refers to assigned personnel who are often unavailable, especially during key periods. Project manager efforts to complain to the permanent managers of such resources often lead to hostility and greater lack of cooperation. Timely and accurate feedback throughout the project is a way to overcome this potential problem.

Political aspects of information systems projects can be very important for project success. This is complicated by the fact that there will always be those who feel that their own interests are furthered by project failure. Human aspects of information systems projects are very complex, and tax the ability of project participants to get along with other people, and to gain progress individually as well as for the benefit of the organization.

Another obvious key need in making a project work is communication. Without clear and accurate communication, project members cannot be expected to know what they are to do, and whom to contact for information.

2.3 Project Managers

It has been reported that over 30 percent of new software projects are canceled before completion, and that over half are more than 180 percent over budget. A commonly cited reason is poor project planning and management. At the same time, there is a shortage of qualified, large-scale project managers.

Project management involves getting work done through outsiders. We will see in Chapter 3 that projects often involve dual lines of authority. Project managers are not

in as powerful a position as managers in other forms of organization, and therefore they must rely on influence and persuasion to get people to work toward project ends. The survival of a project manager depends a great deal on the strength of the alliances that the manager can make with powerful stakeholders and the success in competing for resources within the firm. The project manager’s mission is to integrate diverse activities in a highly dynamic environment, to produce technical deliverables with a team whose members are temporarily assigned from different parts of the firm and therefore have other loyalties. This must be done within the constraints of a budget for cost and for time, and meeting quality specifications.

The project manager has to make decisions and provide a sense of direction for the project’s organization and serve as the hub for project communication. The ability to take on several roles is required. The project manager must be an evangelist, to keep everyone believing that the project will work. He or she must also be an entrepreneur, getting the necessary funds, facilities, and people required for project success. The project manager must also be a change agent, orchestrating diverse activities and facilitating these efforts to ensure project success.

The project manager has the responsibility of all managers, to get the job done on time, within budget, and satisfying specifications. The job also includes planning and organizing, dealing with groups representing the owner, and subcontractors. In addition to people skills, the project manager must understand enough technology to realize what is possible and what is not, and to recommend project termination if things are not working out. The project manager needs to be personable, to use a leadership style capable of motivating diverse people who do not work directly for the project manager. The project manager also needs to be able to understand budgeting.

2.3.1 Comparison: Functional and Project Managers

Project managers operate in a very turbulent environment. Table 2.2 shows that there are many differences between the jobs of a project manager and a functional organization manager.

Table 2.2 Comparison of managerial features

Functional manager	Project manager
Clear chain of authority Quasi-permanent relationships Can direct	Often operates in matrix structure, with low authority Temporary, shared relationships Often must motivate positively
Established organization	Developing and changing organization
Long-term relationships	Short-term relationships
Directs a small set of skills	Directs a diverse set of skills

The selection of a project manager should be based on a number of characteristics. Project managers need flexibility, leadership, confidence, and organizational skills. They should be generalists rather than specialists. They need to communicate well, and to be able to build trust and team spirit. They need general business skills, as well as technical understanding. Project management requires a well-rounded individual.

Team cooperation is critical in projects. Matrix organizations involve high levels of conflict, in large part due to the dual chain of authority found in matrix organizations. Project managers operating in that environment need to function without the authority often found in functional organizations. Within projects, compromise and dealing with conflict in a positive way are usually associated with reduced conflict intensity.

While not always given direct authority, the project manager has control over some resources. The primary source of authority, however, is often the respect gained from professional expertise, and sometimes charm and personality. Professional expertise can be gained from technical knowledge and administrative competence. Some have suggested that project managers try to appear powerful to workers, so that they maximize their influence.

Project managers can be found either inside or outside of the organization. From within the organization, someone with experience and the right specialties is often impossible to find. Therefore, project managers often need to be found from outside the organization. However, this is also problematic, because it takes time to establish alliances and to understand an organization's needs. But usually, people with better organizational skills and experience are available in the broader market outside of an organization.

2.4 Summary

Information systems projects are very important, providing valuable computer technology for organizations. There is a wide variety of such projects. Web projects tend to involve many similar projects of a short duration. This makes project management easier than the typical kind of information systems project, which may involve a range of time from a month to years and tends to involve more variety. The largest-scale information systems project is often the installation of an ERP system. This third type of project may be quite short if it simply involves the adoption of a vendor product. However, it can involve many years for global firms revamping their entire information systems support.

Human issues are very critical to the success of information systems projects. Interpersonal conflict may arise due to individual characteristics, team characteristics, project characteristics, or organizational characteristics. The role of each of these areas and how problems in each factor can impact projects was reviewed. Successful project completion requires careful management of each of these factors.

The impact of conflict on performance was reviewed. A little conflict is healthy, reducing the likelihood of complacency and urging us to do more and better things. However, conflict can quickly become counterproductive. It is especially difficult to

manage conflict in information systems projects, because there are so many diverse people involved, many of them specialists who deal better with computers than with people.

Projects bring together diverse specialists to work on them. This creates a need for effective communication within project teams. IS/IT projects often involve working with outside organizations, such as vendors and consultants. The need to communicate is the same, but crossing organizational lines can impose additional communications barriers. More formal communication can ensure better understanding in this case. There also is a need to communicate with the users that are to be the beneficiaries of IS/IT projects. Communication of all three types needs to be frequent to assure shared understanding in the highly dynamic project environment common to IS/IT projects.

The project manager is central to controlling the project. The project manager must integrate diverse elements to bring projects in on time, within cost, and with satisfactory performance. The project manager needs to be an individual capable of operating with diverse people, and capable of understanding both technical and administrative aspects of the project.

Chapter 3

Project Organization



Several different organizational forms are found in projects. This chapter reviews the primary types of organization used for projects, and the reasons they are designed that way. The chapter also discusses features desirable in project managers, and critical factors in project success.

The purpose of organization is to coordinate the efforts of many to accomplish goals. Organizational structure shows reporting relationships. Figure 3.1 shows a sample organizational set of relationships:

3.1 Alternate Organization Structures

Organization structures represent the hierarchical reporting and official communications networks within organizations. The management hierarchy consists of reporting relationships, an official chain of control or authority. This chain of authority deals with official activities, such as hiring, firing, and promotion. It also includes directing the activities of subordinates. Organizations can be grouped into major subdivisions based on a number of frameworks.

Informal organization can exist in parallel to the official organization structure. Informal organization consists of the network of personal contacts within the organization and may consist of cliques and groups of people who work well together, and who may not work well with those outside of their subgroup. In organizations with high levels of professionalism (such as in information systems work), informal networks can be very powerful and positive forces. Informal communication is socially motivated. It is very fast but is not necessarily thorough or dependable. Project managers, as we shall see, have relatively low levels of official authority.

There are three basic forms of project organization: functional, project, and matrix. Each form of organization has its own benefits, and each works well in certain types of environments. The appropriate organization structure depends on the goal of the

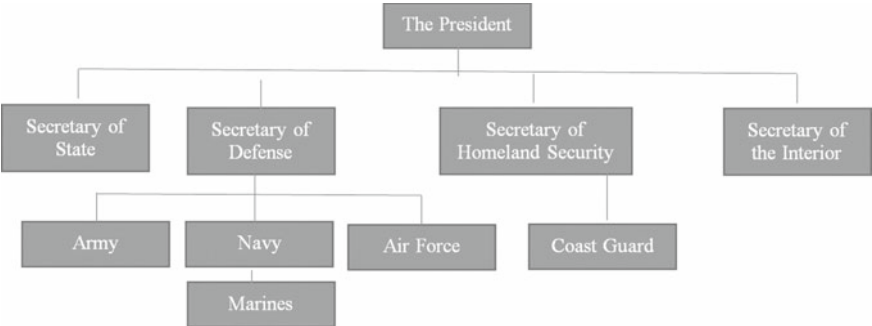


Fig. 3.1 Sample organization chart

organization, the type of work it is supposed to do, and the environment within which it operates.

3.1.1 Functional Organization

Functional differentiation organizes elements by specialization (see Fig. 3.2). This form of organization relies more on formal rules, procedures, and coordinated plans and budgets to control operations. In a project context, the project is divided into segments that are assigned to the appropriate functional groups, with each functional group head responsible for his or her segment of the project.

Functional organization works well in repetitive, stable environments. Organizational sub-elements are defined by activity or specialty function. All the accountants in the firm are collected in one location, where they work together. The same is true for the functions of finance, marketing, and MIS. Operations is a separate function, and operational suborganizations are often grouped by geographic location, another

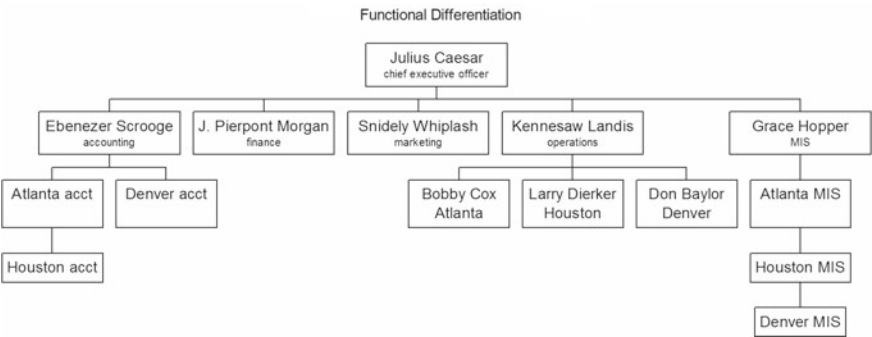


Fig. 3.2 Organizational elements by specialization

optional form of organization. The primary benefit of the functional form of organization is that specialists work together and can develop professional skills in the most efficient manner. Accountants focus on accounting problems and become very well trained in accountancy. On the other hand, they gain little exposure to the problems of other organizational elements, such as operations or marketing.

3.1.2 Project Organization

Pure project organization involves creating a separate, independent organization specifically for accomplishing a particular project. One type of example would be the Olympic Committees created to make each Olympic Games work. Once the project is complete, there is no reason for the organization to continue (the next round of games or elections will involve other locations and candidates). Figure 3.3 demonstrates how skills can be grouped by project.

The project center is linked to the parent organization to draw resources and personnel as needed. In the case of Olympic Committees, the permanent International Olympic Committee is available to provide continuity. In the case of committees for elections, there are permanent political parties to draw resources from. Task forces are often announced at a high level, such as the Task Force on Delivery System Reform and Health Information Technology (HITECH) where about \$30 billion was invested as part of the Patient Protection and Affordable Care Act. Similar task forces are often created for relocation operations.

Sometimes project organizations are stand-alone organizations. These are newly created organizations for a specific mission, drawn from several organizations. Examples are large-scale public works, like Hoover Dam, or the Dallas-Fort Worth International Airport. NASA space station development drew people from a number of organizations. Often stand-alone joint ventures are used in the construction business for very large projects beyond the scope of participating firms on their own.

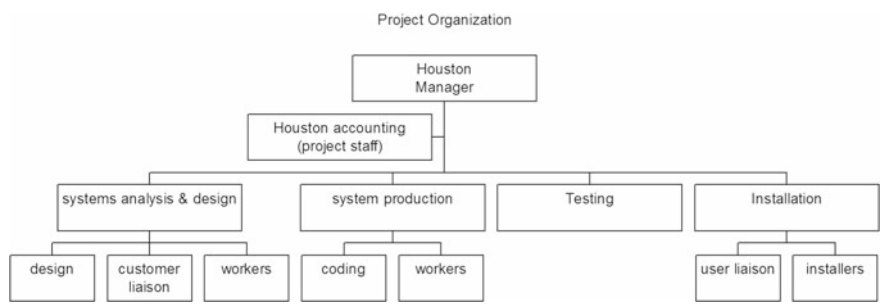


Fig. 3.3 Skills grouping

Partial project organizations also exist. In this form, the project manager is responsible for some activities, while other activities that are more support oriented, like accounting, remain with functional divisions. This is a common arrangement.

3.1.3 Matrix Organization

If an organization continually operates in a project mode (many organizations do in construction, in information systems, and in consulting), there is a need to quickly create large project groups. The matrix organization form is a grid-like structure of reporting and authority relationships overlaying those of a traditional functional organization (see Fig. 3.4). It is used within organizations that make more than minimal use of project teams or product groups. The improved coordination obtained from project organization is combined with the strengths for each specialty that are provided by functional forms of organization. The matrix form of organization was originally adopted by NASA and by the Department of Defense in the 1960s, when contracting practices required contractors to use project management. For each particular project, the contracting firm had to develop a project organization. PMBOK has unique terminology, referring to functional matrix organization as a weak matrix and project matrix form as a strong matrix. These terms are commonly used by those strong advocates of the Project Management Institute.

The key feature of matrix organization is multiple lines of authority. Specialists report to their functional managers with respect to issues involving their specialty, and report to their project managers for specific assignments. Functional specialists are assigned to the project, usually physically located wherever the project is being implemented. But these specialists depend for their personal career decisions on their permanent functional homes. The project accountant works with the project manager,

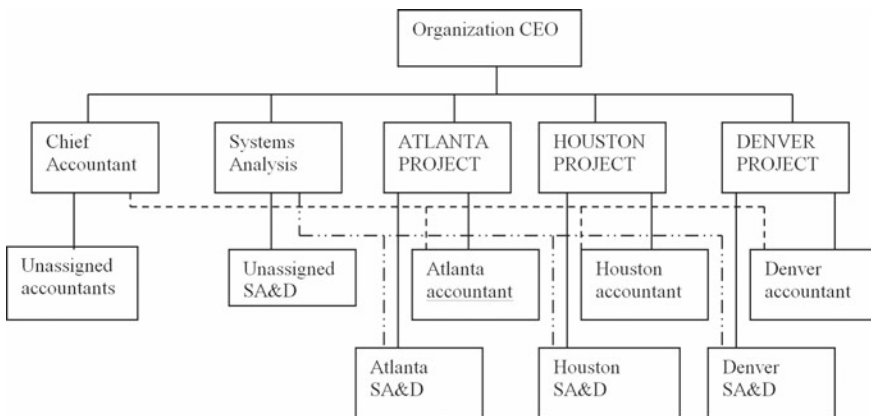


Fig. 3.4 Matrix organization

and the project accountant's job is to keep the project manager informed of the cost performance on the project. Since the accountant will work on several projects during his or her career, the project accountant's promotion and raises are often decided by the chief accountant at the accountant's permanent organization. Project managers have some input, but the chief accountant would be the project accountant's permanent supervisor. Once the project is completed, the project accountant would return to the organization's accounting office, where he or she would await the next project assignment, or else undertake professional development such as training, or maybe stay with the accounting office on a permanent basis. The same is true of the project engineer. It could be true for the project manager and production personnel as well, although these people often go into the open market to another organization when the project is completed.

Some problems are introduced by the matrix form of organization. The dual reporting structure creates a state of confusion for those who like high levels of structure. It is said that no man can serve two masters. There is also a military principle of unity of command. The matrix system, with two potential sources of direction, has been found to lead to conflict due to incompatible demands and priorities from two managers of a specific individual. Most people can cope with such mixed signals, but it causes distress in others. The ability of managers to compromise, and to deal with conflict, has proven very important in project management. While conflict can sometimes lead to improved performance, it needs to be managed carefully. Matrix forms of organizations call upon managers to do a great deal in a difficult environment.

The level of control used within a project organization should vary with the level of difficulty of the problems expected. If there are major technical issues, stronger centralized organization is merited, with a focus on the technical lines of organization. For instance, if a major engineering problem is expected to be critical to the project, engineering components would be centralized under a project engineer. On the other hand, if no major engineering problems were expected, there might not be a project engineer, but rather independent engineers assigned to each work group that needed one. This independence would allow focus on the tasks assigned to the work group rather than overall engineering coordination.

3.2 Comparison of Organization Structures in Projects

There are many variations to the three basic organization structures discussed so far in this chapter.

3.2.1 Task Forces

Task forces are temporary groupings of individuals created to solve a particular problem. The task force idea comes from the military, where combining different

types of units under a temporary leader for some specific mission is often practiced. In project organizations, individuals with different specialties are grouped together to develop a solution for a specific problem. Long-term task forces can be turned into permanent teams.

3.2.2 Hybrids

Functional matrix and project matrix structures are hybrids of the three basic forms presented above.

A functional matrix organizational structure is used when a project manager is restricted to coordinated functional group assignments. Functional managers are responsible for technical work assignments. The project manager in this case acts as a staff assistant with indirect authority to expedite and monitor project activities.

In a project matrix form of organization, the project manager has more direct authority to decide personnel and work assignments. The functional manager is responsible for providing resources and advisory support to projects.

In the balanced matrix organization, project managers and functional managers have roughly equal authority and responsibility for the project. The project manager typically decides what needs to be accomplished, while the functional managers are responsible for deciding how work will be done and by whom.

3.2.3 Levels of Project Organization

Projects involve a nontraditional form of organization. Traditional organizational design is much more rigid. While no organization is truly permanent, traditional organizational forms are designed on the assumption that they will continue into the foreseeable future. They are not very flexible, and they react slowly to change.

Projects, on the other hand, are organized with the understanding that they are temporary. Some projects can be very long in duration, such as the construction of a cathedral in the 11th to fourteenth centuries, taking hundreds of years. Other projects may last only weeks, and therefore have very temporary lives. Projects involve very high levels of uncertainty and change, so project organizations need to be flexible and adaptive. In the traditional form, people can enhance their professional training and development. Many organizations that contract to undertake projects typically adopt a functional form of organization for permanent assignments, drawing people for individual projects from their permanent assignments to temporarily work with other specialists on specific projects. This is true in construction, engineering, and large accounting firms specializing in consulting, including information system consulting. Careers with such organizations involve a great deal of relocation as individuals leave their home base for the duration of projects to which they are assigned. Between projects, the organizations may value their specialty skills enough to keep them on

Table 3.1 Comparative organizational forms

Form	Size	Duration	Complexity
Task force	Small to medium	Short to medium	Low to medium
Project team	All	All	Low to medium
Multiple project teams	Medium to large	All	Medium to high
Matrix	Medium to large	All	Medium to high

the payroll. Ultimately, however, new projects are needed in which these skills can be utilized.

Within project organizations, integrators are often used to facilitate communication. **Liaisons** are used to integrate two groups that are not part of the same organization. Liaisons are especially useful between the people who fund the project and the project team. Project expeditors or **coordinators** are individuals whose job is to make sure that something happens. These individuals have no authority, but are assigned to be on top of problems, so that they can inform project managers of the need for additional resources for specific activities.

3.2.4 Criteria for Selection

Table 3.1 outlines the conditions for which a particular organizational form is most often appropriate.

Other criteria that bear on the project type include uncertainty, cost and time criticality, and the uniqueness of the project. If the project involves high stakes, a matrix form or pure project form gives better control. If there are high levels of certainty, task forces and teams are appropriate because they involve less investment. If time and cost are not critical, task forms and teams are also appropriate. If the project is unique, a partial or full project form is appropriate.

3.3 Summary

Organizational structure is a means of achieving goals and responding to problems. Project personnel are often organized into project teams or matrix structures. These organizational forms are more flexible than functional structures that tend to be more bureaucratic. The positive and negative features of alternative organizational forms and their variants need to be understood by top management, so that they can select the organizational form most suitable for their situation. Understanding why these forms exist is also important to those working for them. The matrix form of organization is especially suitable for many projects for large organizations.

Chapter 4

Project Estimation



There are several different types of projects. Development of products can involve a great deal of uncertainty, as usually they involve new technology or materials. New products such as new airplanes or electric vehicles provide opportunities for investors to get in on the ground floor of spectacular successes, with corresponding risks of losing all. This is especially true in development of new drugs, such as something to cope with new diseases such as COVID-19. The high risks involved are often shared by the government in such cases involving national emergencies. That also is true for projects to develop new armament systems.

Engineering/construction projects are usually estimated by construction firms on a repetitive basis for the specific type of construction that firm does. They can develop extensive databases of productivity and costs, enabling estimation of the resources needed and time expected, which can be used to generate fairly accurate cost estimates. For large-scale projects, there often are consortiums developed, with extensive subcontracting, each subcontractor providing bids that can be collected by the general contractor.

In the field of software engineering, there are seemingly unlimited proposals for implementation of computer systems to aid organizations. Every time one of these proposals is adopted, it creates the need for an information systems project—to bring an application online.

This chapter will review methodologies related to project estimation, focusing on that used in software projects.

4.1 Software Project Estimation

Project management is one of the most important developing areas in information systems. It is difficult to bring an information systems project to completion on time, within budget, and meeting specifications. There are many citations of the scope of

this problem. A partner of KPMG, Peat Marwick, said that, based on a survey of 250 companies, some 30 percent of information systems projects exceeded the original budget and time frame by at least a factor of 2, or did not conform to specifications. A report issued by The Standish Group reports industry statistics every year, but these statistics are very stable. Only around 15 percent come in on time and within budget. For large companies, the success rate is closer to 10 percent. It was also reported that only about 40 percent of planned features and functions end up in the final version of the software. Over half of the software development projects initiated by large companies cost nearly 200 percent more than the original estimation. Yet information systems projects offer great value for companies.

To develop systems efficiently, a methodology needs to be applied. Organizations use a wide variety of methodologies. The purpose of this chapter is to provide an overview of traditional systems development methodologies. It is becoming more and more important to apply continuous improvement standards to the processes used within businesses. Standards such as ISO 12207 (disseminated originally in 2008) and Software Engineering Institute's Capability Maturity Model (CMM) focus on improving the processes of software development. Software development organizations are being required by some user organizations to certify compliance with these standards. Compliance is also very beneficial to software-producing organizations. We will review these standards. Types of information systems projects and their features are examined, followed by a broad framework of the systems development approach.

Requirements analysis identifies the data and information needed to automate some organizational task and to support the achievement of organizational objectives. Many information systems failures have been attributed to a lack of clear and specific information systems requirements. Accurate identification of requirements early in the process has been reported to result in successful systems and lower costs for error correction.

Requirements analysis can consist of four processes:

- conceptual design
- logical design
- validation
- formal specification

Conceptual design is the process of developing a model of what the system should do. Critical factors such as the implementation environment, organization goals and policies, product and service flows, and anticipated problems need to be identified.

The **logical design** process is where the strengths and weaknesses of the conceptual design are assessed. Organizational and technological factors both should be considered. Organizational factors include resources required, organizational politics, and priorities. Organizational factors are better understood by viewing projects as systems collecting interacting components with a common purpose. Technological factors refer to existing systems capabilities, the availability of needed data, and the availability of needed personnel. The logical design process is a system design considering the organization's strengths and weaknesses.

Validation is a process meant to ensure that a valid set of requirements has been developed. Features that need to be considered in the validation process include data entry methods, system outputs, and other impacts of the proposed project on the overall system.

The **formal specification** is the result of requirements analysis. The statement of work and business case can be used to document the project in terms of its intended mission. An ideal formal specification clearly specifies a complete set of information requirements, to include inputs and outputs, and what these elements are to do.

This process then culminates in the project sponsors creating a project **charter**, a formal authorization for work to proceed. The formal specification focuses on project technical matters. The project charter can be used for the organizational budgetary process to enable activity to proceed within proper control of authorized stakeholders. The charter includes a detailed description of the project in the form of project **scope**, which defines what will be included in the project.

4.2 Overview of Analysis and Design Methods

There are a number of approaches that traditionally have been used to analyze and design information systems projects. The laissez faire “code-and-fix” model, where code is written and problems are fixed, resulted in many problems when it came to implementation, and fixing was often expensive.

4.2.1 *The Waterfall Model*

The waterfall model recognizes feedback loops between stages of software development to minimize rework, as well as incorporating prototyping to more thoroughly understand new applications. The waterfall model (named because each step follows its predecessor in sequence) consists of the following stages, each of which can involve reversion to the prior stage if attempts at validation uncover problems. The waterfall model (Table 4.1) has the advantages of encouraging planning before design and decomposes system development into sub-goals with milestones corresponding to the completion of intermediate products. This allows project managers to track project progress more accurately. This aid in developing project structure.

The list shown is for a software life cycle product. Variations in the stage labels are used for different types of projects, such as acquisition of software, implementation of a vendor system, or other kinds of projects. Each stage involves a test, either validation or verification. Validation is the process of evaluating software to ensure compliance with specification requirements (Is this the right product?). Verification is the process of determining whether the software component functions correctly (Is the product built right?).

Table 4.1 The waterfall model of software life cycle stages

Stage	Feedback determinant
System feasibility	Validation
Software plans and requirements	Validation
Product design	Verification
Detailed design	Verification
Code	Unit test
Integration	Product verification
Implementation	System test
Operations and maintenance	Revalidation

In the original waterfall model, problems accumulated over stages and were not noticed until project completion, resulting in very expensive code. User needs were often not met, resulting in a rejection of products after they were built. Therefore, feedback loops were added, along with prototyping to catch problems early. The waterfall model does not allow a rapid response to the pervasiveness of change in information systems projects. The orderly sequence of activities in the waterfall model does not accommodate new developments. Some systems, especially those involving higher levels of uncertainty and with less investment at stake, are often designed and built using more flexible development methods, such as rapid prototyping, object-oriented process, or rapid application development.

Prototyping is the process of developing a small working model of a program component or system with the intent of seeing what it can do. Thus, it is a learning device, appropriate especially for users who are not sure of what they want from a system. Prototyping can be a valuable part of the waterfall approach.

4.2.2 Prototyping

When dealing with systems that involve beneficial features that are difficult to both predict and price, the systems development approach has proven ineffective. The hard, clear dollar benefits rarely are sufficient to justify adopting the system.

A prototyping approach involves building a small-scale mock-up system, allowing the user to try it out. The user could then ask for modifications based on a better idea of what the system could do. Prototyping is a much less thoroughly planned approach but is often appropriate for applications with low investment and low structure. This can result in much lower development cost and time, especially when there is a lack of clarity on what a user wants from a system.

Prototyping is very useful in leading to a greater understanding of project requirements. It has also been found to improve design effectiveness, because users are directly integrated into the design process in a manner that they clearly understand. Prototyping is not appropriate for all types of system developments. However, for

smaller scale systems, prototyping is a very effective means of demonstrating what proposed systems would be able to do.

4.2.3 Agile Software Development

A response to the poor performance record of information systems with respect to time, budget, and functionality, some in the software industry have developed collaborative techniques to develop software much faster and closer to user requirements. These techniques are known as agile methods and include a number of versions. However, the common bases for these techniques include working closely with users throughout the project, iteratively developing software versions in the spirit of prototyping, demonstrating these prototypes to users to identify required changes in direction, and avoiding heavy documentation in favor of getting software developed faster.

Agile methods have proven very effective when applied to small projects. They also work (with care) for larger projects, although effort is needed to get teams to work well together. The best case is when a small, self-organized, and motivated team located in the same location works with a small number of on-site customers. There are even cases when teams have worked in a geographically separated environment, although synchronization of efforts is required.

The economy in recent years has had a major impact on the software industry. Because of the increased importance of bringing software products to the market faster, a revision of traditional methods has been apparent. Agile methods are techniques of software development designed to deliver these products faster, with more functionality necessary for users.

There are many different techniques that have been used in this effort to attain agile programming. These include collaborative techniques (pair programming, having customers work on-site as team members). This amounts to is working quite closely with customers to ensure that the software products developed are exactly what the customers needed. For instance, a common agile practice is to have customers write acceptance tests. This approach emphasizes that:

- Focus on individuals and their interactions is preferred over a focus on processes and tools.
- Delivering working software is more important than comprehensive documentation.
- Customer collaboration is sought rather than contract negotiation.
- Response to change is preferred to following plans.

The agile approach maximizes cooperation with users that has been found to be so critical to information systems project success. The principles behind the agile movement are:

- Highest priority is given to satisfy customers through early and continuous delivery of valuable software.
- Requirement changes are welcomed, even late in the development process. Agile processes harness change with the aim of improving customer competitive advantage.
- Working software is delivered frequently, from a couple of weeks to a couple of months, preferably to shorter timescales.
- Cooperative work between businesspeople and developers is needed daily.
- Projects are built around motivated individuals, who operate in a positive environment with all the support they need, and who are trusted to get the job done.
- Face-to-face conversation is relied upon as the most efficient and effective method of communication.
- The primary measure of progress is working software.
- Agile processes promote sustainable development.
- Technical excellence and good design are sought continuously, enhancing agility.
- Simplicity is essential.
- Self-organizing teams develop the best architecture, requirements, and designs.
- Teams reflect on ways to become more effective at regular intervals, and tune and adjust behavior accordingly.

The agile approach is often combined with other methodologies in hybrid forms.

4.2.4 Other Options for Systems Development

Many projects are developed from scratch, applying conventional systems development approaches. In the following iterative life cycle approaches, tasks are performed on a crash basis to meet tighter time goals.

- **Component assembly projects:** typically, object-oriented modules for use in a variety of other applications
- **Rapid application development:** techniques to compress the life cycle, to include computer-aided software engineering (CASE) and Joint Application Development (JAD). CASE tools involve a degree of reuse of previously developed systems. To complete projects on an accelerated timescale, project objectives and scope need to be identified early, and joint development of user requirements must be effective. Group systems may be of use in this type of application. Care must be taken in assigning team members on a full-time basis, and heavy user participation is very important. Rapid application development can lead to reduced development time, lower development cost, greater user and employee satisfaction, and lower ultimate maintenance costs. However, it is appropriate for small projects, and not those involving significant levels of complexity.

4.3 Software Estimation Methods

Estimation is a challenging activity in any project domain. Here, we demonstrate with information systems project methods, which include three broad types of estimation that appear with some similarity in other project types. These approaches are:

1. to base estimates on a key metric.
2. to identify blocks of work to be accomplished and aggregate on that basis, and
3. to apply a form of regression over multiple variables.

In information systems projects, the most used approaches are based on source lines of code (SLOC, or LOC), type 1 estimation. The number of lines of code seems to be of less and less importance as productivity tools are developed and more productive languages are used. An example of approach 2 is to base estimation on relevant project features as in FP analysis, focusing on the functions the proposed system needs to accomplish. However, most research has not found any improvement in estimation through use of FP analysis in practice. SLOC is still widely used and is the basis of the constructive cost model (COCOMO) system covered below, a nonlinear regression (type 3). Although these systems have noted flaws, estimating methods significantly better have not been widely reported.

In construction, similar approaches are used. Probably the most accurate is type 2, based on estimator experience. In information systems projects, the most important variable is person-hours. In construction, you need to also add equipment hours by type of equipment and materials.

The software production cycle can be described in many ways. A simple view consists of the following major elements: design and development, production, testing, installation, and maintenance. The amount of time required to develop a set of code is a function of its size. The following demonstration of the source lines of code (SLOC or LOC) and FP methods demonstrates this point.

4.3.1 *Lines of Code*

Both LOC and FP methods begin with the scientifically sound approach of gathering historical records of experiences of past projects. This historical data is the basis for identifying the relationship between key measures of importance (such as the person-months of effort and dollars expended) and other factors of importance (such as the pages of documentation generated, errors encountered, system defects, and people assigned). An implementation of the LOC approach (Table 4.2) uses the hypothetical average project numbers given to demonstrate:

Estimation of simple lines of code amounts to estimating how many thousand lines of code are expected and multiplying that number by each of the averages per KLOC (thousand lines of code). When a new project is encountered, an estimate is made of the lines of code that the project will require. For instance, if a new project

Table 4.2 Lines of code operation

Averages of past projects		Effort	\$(000)	pp. Documentation	Errors	Defects	People
LOC	31,562	40 mos	512	1303	189	36	7
Avg per KLOC		1.267	16.222	41.284	5.988	1.141	0.222

Table 4.3 Application of LOC

Effort	1.267×10 thousand LOC	=	13 person-months
Budget	$16.222 \times$ “	=	\$162,220
Documentation	$41.284 \times$ “	=	413 pages
Errors	$5.988 \times$ “	=	60
Defects	$1.141 \times$ “	=	11
People	$0.222 \times$ “	=	2 people

is estimated to involve 10,000 lines of code, estimates of these measures would be as presented in Table 4.3:

This approach is admittedly rough, but provides a very easy way to implement estimation method. Although gathering the data is time consuming, once it is obtained, it is very quick. It will be more accurate the more appropriate the data is, of course. Ideally, firms could build their database in categories by type of work, as averaging radically different projects together will lead to obvious inaccuracies. Another limitation is that it takes time to generate a database of historical project results. As with any statistical approach, the more data, the greater the confidence. However, the older the data, the less likely it is appropriate to current operations. Additionally, the model as demonstrated is totally linear, while project size may have different impacts at different size levels. More refined methods have been developed.

4.3.2 Function Point Analysis

Albrecht’s Function Point Analysis Method supplements these project size bases of estimation. The aims of this approach are to provide a consistent measure meaningful to the end user with rules that are easy to apply. The method can be used to estimate cost and time based on requirement specifications and is independent of the technology used.

The FP method works in a very similar way to lines of code, except that the basis for estimation is FP (reflecting, in essence, work to do). The original FP method involved counts of the number of activities in five categories (user inputs, user outputs, user inquiries, files accessed, and external interfaces). The number of functions is

counted by complexity level (low, average, or high) for each factor and multiplied as in Table 4.4:

The count-total is the sum of the product column. To demonstrate a hypothetical software proposal, see Fig. 4.1 for a situation involving development of a piece of software.

Table 4.4 Function point count calculations

Measurement parameter	Simple		Average		High		Product
Number of user inputs	_____	× 3	+ _____	× 4	+ _____	× 6	= _____
Number of user outputs	_____	× 4	+ _____	× 5	+ _____	× 7	= _____
Number of user inquiries	_____	× 3	+ _____	× 4	+ _____	× 6	= _____
Number of files	_____	× 7	+ _____	× 10	+ _____	× 15	= _____
Number of external interfaces	_____	× 5	+ _____	× 7	+ _____	× 10	= _____

Demonstration of Count-Total Calculation

A bank accounts record system with the following inputs:

40 user inputs	classified as simple in complexity
4 user outputs	classified as average in complexity
18 possible user inquiries	classified as simple in complexity
32 files accessed	classified as average in complexity
4 external interfaces	classified as high in complexity

:

Measurement parameter	Simple		Average		High		Product
Number of user inputs	<u> 40 </u>	× 3	+ _____	× 4	+ _____	× 6	= <u> 120 </u>
Number of user outputs	_____	× 4	+ <u> 4 </u>	× 5	+ _____	× 7	= <u> 20 </u>
Number of user inquiries	<u> 18 </u>	× 3	+ _____	× 4	+ _____	× 6	= <u> 54 </u>
Number of files	_____	× 7	+ <u>32 </u>	× 10	+ _____	× 15	= <u>320 </u>
Number of external interfaces	_____	× 5	+ _____	× 7	+ <u> 4 </u>	× 10	= <u> 40 </u>
TOTAL							554

Fig. 4.1 Count-total calculation

4.4 Demonstration of Count-Total Calculation

A bank accounts record system with the following inputs:

40 user inputs	Classified as simple in complexity
4 user outputs	Classified as average in complexity
18 possible user inquiries	Classified as simple in complexity
32 files accessed	Classified as average in complexity
4 external interfaces	Classified as high in complexity

The next step is to calculate F_i , which involves adding the ratings over the following 14 factors on a 0 to 5 scale, with 0 representing no impact, 1 incidental, 2 moderate, 3 average, 4 significant, and 5 essential impacts. The calculation, with a hypothetical set of impacts, is shown in Table 4.5:

FPs are then calculated by the formula:

$$FP = \text{count-total} \times [0.65 + 0.01 \times \sum F_i]$$

In the hypothetical example, estimation of effort would be:

$$FP = 554 \times [0.65 + 0.01 \times 34] = 548.46$$

Table 4.5 Function point calculation

			0–5 scale
F1	Does the system require reliable backup and recovery?	Significant	4
F2	Are data communications required?	Moderate	2
F3	Are there distributed processing functions?	Average	3
F4	Is performance critical?	Significant	4
F5	Will system run in existing, heavily utilized environment?	Essential	5
F6	Does the system require online data entry?	Moderate	2
F7	Does online data entry require input on multiple screens?	Incidental	1
F8	Are the master files updated online?	No influence	0
F9	Are the inputs, outputs, files, or inquiries complex?	Average	3
F10	Is the internal processing complex?	Incidental	1
F11	Is the code designed to be reusable?	Average	3
F12	Are conversion and installation included in the design?	Average	3
F13	Is the system designed for multiple installations across units?	No influence	0
F14	Is application designed to facilitate change and ease of use?	Average	3
		TOTAL F	34

Table 4.6 Function point estimation

Averages of past projects	Effort	\$(000)	pp. doc	Errors	Defects	People
FP = 1523	40 mos	512	1303	189	36	7
Average per FP (divide by 1523)	0.0263	0.3362	0.8555	0.1241	0.02364	0.0046
Multiply by new FP (548.46)	14.4	184	469	68	13	3

Then the estimation of required resources (Table 4.6) would be similar to that for LOC:

Although lines of code have proven useful and the FP method is also useful and widely used, estimation of software project effort continues to be a difficult task. Many other approaches continue to be generated. In part, new developments are needed because the software project environment is changing, with far less reliance on actual coding (the most predictable part of a software project).

The next model applies logarithmic regression on past data to more accurately reflect the time required to accomplish code development. COCOMO reflects the impact of learning, in that as the project progresses, programmers and developers are expected to gain in the rate of productivity.

4.4.1 Constructive Cost Model

Constructive cost models (COCOMO) for estimating the effort required for developing software have been promulgated. The basic COCOMO model computes software development effort on the basis of program size. An intermediate model also considers a set of cost drivers reflecting specific product, hardware, personnel, and project characteristics. The advanced model breaks out cost driver impact on each step of the software engineering process.

The first step is to identify the effort adjustment factor (EAF). This consists of 15 cost drivers (CD) which are shown in Table 4.7 reflecting the degree of effort from normal.

$$EAC = \prod_{i=1}^{15} CD_i$$

For purposes of demonstration, assume all ratings are nominal except for low analyst capability and low use of software code. *EAC* will then be multiplying thirteen 1.0 s by 1.19 for analyst capability and 1.10 for use of software code, yielding an *EAC* of 1.309.

For relatively small, simple software projects built by small teams with good experience, COCOMO formulas for person-months of effort and development time in chronological months are:

Table 4.7 COCOMO effort adjustment factor cost drivers

Factor	Very Low	Low	Nominal	High	Very High	Extra High
<i>Product attributes</i>						
Required software reliability	0.75	0.88	1.00	1.15	1.40	
Size of application database		0.94	1.00	1.08	1.16	
Complexity of the product	0.70	0.85	1.00	1.15	1.30	1.65
<i>Hardware attributes</i>						
Run-time performance constraints			1.00	1.11	1.30	1.66
Memory constraints			1.00	1.06	1.21	1.56
Virtual machine environment volatility		0.87	1.00	1.15	1.30	
Required turnabout time		0.87	1.00	1.07	1.15	
<i>Personnel attributes</i>						
Analyst capability	1.46	1.19	1.00	0.86	0.71	
Applications experience	1.29	1.13	1.00	0.91	0.82	
Software engineer capability	1.42	1.17	1.00	0.86	0.70	
Virtual machine experience	1.21	1.10	1.00	0.90		
Programming language experience	1.14	1.07	1.00	0.95		
<i>Project attributes</i>						
Use of software engineering methods	1.24	1.10	1.00	0.91	0.82	
Use of software code	1.24	1.10	1.00	0.91	0.83	
Required development schedule	1.23	1.08	1.00	1.04	1.10	

$$\text{Person-months} = 2.4 \times \text{KLOC}^{1.05} \times \text{EAC} = \text{E for effort}$$

$$\text{Duration (months)} = 2.5 \times \text{E}^{0.38}$$

Thus, for a job of this type involving 10,000 lines of code,

$$\text{Person-months} = 2.4 \times 10^{1.05} \times 1.309 = 35.25 \text{ months}$$

$$\text{Duration} = 2.5 \times 35.25^{0.38} = 9.68 \text{ months}$$

This implies a large staff of 3.64 (say 4) coders.

For software projects of intermediate size and complexity, built by teams with mixed experience and facing more rigid than average requirements, the formulas are:

$$\text{Person-months} = 3.0 \times \text{KLOC}^{1.12} \times \text{EAC} = E$$

$$\text{Duration (months)} = 2.5 \times E^{0.35}$$

Therefore, a job of this type involving 10,000 lines of code would have the following estimates:

$$\text{Person-months} = 3.0 \times 10^{1.12} \times 1.309 = 51.77 \text{ months}$$

$$\text{Duration} = 2.5 \times 51.77^{0.35} = 9.95 \text{ months}$$

This entails a staff of 5.2, or 6 coders.

Finally, for a software project built under rigid conditions, the formulas are:

$$\text{Person-months} = 3.6 \times \text{KLOC}^{1.2} \times \text{EAC} = E$$

$$\text{Duration (months)} = 2.5 \times E^{0.32}$$

A job involving 100,000 lines of code under these conditions would be estimated:

$$\text{Person-months} = 3.6 \times 10^{1.2} \times 1.309 = 74.69 \text{ months}$$

$$\text{Duration} = 2.5 \times 74.69^{0.32} = 9.94 \text{ months}$$

This indicates a staff of 7.5, or 8 coders.

There are variations in COCOMO. Companies could generate their own using regression on their productivity data. There are many causes of productivity loss. For one thing, the amount of time applied by humans to actual coding is only about 20 h per week, owing to machine downtime, emergency diversion, meetings, paperwork, sick time, and many other factors. A great deal of this waste, except for machine downtime, is recovered by using automated tools.

Interactions are activities involving coordination with others. The more interactions, the less productivity. High-level languages increase productivity by almost eliminating the programming component, but the more unpredictable activities still need to be accomplished by the project team.

4.5 Capability Maturity Model

Quality improvement has been an area of major development in the operations management area. One of the key ideas involved is that of continuous improvement—developing a philosophy of business in which the way in which work is

done (processes) is continuously reviewed with the intent of improving them. This emphasis on the quality process has been widely implemented in the United States in the annual Baldrige Award competition, as well as the European Quality Award Assessment Model. Two other programs which have had a major impact in the software development industry include ISO 9000 (European standards for production and management processes) and the Software Engineering Institute’s CMM.

The Software Engineering Institute’s **Capability Maturity Model** includes five levels of maturity. These are shown in Table 4.8, along with actions generally required to move from one level to another. Organizations that have done nothing to improve their software development methods are assigned to level (1) and are considered to involve high levels of risk and a focus on survival.

Moving from the initial level (1) to level (2) requires instilling basic discipline in people management activities. This is accomplished through adopting software management approaches.

Moving from level (2) to level (3) requires that organizations identify the primary competencies required in processes, and taking action to make sure that the people assigned to these activities have these competencies. This can be done through training or hiring. Processes are defined and institutionalized in level (3) organizations.

Moving from level (3) to level (4) requires the quantitative management of organizational growth in human management capabilities, and establishment of competency-based teams. Process measurement and analysis needs to be implemented.

Table 4.8 Capability maturity model features

Level	Features	Key processes
1. Initial	Chaotic	Survival
2. Repeatable	Individual control	Software configuration management Software quality assurance Software subcontract management Software project tracking and oversight Software project planning Requirements management
3. Defined	Institutionalized process	Peer reviews Intergroup coordination Software product engineering Training program Organization process definition Organization process focus
4. Managed	Processes measured	Quality management Process measurement and analysis
5. Optimized	Feedback for improvement	Process change management Innovation involving technology Defect prevention

Table 4.9 Capability Maturity Model Integration (CMMI) effectiveness

Level	Development cost \$ million	Development time Months	Product quality Defects/kLOC	LOC/h
1	33	40	9	1
2	15	32	3	3
3	7	25	1	5
4	3	19	0.3	8
5	1	16	0.1	12

Moving to the highest level requires the continuous improvement of methods adopted to develop personal and organizational process abilities. In this state, greater technology innovation is expected by some. All expect better defect prevention.

Projects accomplished by organizations at various levels for a typical software project with 500,000 lines of code (500 kLOC) have been studied. The results dramatically indicate the benefits of attaining higher levels of capability maturity, as shown in Table 4.9:

Adopting capability maturity principles pays. Level 1 projects are expected to take far longer (thus costing far more). They also have much inferior quality as measured by defects. Implementing the CMM requires significant investment, effort, and discipline. However, it is clearly better than the ad hoc state of doing nothing.

4.6 Systems Development Approach

The systems development approach provides a rational way to evaluate information systems projects. The systems development approach is based on a complete life cycle analysis. Project proposals evaluated by this approach are measured on cost, time, and performance. **Cost** is concerned with resources being spent as expected. Accurate budgets and benefit estimates are needed to accurately evaluate proposals. They also provide valuable control mechanisms during project implementation. **Time** is a critical variable in projects. First, it is highly correlated with cost. Second, delayed benefits are worth much less than early benefits, due to the time value of money. Keeping a project on schedule is a major challenge. **Performance** is a critical third measure. Projects need to perform to specifications and user requirements.

Later, when the project is being implemented, these measures are critically important. Performance needs to be ensured through quality testing. Time is a major means of evaluating how well the project is progressing. We will later look at Gantt charts as a means of planning the time dimension of projects. Cost budgets are also critically important in controlling project implementation. Because of the high levels of uncertainty inherent in projects, management must closely monitor project progress, and be prepared to shift resources and replan activities as required.

System life cycles consist (in broad terms) of the activities of specification, design, coding, data conversion, testing, and implementation. These activities tend to be serial, although data conversion can proceed in parallel with coding.

4.6.1 *Specification*

Projects start with somebody's problem. Computer systems are powerful tools that solve many problems. Therefore, there are many times when they offer improved means of doing things.

Once a proposed application is identified, a systems analyst who understands how to build such systems needs to be obtained to plan a solution. The systems analyst needs to start by talking to the people that proposed the application to find out what they want, and to the people that are budgeting the project to find out the constraints in terms of costs that need to be considered. It is best to discuss the proposed project with all groups affected, so that something is not designed that will be counterproductive or create unexpected problems. After these interviews, a clearer statement of the problem should be developed. An effective way to proceed is to identify alternate solutions, and to determine the costs and expected performance features of each, so that the budgeting authority can select the best system possible. A **feasibility study** is a clear, concise statement of the problem, followed by a detailed formal description of the current system describing the problem. Adequate qualitative and quantitative information should be provided to determine whether the effort should be continued. The elements and components of the proposed system are identified.

The specification phase should provide a clear problem statement of what the system is intended to do, with a rudimentary idea of a systems solution. Once the initial authorization is obtained, the system is defined in greater detail. A **statement of work** specifies what is to be done. This needs to specify new system objectives and provide measures for the acceptability of the system upon completion. Performance objectives should not be constrained by the existing system. One approach that works well is to start with general objectives. Objectives should include sub-elements that are measurable. This phase of the systems development approach results in a comprehensive list of activities, along with their schedules, costs, and required resources. This includes hardware and software requirements. When presented with the costs of proposed systems, many are rejected. It is in the specification phase that most projects die.

4.6.2 *Design*

The design phase develops how software will meet requirements. One of the classic business analyses is the decision to **make or buy**. This decision is very pertinent to computer systems, because there are many vendors that produce and sell many

useful computer systems. Every organization has the option of buying products from vendors, or of building the product themselves. In general, buying products from vendors is a much lesser hassle. But you have to live with the features that the vendors put into the product. They make a living telling everyone that this is exactly what they need. In truth, it often isn't. Furthermore, even if it is, vendors may charge more than the product is worth. But building products in-house requires a lot more risk and time. If required expertise is not available, it may well be worth spending a little more on the vendor. Usually the vendor route is faster, and quite often it is cheaper. The problem usually lies with matching the existing system, and doing the job required.

Recently, it has become very popular to hire large portions of information processing, or to **outsource**. One purpose of outsourcing is downsizing. Many functions can be outsourced, including data center management, telecommunications, disaster recovery, and legacy systems maintenance. This avoids the need to waste scarce resources and can gain efficiency by hiring vendors with expertise. Outsourcing can also be used for the company's Internet operations. If plans are clearer, Internet functions that could be outsourced include connectivity, Web server hosting, firewall security, website development, and content development. These activities are complex, subject to change, and not particularly relevant to organizational core competencies. Outsourcing makes sense when a fast start-up is important, internal skills are lacking, and the vendor can provide strong features. Outsourcing for Internet operations is not as worthwhile if they are of strategic importance to the business, or requirements are ill-defined. Rarely is outsourcing used for all tasks that a project entails. A need for internal training to implement the system and to integrate it with the existing system will arise for some tasks.

If vendors or outside contractors are being considered, a **request for proposal** is required including the feasibility study and the plan for project development. The request for proposal states the user requirements in terms of system objectives, project scope, and performance specifications as well as constraints, especially in terms of time. It is necessary to develop a qualified bidders list of those with the ability to accomplish the work required.

The project team needs to be selected at this stage. The project manager should be selected, to suit certain characteristics, which we will discuss later. Team members should be drawn from functional areas. Functional managers should be sold on the project, so that good team members are obtained.

The output of the design stage is a detailed list of user requirements and system requirements. Tasks are broken down into work packages, and team members are given specific assignments. The project manager is responsible for setting up schedules, budgets, and controls. The output of the design stage is a task breakout, with each task scheduled by date. It is necessary to continue close coordination between the systems analyst, the ultimate user of the system, and the budget authority. A project never is completely designed, nor finally adopted, until it is complete. A very accurate understanding of the analyst, user, and owner is necessary to develop systems that are useful and cost-effective.

4.6.3 *Code (or Acquisition)*

Options, in-house or vendor, need to be evaluated. If in-house, the conventional phase of coding to implement the design is applied. If options including purchasing the products or services of a vendor are adopted, the term acquisition seems more appropriate. Whatever option is adopted, project sponsors should conduct a **business case** (financial analysis, which in projects should consider risk).

If outside vendors or contractors are being considered, selection of the bidder to build the project (or project components) involves some options. There are several bases for selecting a bidder. If the system quality has been thoroughly defined to the extent that every bidder has the knowledge and if each bidder has been screened to ensure their competence, selection on the basis of a low bid is usually used. This has obvious advantages, using the competitive system, to lower costs. However, selecting the low bidder has obvious risks if the bidder is not truly qualified. In fact, in some parts of Europe, the winning bidder is the one closest to the average bid, using the logic that they must know what they were asked to do. Regardless, there are other considerations besides the low price.

- Cost—need to ensure that it is within the allowable budget
- Feasibility—need to ensure that the bidder can actually do the project
- Experience—look at the bidder's record on similar projects
- Reputation—bidder's record with respect to quality work.

Oftentimes, if the bid is too high, the user can negotiate with the bidder. Ethics are involved if an open competitive bidding process was used. Those not selected may not have been treated fairly if the rules are changed after the initial bid. However, in the private sector, this is a legal way to proceed, and the owner and the bidder can work together to negotiate an acceptable agreement for both. Negotiation is appropriate especially when dealing with complex systems when it is attractive to share risks.

In-house systems development begins with the design stage, which involves converting specifications from the definition phase into plans. The proposed system is broken down into subsystems, components, and parts. All elements need to be checked for compatibility, as well as for their ability to meet specifications. Prototyping can be used if there is benefit from seeing what the system will look like.

The system is developed and whatever hardware and software is needed is procured. The system is constructed, and any code and interfaces that are needed are programmed. System testing is conducted concurrently with the assembly of the system to catch errors as quickly as possible. Development of training materials is often accomplished concurrently as well.

Production of a system follows analysis and design as it can involve a large group of diverse people, from programmers, people building user interfaces, people designing the database interfaces, people dealing with the users to design reports, and people to set up any required networks for multiple users. The project manager must consider availing people with the right skills at the right time, obtaining needed

facilities in terms of tools and places to work, realistic time estimates allowing for the appropriate level of uncertainty, and of course capital. Quality testing should be accomplished throughout production. It is a good practice to build systems in modules, and thoroughly test each component before adoption. If possible, it would be best to include the user in this testing.

4.6.4 Testing

Quality control is very important throughout the system development cycle. Each module needs to be checked before a block of work is considered complete. Organizations usually have independent testing groups assigned to review system components to ensure that they can deal with the expected workload.

4.6.5 Implementation

Once the system is satisfactorily built and tested, it is moved from the builder to the user. This requires that the system be installed and checked out to ensure that it does what was specified. Training the users with technical support available once the system is turned over to the user wraps up the project cycle. The user evaluates system performance and if any flaws are detected, the builder would fix them. Sound contractual agreements spelling out procedures before starting the project are very useful at this stage. Builders often provide maintenance support for systems, usually at a nominal fee. The operation phase includes system improvement.

4.7 Summary

Many systems and analysis methods have evolved to support information systems projects. The different methodologies discussed in this chapter all have important roles to play for software projects with specific environmental conditions. Each organization tends to adopt a particular methodology. It is not as important which methodology is adopted as it is that a methodology be adopted. It is also important that software-producing organizations adopt process standards. The SEI/CMM has proven to be dramatically effective at improving process quality and productivity.

Systems development needs to consider cost, time, and performance in addition to specific criteria of importance to a specific project. The activities by system stages were reviewed.

Chapter 5

Planning



The chapter demonstrates concepts using information systems project contexts. Other types of projects have similar features. For instance, in construction the human emphasis of information systems projects is less dominant, relying more on equipment and materials. Both contexts often rely on subcontracting in various forms. But the principles demonstrated are transferable. Two dominant estimating methods are demonstrated. In information systems, lines of code utilize some broad metric, whereas function point (FP) analysis focuses on types of activities. Regression has been used in information systems estimation, and also has been applied in construction, but care needs to be taken in both contexts to obtain accurate data if regression is applied.

5.1 Planning Process

All projects involve uncertainty in large part because they are first-time activities. This uncertainty can be reduced if similar projects have been undertaken by the organization. Information systems projects have high levels of uncertainty. The size of the project is usually not well understood until systems analysis has been completed. Most of the unexpected delays in these projects occurs during the latter stages of testing, late in the project. Systems builders tend to report projects as on time and budget targets until the last stages of testing. But at this point, most projects are found to require additional work, often substantial in scope.

Information systems projects have several unique factors that lead to high variability in project durations, including variances in the productivity across people for specific work, and in the technology used. Resources available are often not known at the beginning of a project. Many specific resources may be planned for use on a project during the planning stages, but months later other events may arise that make it impossible to use the best people on a project.

The planning process consists of a number of steps, including the following:

- Set objectives and requirements—determining requirements
- Specify work activities—identifying specific work to be done
- Plan project organization
- Develop the schedule
- Control points—establish control mechanisms to include budgeting resources

5.1.1 Set Objectives and Requirements

The first step is to determine requirements. Project objectives are set in the early stages of project proposals. Once these objectives are identified, the measures of accomplishment needed to identify successful completion of the project have to be established. These are the standards for each element of the project. In building a bridge, this would be the carrying capacity of the structure. In terms of information systems projects, this would be a statement of what the final project is ultimately supposed to do.

A **statement of work** can be viewed as a contract between the project organization and its clients (see Fig. 5.1). The statement of work can contain product descriptions, discussion of project constraints, schedule requirements, budget limits, and an explanation of the roles and responsibilities of project participants.

Constraints under which the system must be developed should be identified early in the project. These include time parameters, such as deadlines, and budget estimates. The project may have to be accomplished within existing skill levels. Levels of project complexity can be specified.

Statement of Work for Construction of a House

1. Project goal – built a house
2. Identify project phases
 - a. Design
 - b. Construction
 - c. Subcontracting
 - d. Turnover
3. The house is to be built to owner specifications
 - a. Four bedrooms, attached garage, stone masonry
 - b. A budget of \$350,000 is allowed
4. The house is to be available for occupancy no later than 31 May 2027
5. The construction manager is Fred Swensen, responsible for; the construction foreman is Herman Goertz
6. Any proposed changes will be priced by the builder and approved by them owner.

Fig. 5.1 Statement of work

5.1.2 Specify Work Activities

A **work breakdown structure** is a top-down hierarchical chart of tasks and subtasks required to complete the project (see Table 5.1). The work breakdown structure can be focused on a product, a function, or anything describing what needs to be accomplished. The work breakdown structure is hierarchical, in that different levels of detail can be described. The overall project consists of a set of major activities, or project sub elements. The schedule consists of a set of tasks, usually denoting work done by a specific worker or work group. This is usually the lowest level of project activity that is used for planning.

A **detailed task list** is a listing of unique work packages briefly describing work to be done. The detailed task list also includes the assignment of responsibilities by job title and predecessor relationships. Predecessor relationships identify the conditions required for tasks to begin. Resources required, deliverables, and estimated durations are also provided. Table 5.2 demonstrates a detailed task list for an example project. This information makes it possible to schedule and plan the project.

The work packages for tasks can include a summary of work to be done, predecessors (activities that must be completed before this task can be started), who is responsible, specifications for output, resources required, and deliverables. In Table 5.2, predecessors and durations are provided for those tasks that are not subdivided. Specifications or required output could be provided as needed.

This detailed task list is the basis for estimating, scheduling, and allocating resources. The time required to accomplish each task, as well as the resources required by task, are included. Identifying these numbers is the task of **estimation** at this stage. Accurate estimation is a very difficult task. The best way to increase accuracy is to rely on experience supplemented by careful study and record keeping. It is wise to build a database of norms based on past experiences for reference in estimating future projects. Any project environment involves difficulty in estimating, because, by their nature, projects involve dealing with new activities. This is especially true in the field of information systems because of the phenomenal rate of change in technology.

There is a natural human tendency to “pad” estimates of the time required to do things. If your professor asks you how long it will take you to write a ten-page paper, would you respond with the fastest time you have ever written such a paper?

Table 5.1 Work breakdown structure for housing project

Elements	Design	Construct	Subcontracts	Turnover
Deliverables	1.1 Architecture	2.1 Excavate	3.1 Plumbing	4.1 Inspection
	1.2 Permits	2.2 Concrete	3.2 Electrical	4.2 Walkthrough
		2.3 Lumber	3.3 HVAC	4.3 Close
		2.4 Masonry		
		2.5 Drywall		
		2.6 Paint		

Table 5.2 Detailed task list for housing project

Task	Activity	Predecessor	Duration	Resources
1.1	Architecture	None	15 days	Architect
1.2	Permits	1.1	3 days	Manager
2.1	Excavate	1.2	4 days	Crew
3.1.1	Underground Plumb	2.1	2 days	Plumber
2.2	Concrete	3.1.1	3 days	Crew
2.3.1	Lumber Frame	2.2	6 days	Crew
2.4	Masonry	2.3.1	7 days	Crew
3.1.2	Plumbing	2.2, 3.1.1	3 days	Plumber
2.5	Drywall	2.3.1	2 days	Crew
3.2	Electrical	2.3.1	4 days	Electrician
3.3	HVAC	2.5	1 day	HVAC
2.6	Paint	3.3	2 days	Painter
4.1	Inspection	3.1.2, 3.2, 3.3, 2.6	1 day	City, Manager
4.2	Walkthrough	4.1	1 day	Owner, Manager
4.3	Close	4.2	1 day	Bank, Owner

You would not even respond with the average time you have taken to write such a paper. You would most likely respond with an estimate that you felt was safe. If you ask a programmer how long it will take to write a specific piece of code, you will probably receive an estimate that the programmer expects to beat 80 or 90 percent of the time. The programmer’s supervisor will in turn probably add an inflationary factor to allow for contingencies, such as having to use a slower programmer. This continues up the chain of command until the final estimate is made. Everyone, even management, realizes that the estimate is padded. Management then adjusts for this padding by *cutting* the estimate drastically. What results is a wild guess that may or may not have much relationship to reality.

A due date can become a self-fulfilling prophecy. If an activity takes less time than the due date, work is usually slowed until the due date becomes accurate. Due dates can thus have a negative impact on project performance.

Those doing the work need to be convinced that accurate estimates are in their best interest (the ability of the firm to accurately estimate what is required to do work so that they can submit competitive yet realistic prices for project work). The focus should rather be on the chain of critical activities and those activities that might become critical. (The critical chain consists of those activities that cannot be delayed without delaying the project completion time.) This is accomplished by close management attention to make sure adequate resources are provided to critical activities, and frequent reporting of the need to finish this critical work to those who are doing it.

It is very easy for estimators to be overly optimistic. At the beginning of a project, it is natural to assume that everything will work as planned. But because of the need

to accomplish a wide variety of tasks, and their interrelationships, projects rarely proceed as scheduled. Rarely will things go faster than planned. This is because of the need to obtain approval of end users and the need for delivery of materials and other components.

5.1.3 Plan Project Organization

Organizing refers to identifying the roles in the project organization. A **responsibility matrix** is a way to allocate tasks to individuals by responsibility (see Table 5.3). Such a matrix can be invaluable in complex projects, enabling management to quickly identify who is responsible for each activity. This matrix can also be used when forming the project team, enabling identification of what needs to be done for comparison with the capabilities of candidates.

An **organization chart** shows the reporting relationships of all involved in the project. Figure 5.2 shows the organization for the project we have been demonstrating. The organization chart outlines the communication network and coordination patterns required within the project as well. Within matrix organizational forms, this organization chart should show both primary and secondary reporting

Table 5.3 Responsibility matrix building house for owner

Code	Task	Mgr	Arch	Fmn	Elec	Plumb	HVAC	Paint	Owner
1.1	Architecture		X						
1.2	Permits	X							
2.1	Excavate			X					
3.1.1	Underground plumb			X		X			
2.2	Concrete			X					
2.3.1	Lumber frame			X					
2.4	Masonry			X					
3.1.2	Plumbing					X			
2.5	Drywall			X					
3.2	Electrical				X				
3.3	HVAC						X		
2.6	Paint							X	
4.1	Inspection	X							
4.2	Walkthrough	X							X
4.3	Close								X

Key **Mgr** Construction manager, **Arch** Architect, **Fmn** Foreman, **Elec** Electrical sub, **Plumb** Plumbing sub, **HVAC** HVAC sub, **Paint** Painting sub, **Owner** House purchaser

relationships. These can include not only those temporarily assigned from functional elements to the project, but also external relationships, such as the project owner and vendors.

RACI (Responsible, Accountable, Consult, and Inform) charts are often used to clearly define roles and responsibilities.. Responsible individuals own the project element. For a given activity, no more than one individual should be assigned responsibility. Accountable people are given the task of completing the activity, in support of the responsible individual. Consulting personnel are those who have the expertise needed to accomplish the project and who are available to support the responsible and accountable people. Inform people are those who need to know project status. Table 5.4 displays a RACI chart for the project outlined in Table 5.3:

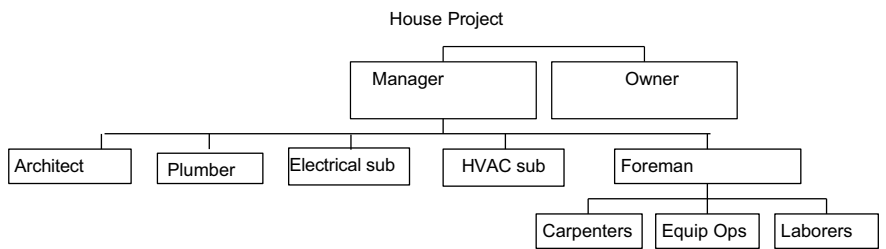


Fig. 5.2 Organization chart

Table 5.4 RACI chart for house project

Code	Task	Mgr	Arch	Fmn	Elec	Plumb	HVAC	Paint	Owner
1.1	Architecture	C	R						I
1.2	Permits	R		I					I
2.1	Excavate	I		R					
3.1.1	Underground plumb	I		C		R			
2.2	Concrete	I		R					
2.3.1	Lumber frame	I		R					
2.4	Masonry	I		R					
3.1.2	Plumbing	I		C		R			
2.5	Drywall	I		R					
3.2	Electrical	I		C	R				
3.3	HVAC	I		C			R		
2.6	Paint	I		C				R	
4.1	Inspection	R		A					I
4.2	Walkthrough	R		A					C
4.3	Close	C							R

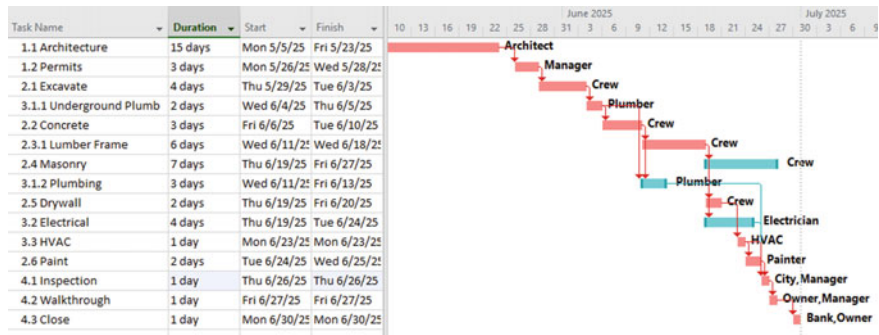


Fig. 5.3 Project schedule

5.1.4 Develop the Schedule

Schedules consist of the sequence of tasks from the work breakdown structure, along with their start and completion dates, and relationships to other tasks (see Fig. 5.3). Schedules include not only time estimates but also resource requirements. Microsoft Project allows display of resources to the right of the bar, representing the planned activity schedule. Schedules give everyone an understandable plan and impose structure on the project and should therefore be widely distributed among members of the project team.

Figure 5.3 displays the sequence of activities. Those activities with slack are 2.3.1 Lumber Frame (1 day), Plumber (8 days), and Electrical (1 day). All other activities are critical. Schedules are the basis for resource allocation and estimated cost of monitoring and control purposes. The focus is on identifying when activities (tasks) are completed so that work can proceed on the following activities.

5.1.5 Control Points

Control points include milestones, checkpoint reviews, status review meetings, and staff meetings. When planning the project, control mechanisms should also be included. Because of the existence of interrelated activities, **milestone** events are often used at the end of phases or groupings of activities resulting in the completion of project subcomponents. Meetings are an important control device, providing a way to keep each element of the organization informed of what is going on. Milestone activities have 0 duration.

There are three types of meetings that are often useful in information systems projects. **Checkpoint reviews** are held at the conclusion of each phase to determine whether or not to proceed with the rest of the project. In information system projects, this is the appropriate time to test the performance of the completed component.

If its performance is less than planned, it may be necessary to change the rest of the project, and possibly even cancel it. **Status review meetings** are used to gather cost, quality, and schedule information. These give project management the measures needed to control the project. Finally, **staff meetings** are regularly held to maintain communication.

The primary purpose of budgets is to give management control over expenditures. A key activity is tracking actual expenditures relative to the budget. Variance reports are often used to focus on those project elements that are in the greatest amount of difficulty. When such problems are encountered, plans need to be developed to adjust to the new circumstances.

5.2 Planning for Change

As we stated at the outset of the prior section, software estimation methods have been found wanting. LOC would seem to be irrelevant to contemporary system development environments, but FP analysis does not appear to be much better. As we also said at the beginning of the prior section, these methods appear to be about as good as anything at estimating software projects. Therefore, systems developers can expect high levels of change in project development duration.

Given the pace of technological business change, systems should be developed with anticipation of change. The prototyping approach, where a working version of the system is built before final design, is appropriate for some projects, where the idea is to design the product's final details *after* seeing a mock-up in action. Development principles that make sense in this environment include modularization, use of subroutines, documentation of interfaces, and high-level languages.

There are several implementations of risk analysis in software development. Buffer approaches will be discussed in Chapter 8. Another approach is to identify threats to project completion with respect to time, budget, and quality and develop contingency plans to be implemented if these threats materialize. One obvious contingency plan is to cancel the project should it encounter too many unexpected delays and budget overruns. Other contingency alternatives are to pull internal experts off other projects to get this project back on track. Outsourcing can also be used, retaining external experts from the consulting field.

5.2.1 Scheduling

It is a rule of thumb that coding is 90 percent complete for about half of the time it was in process. Debugging was estimated to be 99 percent complete most of the time it was underway. You have probably written a computer code and should understand this very well. Humans are optimistic. The time required for computer tasks is very difficult to estimate accurately.

In studies of government projects, where estimates were carefully updated every 2 weeks, there was very little change in estimated duration until the activity actually started. While the activity was underway, those activities that took less time than the estimate dropped, to reflect the new knowledge. On the other hand, when things were clearly going to take longer than estimated, the estimates were rarely changed until deep into the activity.

Milestones are concrete events denoting the completion of a project phase. When a phase of project activity (a set of related tasks, such as completion of the preliminary architecture and permits tasks in Fig. 5.3) is over, including successful component testing, a milestone is completed. Milestones can be scheduled as activities with zero duration, reflecting completion of a block of work.

Critical path models, to be elaborated on in Chapter 7, are network descriptions of project activities that identify activity start times and completion times. Brooks found preparation of critical path models to be their most valuable aspect. Dependencies are clearly identified, and there is value in estimating activity durations. Critical path models were meant to aid project control by allowing comparison of actual performance with planned performance. However, as a control device, model elements such as task completion status are often not updated until it is too late, at which time the system change is dramatic. When a delay is first noticed, the tendency is not to report it. Project management systems provide status information giving the latest reported picture of what is going on. The key to successful management is determining when something needs to be done to correct what is going on.

5.3 Summary

Estimation of activity duration and cost is key to sound scheduling and budgeting. It is a very difficult task because project activities are so variable in these terms.

A great deal of effort has been devoted to the accurate estimation of software development projects. Specific measures of project size include source lines of code (SLOC or LOC), the most commonly used measurement. This method is based on a concrete measure, which is attractive. But it is deficient in that it is often difficult to specify the specific lines in question. Different languages can require very different approaches for different tasks. In former times, when FORTRAN (Formula Translation) was the standard language for numerical computation and when COBOL (common business-oriented language) was the standard for data processing, estimates of lines of code were quite consistent. Now, however, with the many productivity tools available and a much greater variety of languages used, lines of code are not as useful as measures. FPs focus more on what software is intended to do. COCOMO reflects learning aspects of software productivity. These information system project estimation techniques can be applied by analogy to other project contexts.

A sound estimating procedure would identify the work to be done (the work breakdown structure), estimate the time required by resource, and estimate the money required to pay the required people or hire the required consultants or equipment. To

that, time-independent items such as materials or external purchases can be added. Overhead can then be allocated following company policies.

Uncertainty is a major factor with respect to the time project activities will take. Cost is often correlated with time. Most delays are incurred in the testing phase, late in the project. It is even rarer for a project to take less time than originally estimated. This is largely due to the human factor of rechecking work if it is completed early, or sometimes stalling. Projects have on occasion been completed ahead of schedule, but not very often.

Chapter 6

Selection from Project Options



It is important to keep track of the progress of a project, so that those that are not going to provide value to the organization can be canceled. Analyzing the expected financial impact of a project is referred to as a **business case**. Business cases can be accomplished in a number of different ways—some financial, others more subjective; some widely used, some offered as potential improvements. This chapter discusses the decision problem of project selection, and shows how the most commonly used methods work. Analytic methods support this selection process in two ways. First, they provide decision makers with analysis of expected outcomes from adopting specific projects. Second, they provide a basis for communication, so that the reasoning behind a selection decision can be explained to others.

6.1 Measurement of Project Impact

Projects typically involve benefits that are difficult to measure in terms of concrete monetary benefits. This is especially true for information systems projects. This vastly complicates sound management, because cost–benefit analysis, the ideal tool for evaluating project proposals, will not always accurately reflect project benefits.

Intangible factors: Both costs and benefits tend to have intangible features. The tangible costs and benefits tend to be historical, backed by data or solid price quotations from vendors. But many of the benefits are expected in the future and are very difficult to measure. These include expected increases in market share, improved customer service, and a better corporate image. These would all have a significant impact on the corporation's bottom line, but guessing exactly what that impact would be is challenging at best.

6.2 Selection Practice

Information technology tended to be treated as a capital investment. The following financial techniques have often been used.

1. Payback
2. Discounted cash flow
3. Cost–benefit analysis

Each of these methods will be demonstrated in this chapter. Decision makers treat information technology investment more like operations projects with measurable profitability requirements than the revenue-based appraisals appropriate for training and marketing. Operations investments focus on efficiency-related measures. Conversely, marketing investments are usually viewed in terms of their expected impact on improving their competitive position or increasing market share. The capital approach was very appropriate for hardware proposals, but difficulties were encountered when justifying software. **Intangible benefits** such as competitive advantage or improved practices tend to be disregarded because they are hard to quantify. The most cited justification was the reduction in expenses, usually in payroll. The second most common justification involved the subjective aspect of accomplishing some strategic objectives. Intangible benefits cited were enhanced patient care and satisfaction for a health care organization, better speed of responding to customers, and the need to satisfy governmental regulations.

Net present value masks some of the true value of information technology proposals. On the other hand, some projects that have a low impact on corporate performance often appear attractive to cost–benefit analyses. This is because cost–benefit analysis emphasizes those features most easily measured. These are sometimes the intangible strategic benefits that are often disregarded because they were not measurable.

Cost–benefit analysis should consider costs over the entire life cycle of the project. Life cycle costs are often four times those of development costs for many projects. But these long-range costs are much less predictable, and therefore often not included in cost–benefit analyses.

Another issue involves who should do the estimating. In construction, a common practice is to have those who will be responsible for implementing the project do the estimation. This is in a competitive bidding environment, where there is great danger in estimating too low. The motivation is for these estimators to be very cautious. This tends to result in safe estimates considering every possible risk, which is another way of saying that it results in inflated estimates. (Inflating estimates is how you cope with risk at the estimation stage.) Software developers who estimate their own performance tend to be optimistic.

6.3 Project Evaluation Techniques

A number of methods exist to evaluate project proposals, either from the perspective of selecting the best option available, designing an ideal option, or rank-ordering options. This section demonstrates some of the most widely used methods and shows how other methods can be used to consider other factors describing expected project performance.

6.3.1 General Project Selection

Economic and financial analyses include payback (determining the expected time until investment is recovered) and cost–benefit analysis. Net present value and internal rate of return extend cost–benefit analysis to consider the time value of money, appropriate when projects are lengthy (3 years or more).

Checklists describe the criteria of importance and their minimum acceptable levels of requirement. Screening methods are a variant of checklists, eliminating projects that do not have minimum estimated performance on specific measures. Project profiles describe the performance of each project on criteria, so that the decision maker can see each project’s strengths and weaknesses. Scoring and rating models are a simple form of multicriteria analysis where measures are obtained on each criterion of importance and combined in some fashion. Multicriteria decision models are in general more formal than scoring and rating models but operate on essentially the same principle—identify factors that are important, measure how well each project does on each factor, and combine these into some value score that can be used for ranking.

The rest of this chapter will demonstrate most of the basic methods. The first method demonstrated is screening, a common form of the checklist method that can be and often is combined with other criteria.

6.3.2 Screening

Screening is a process that is very useful in cutting down the dimensions of the decision problem. The way in which screening operates can vary widely in detail, but essentially involves identifying those factors that are important, establishing a minimum level of importance, and eliminating those projects that fail on any one of these minimum standards. Obviously, if the standards are set too high, the decision problem disappears as no projects survive the screening. This is appropriate if the minimum standards reflect what management demands in return for their investment.

To demonstrate screening, assume that 100 information systems project proposals are received this month. All the projects were evidently worthwhile in someone’s

mind, but management must consider budgets and other resource limitations. Assume that the criteria and minimum performance levels given in Table 6.1 are required:

If any of the 100 proposed projects failed to meet all four of these standards, they would be rejected preemptively. This reduces the number of proposed projects for a more detailed analysis. This approach can be implemented by checklists, which give clearly defined standards on those areas of importance to management.

Screening is good at quickly weeding out those projects with unacceptable features. The negative side of screening is that trade-offs between very good features and these unacceptable features are disregarded. The willingness of decision makers to accept lower return on investment (ROI) for projects with strategic importance is disregarded. For those projects for which such trade-offs are not important, screening is a very efficient way to reduce the number of proposals to a more manageable number.

In the prior section, we gave a list of risk factors for information system projects. These could be implemented as a **checklist** by management specifying minimum acceptable measures that can be used to screen individual projects. Not all risk elements might apply for a given organization’s checklist. An example checklist is presented in Table 6.2:

Checklists ensure implementation of policy limits. Checklists are a way of implementing screening from the perspective of what features management feels are important. The next step in analysis is to compare alternative project proposals more directly.

Table 6.1 Screening example

Expected return on investment	At least 30%
Qualified project team leadership	Available
Company expertise in this type of work	Either company has experience or desires to gain it
Project completion time	Within 48 months

Table 6.2 Example checklist

Factor	Minimum standard
Project manager ability	Qualified manager available
Experience with this type of application	Have experience, or application is a strategically key new technology
Experience with system or language	Personnel with experience can be obtained
Familiarity with modern programming practices	If not, training is available
Availability of critical equipment, software, and programming language	Each critical component available
Project team complete	Key personnel identified and agreed to work; Support personnel easily available

The intent of a **project profile** is to display how the project proposal compares to standards as well as how the project compares to other proposals. Profiles have a benefit over screening limits, because poor performance on one factor can be compensated for by strong performance on another factor. For instance, matching with company strategic programs can be an important factor. There could be other project proposals that contribute nothing to the firm’s strategic program yet have an outstanding cost improvement for administrative work. This would be reflected in very strong performance on ROI. Conversely, another project may have a slightly negative ROI calculation, but may involve entering a new field in which the firm wants to gain experience. Here we give a short list of six proposals in Table 6.3, measured for resources used, as well as benefits expected.

A profile displays the characteristics of individual projects. Estimated cost is needed to determine if available budget can support a project. A tabular form is given above. Figure 6.1 gives a relative graphical display of how each project performs relative to the others, with 1 being the best measure and 0 the worst:

Table 6.3 Project profiles

Project identifier	Estimated cost	Cash flow this period	NPV/Cost ratio	Key to strategy
A1	2,580,000	100,000	0.43	No
A2	3,570,000	−190,000	0.51	Yes
A3	2,790,000	360,000	0.46	No
B1	3,480,000	−52,000	0.11	Yes
B2	1,910,000	−200,000	0.22	Yes
C1	6,410,000	170,000	0.41	No

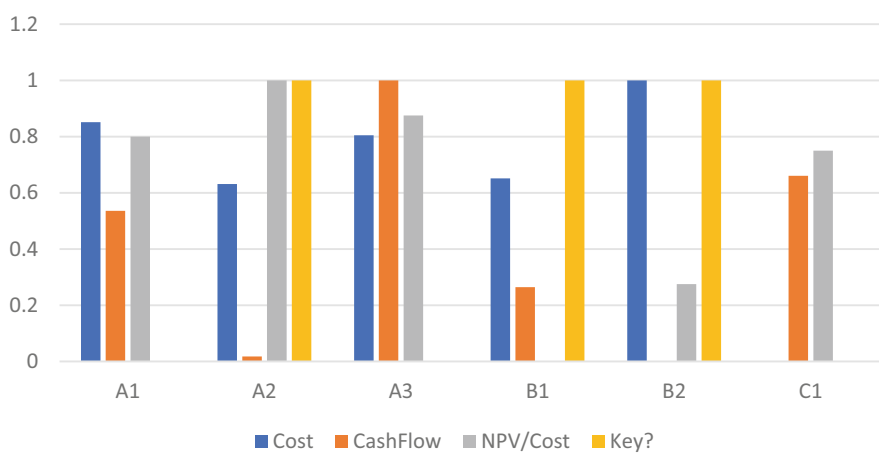


Fig. 6.1 Graphical display of relative project performance

Project B2 is shown as the best on cost, but the worst on cash flow. Project A2 is best on NPV/cost ratio, but very bad on cash flow. Project Ac is strong on everything but is not key strategically. Project B1 is key strategically, but not that great on any other measure. Project B2 is very strong on cost as well as being key strategically, but very bad on cash flow and not much good on NPV/cost ratio. Project C3 is weak on cost and is not key strategically.

Graphical displays and ratios are often valuable to give a measure with which relative performance can be measured. For measures such as NPV/cost ratio, cutoff levels can be used to screen out projects. For instance, a 40 percent return on estimated cost in net present terms might be desired. Projects B1 and B2 are both below this limit and might be screened out. However, both projects are listed as key to the organization’s strategy, and management might be willing to accept a lower return for the potential for advancing organizational strategy.

6.4 Traditional Analysis

Assume that a company needs a machine for a three-year project. They have two choices—option A and option B. The company has a marginal value of capital of 10 percent per year.

6.4.1 Cost–Benefit Analysis

The cost–benefit calculation for a new project requires identification of benefits in monetary units. Use of net present value requires identification of the timing of monetary exchanges. Option A is more expensive but has lower expected operating expenses. The expected revenue is the same. Numbers are given in Table 6.4:

Table 6.4 Simple cost–benefit calculation

Year	Invest A	Cost A	Revenue A	Invest B	Cost B	Revenue B
0	600,000			500,000		
1		90,000	350,000		100,000	350,000
2		100,000	380,000		130,000	380,000
3		110,000	400,000		200,000	400,000
	600,000	300,000	1,130,000	500,000	430,000	1,130,000
		Numerator	830,000		Numerator	700,000
		Denominator	600,000		Denominator	500,000
		Ratio	1.383		Ratio	1.400

Table 6.5 Payback

Year	In-house net	Cumulative	Outsource net	Cumulative
0	(\$600,000)	(\$600,000)	(\$500,000)	(\$500,000)
1	\$260,000	(\$340,000)	\$250,000	(\$250,000)
2	\$280,000	(\$60,000)	\$250,000	\$0,000
3	\$290,000	\$230,000	\$200,000	\$200,000

Here option A has the lower cost benefit ratio by a slight margin (1.383 to 1.400). The cost–benefit ratio requires some convention to describe just what costs are assigned to investment and which ones to benefits. Operating expenses seem more appropriately combined with savings, but note that different views might yield opposing conclusions.

Ratio: The nominal cost–benefit ratio (disregarding the time value of money) for option A is $830,000/600,000 = 1.383$, slightly lower than option B’s ratio of $700,000/500,000 = 1.4000$. This indicates that either option would be worthwhile.

Return on investment (ROI) is defined as net project benefits divided by investment (times 100 if you want to view ROI in percentage return). It can be applied to non-discounted cash flows as with cost–benefit analysis. For long-term projects, discounted ROI should be used, which we will demonstrate in the net present value section.

6.4.2 Payback

Payback is a rough estimate of the time required to recover investment. While being simple, payback presents a view of the transaction that is very understandable and important to managers. One alternative may be superior to another on the net present value of the total life cycle of the project. However, cost–benefit analysis does not consider the impact of negative cash flow in early periods. For instance, in our example, the cash flow would be as given in Table 6.4. The net benefit column is calculated by subtracting the cost of the new system from the cost of the old system by year (Table 6.5):

Option A takes slightly over 2 years to cover its investment, while option B breaks even after 2 years. Thus, the payback for option B is preferable.

6.4.3 Net Present Value

We can modify the cost–benefit ratio by considering the **time value of money**. For option A in this example, the nominal expected gains of \$830,000 are spread out over 3 years, while the development costs are incurred at the beginning. Having the

Table 6.6 Net present value

Year (t)	In-house net	Divide by 1.10^t	Outsource net	Divide by 1.10^t
0	(\$600,000)	(\$600,000)	(\$500,000)	(\$500,000)
1	\$260,000	\$236,364	\$250,000	\$227,273
2	\$280,000	\$231,405	\$250,000	\$206,612
3	\$290,000	\$217,881	\$200,000	\$150,263
NPV		85,650		\$84,147

\$600,000 (or \$500,000 for option B) would mean that the company would not be able to adopt some other investments (and maybe even force the firm to borrow money). Assume the marginal value of money for the firm is 10 percent per year. **Net present value** converts a time stream of money back to its worth in today's terms (or in terms of the project's start, or any other specific time of reference).

Table 6.6 shows the changes in cash flow between the two systems (shown in the net difference column). Discounting each year's net change in cash flow by the discount rate of 1.10 per year to the t -th power, where t is the time period, we get the following. Note that initial expenses are treated as occurring during year 0.

Viewed in this light, relative to obtaining a return of 10 percent per year on alternative investments, adopting option A would be like receiving a check initially for over \$85,651, while option B would be equivalent to a gain of over \$84,147. Option A is slightly better. This analysis is cut off after 3 years. The trends look like option A would continue to provide better returns should a longer analysis time frame be used.

A related concept is **internal rate of return (IRR)**, which is the marginal value of capital for which the net present value of a stream of cash flow would break even, or equal zero. Note that IRR can equate to ROI in that ROI is viewed with discounted cash flow. In this case, the IRR amounts to 1.178 for option A and while the IRR of option B is 1.197. By these measures, option B is slightly preferable. Here both options are quite close, with different measures indicating different preferred options.

Cost–benefit analysis seeks to identify accurate measures of benefits and costs in monetary terms and uses the ratio benefits/costs (the term benefit–cost ratio seems more appropriate, and is sometimes used, but most people refer to cost–benefit analysis). For projects involving long time frames, considering the net present value of benefits and costs is important.

6.4.4 Other Factors

There are several complications that can be brought into the calculation of cost–benefit ratios. One of the most obvious limitations of the method is that benefits, and even costs, can involve high levels of uncertainty. The element of chance can be included in cost–benefit calculations by using expected values.

The cost–benefit ratio does not reflect intangible benefits unless they are presented in monetary terms. Cost–benefit analyses have included measurements for intangible items, but they tend to be given lower values because of the uncertainty involved in their estimates. Detailed analyses of the decision maker’s willingness to pay for intangible factors have been conducted but can be time-consuming and less than convincing. Governments have encountered some problems in applying cost–benefit analysis to public works, to include (1) evaluating the benefit of recreational facilities and (2) placing a dollar value on human life. When a dam is built, there clearly is benefit obtained from providing many citizens with much improved fishing and water sports. (There is also added cost in depriving citizens of the opportunity to view some flooded historical sites.) The approach usually taken has been to place some dollar value on recreation, based on some very insubstantial measures. The evaluation of human life has also been tackled by economists, who have applied things like the net present value of the expected earnings of those whose lives are expected to be lost in some proposed investment project. This of course involves high levels of speculation as well because the calculation assumes certain ages, assumes that the only value of a human is what they earn, and disregards who pays and who benefits.

If a firm was threatened with a severe monetary penalty for not complying with a governmental regulation with respect to environmental pollution or safe working conditions, a net present value analysis might well lead to the conclusion that it would be rational to pay the penalty and avoid improving operations. For instance, assume that a blast furnace is pouring out black matter at a phenomenal rate that the government finds terribly offensive. Governmental regulations call for a penalty of \$100,000,000 if the pollution source is not cleaned up within 1 year. Hard-core cost–benefit analysis would identify the cost of cleaning up the facility, which might involve an expense of \$30,000,000 in equipment and installation, and an added cost of operations of \$12,000,000 per year growing at the rate of 5 percent per year over the next 8 years, the remaining life of the equipment. At a discount rate of 12 percent per year, the net present value of benefits and costs would be as shown in Table 6.7 the net column shows discounted values for benefits and costs. Totals are presented at the end.

Here the ratio of net present benefits to net present costs is $\$89,285,714 / \$95,919,519 = 0.25$, below 1.0, indicating that the rational economic decision maker would pay the fine and keep operating as is. But the government did not impose the fine limit for the purpose of raising money. They imposed the fine as a means to coerce polluters to clean up operations. The U.S. Congress has no trouble adding extra zeroes to penalties. If the firm continued to pollute, it is not too hard to imagine the penalty being raised in the future to a much larger figure. There have been actual cases similar to this scenario, where within 3 years the penalty was raised to much larger values, providing a much different cost–benefit ratio. Benefits are often difficult to forecast.

Table 6.7 Net present value calculation

Year	Benefit	Net	Costs	Net
1	100,000,000	89,285,714	42,000,000	37,500,000
2			12,600,000	10,044,643
3			13,230,000	9,416,853
4			13,891,500	8,828,299
5			14,586,075	8,276,531
6			15,315,379	7,759,248
7			16,081,148	7,274,295
8			16,885,205	6,819,651
NPV		89,285,714		95,919,519

6.5 Multiple Objectives

Profit has long been viewed as the determining objective of a business. However, as society becomes more complex, and as the competitive environment develops, businesses are finding that they need to consider multiple objectives. Although short-run profit remains important, long-run factors such as market maintenance, product quality, and development of productive capacity often conflict with measurable short-run profit.

6.5.1 Conflicts

Conflicts are inherent in most interesting decisions. In business, **profit** is a valuable concentration point for many decision makers because it has the apparent advantage of providing a measure of worth. Minimizing **risk** becomes a second dimension for decision making. There are cash flow needs, which become important in some circumstances. Businesses need **developed markets** to survive. The impact of advertising expenditure is often very difficult to forecast. Yet decision makers must consider advertising impact. **Capital replenishment** is another decision factor which requires consideration of trade-offs. The greatest short-run profit will normally be obtained by delaying reinvestment in capital equipment. Many U.S. companies have been known to cut back capital investment to appear reasonably profitable to investors. **Labor** policies can also have an impact on long-range profit. In the short run, profit will generally be improved by holding the line on wage rates and risking a high labor turnover. There are costs that are not obvious, however, in such a policy. First, there is the training expense involved with a high turnover environment. The experience of the members of an organization can be one of its most valuable assets. Second, it is difficult for employees to maintain a positive attitude when their experience is that short-run profit is always placed ahead of employee welfare. And

innovative ideas are probably best found from those people who are involved with the grass roots of an organization—the workforce.

This variety of objectives presents decision makers with the need to balance conflicting objectives. We will present the simple multi-attribute rating technique (SMART), an easy-to-use method to aid selection decisions with multiple objectives.

The **simple multi-attribute rating technique (SMART)** method offers a simple way to quantify subjective elements. There are two elements to the value function in SMART: factor weight (w_i) and score (s_{ij}) of each alternative j on each factor. Overall value is then:

$$value_j = \sum_{i=1}^k w_i \times s_{ij}$$

Weights w_i can be obtained in a number of ways, but a simple estimate with some reliability involves ranking the factors, considering the range of likely performances, and then estimating the relative value of moving from the worst to the best performance relative to such a swing from either the most important or the least important. We will do both and use a rough average of these two estimates. In the example used earlier in this paper, assume that the ranges and rankings for the five criteria are as given in Table 6.8.

First, the extreme expected measures for each criterion are identified, for the decision maker to understand the range of possibilities to compare. Then, the decision maker ranks the importance of swinging from the worst measure to the best measure for each criterion. In this case, the decision maker considered it most important to gain net present value from 0 to 100 percent, followed by moving market share from very low to very high, moving risk from very high to very low, control from low to high, and final moving investment from \$1 million to \$100,000. Next the decision maker is asked to quantify these changes. Viewed from the perspective of the most important criterion, if changing return in terms of NPV was worth 100, the decision maker assigned a value of 90 to improving market share, 70 to improving risk, 30 to improving control, and 25 to investment. To get a second estimate, with 10 assigned to the least important criterion (investment), the assigned values were 15 for control, 30 for risk, 40 for market share, and 50 for return. This information is then used to generate the set of weights w_i . Each of the sets of assigned weights is totaled, and each

Table 6.8 Criteria weight development

Criterion	Worst	Best	Rank	Best = 100	Worst = 10
Risk	Very high	Very low	3	70	30
Market share	Very negative	Very positive	2	90	40
Control	Low	High	4	30	15
Investment	\$1,000,000	\$100,000	5	25	10
Return (NPV)	0%	100%	1	100	50

Table 6.9 SMART calculation of weights

Criterion	Best = 100	Divide by sum	Worst = 10	Divide by sum	Compromise
Risk	70	0.23	30	0.21	0.21
Market share	90	0.30	40	0.28	0.28
Control	30	0.10	15	0.10	0.10
Investment	25	0.08	10	0.07	0.08
Return (NPV)	100	0.33	50	0.34	0.33

Table 6.10 SMART value calculation

Weights	0.21	0.28	0.1	0.08	0.33	1
Criteria	Risk	Mkt Share	Control	Invest	NPV	Values
In-house	0.3	0.7	0.9	0.3	0.5	0.538
Outsource	0.9	1	0.2	0.7	0.8	0.809

entry divided by the appropriate total. This gives two estimates of weights, which the decision maker can use to compromise (preferably using rounded numbers) as in Table 6.9.

The scores (s_{ij}) for each alternative on each criterion are now needed. These can be assigned directly, to reflect any kind of utility function for each criterion. Better performance should always receive a higher score than an inferior measure. A possible set of assigned scores are given in Table 6.10.

In this case, the outsourced alternative has a clear advantage.

6.6 Summary

We have reviewed some of the primary methods used to evaluate project proposals. Screening provides a way to simplify the decision problem by focusing on those projects that are acceptable on all measures. Profiles provide information that display trade-offs on different measures of importance. Cost–benefit analysis (with net present value used if the time dimension is present) is the ideal approach from the theoretical perspective but has a number of limitations. It is very difficult to measure benefits, and also difficult to measure some aspects of costs accurately. One way of dealing with this problem is to measure more accurately. Economists have developed ways to estimate the value of a life and the value of scenic beauty. However, these measures are difficult to sell to everybody.

A more common view is that it is wasted effort to spend inordinate time seeking a highly unstable and inaccurate dollar estimate for many intangible factors. Value analysis is one such alternative method. Value analysis isolates intangible benefits from those benefits and costs that are more accurately measurable in monetary terms

and relies upon the decision maker's judgment to come to a more informed decision. The SMART method, one of a family of multiple criteria decision analysis techniques, provides a way to quantify these intangible factors to allow decision makers to trade-off values.

Cost–benefit provides an ideal way to proceed if there are no intangible factors (or at least no important intangible factors). However, usually such factors are present. Intermediate approaches, such as payback analysis and value analysis, exist to deal with some cases. More complex cases are better supported by multiple criteria analysis. In cases of constraints, such as budgets, it is sometimes appropriate to optimize over some objective. Linear programming provides a means of generating the best portfolio of funded projects subject to constraint limits given that accurate measures of project performance are available.

Chapter 7

Project Scheduling



The critical path method provides a way to easily identify the soonest a project can be completed, given that the estimated durations of activities are accurate. Even though estimated durations are usually at variance with actual outcomes, the critical path method provides a useful analysis of which activities are time bottlenecks. The input to the critical path method is a list of each activity, its expected duration, and those activities that immediately precede this activity. “Immediately precede” in this case means that predecessor activities must be completed before the subject activity can begin and that there are no other activities between the predecessor and the activity in question.

The **critical path method** provides a basis for identifying the criticality of specific activities, which can help determine which among competing activities can be delayed to stay within resource levels. A **critical activity** is an activity that must be completed on schedule or else project completion will be delayed. Therefore, critical activities are said to have no **slack**, or spare time to complete. A **critical path** is the chain of critical activities from the beginning of the project to its completion. If a project has no overall slack between the time it is to be completed and the minimum time to complete based on duration estimates, there will be at least one critical path for the project. However, there may be multiple critical paths, or more than one sequence of activities related by predecessor/follower relationships that have zero slack. To demonstrate, consider the following very simple project to make a movie, consisting of six activities, as shown in Table 7.1.

Another graphical output of the critical path method is a **Gantt chart**, which displays the early start schedule versus time. In this case, it is clear that there is only one critical path, consisting of the chain of activities A–B–C–D–F. Activity E has slack. This is all the information required to develop a critical path model. The critical path algorithm is quite straightforward.

Table 7.1 Demonstration activity list

Activity	Duration (weeks)	Predecessors
A Acquire script	4	None
B Cast	7	A
C Film	20	B
D Edit	5	C
E Advertise	3	A
F Distribute (milestone)	0	D, E

7.1 Early Start Schedule

The first step in CPM is to develop an early start schedule (first pass) (Table 7.2). For every activity that has no unscheduled predecessors, schedule the activity to start as soon as possible (either the project start time or the maximum early finish of all predecessors). The critical path schedules are optimal with respect to time. This process continues until all activities are scheduled. The early finish is the sum of the early start time plus the duration.

The project **early completion time** is the maximum early finish. In this case, the project early completion time is 36 weeks. Figure 7.1 displays the schedule:

Table 7.2 Early start schedule

Activity	Early start	early finish	
A Acquire script	0	4	Releases B and E
B Cast	4	$4 + 7 = 11$	Releases C
C Film	11	$11 + 20 = 31$	Releases D
D Edit	31	$31 + 5 = 36$	With E, releases F
E Advertise	4	$4 + 3 = 7$	With D, releases F
F Distribution (milestone)	$\text{MAX}(7, 36)$	$36 + 0 = 36$	

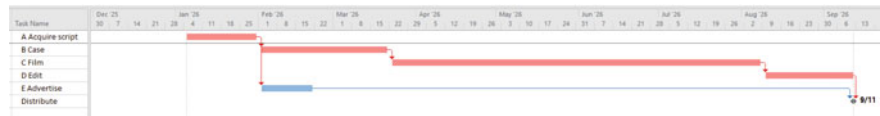


Fig. 7.1 Gantt chart

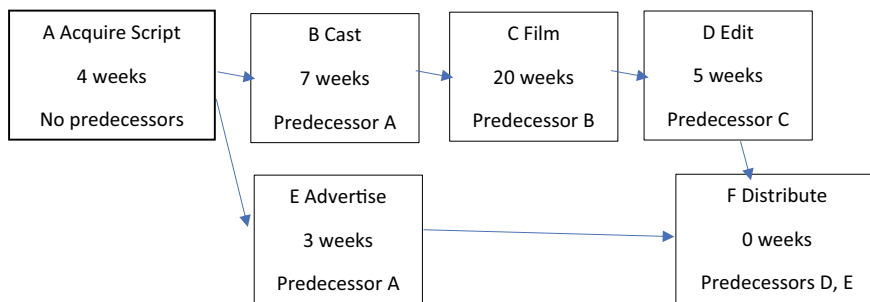


Fig. 7.2 Network

7.2 Networks

Quite often a **network** is displayed, which graphically displays the relationship between activities (Fig. 7.2). Networks are not really needed for the development of early start schedules but are very useful in sorting out the relationships for late start schedules. They also provide a valuable visual aid for managers to identify relationships among activities.

7.3 Late Start Schedule

The next phase of the critical path analysis is the **late start schedule** (second pass). The late start schedule is the latest that an activity can be scheduled without delaying project completion time. The final ending time for the project can be some contract deadline, which may be different from the early finish schedule, or the early finish project completion time can be used. If the deadline is earlier than the project early finish time, the project is infeasible (cannot be completed on time with given durations). If the deadline is later than the project early finish time, all activities in the project will have slack, or spare time. If the deadline coincides with the project early finish time, there will be at least one **critical path**, connecting activities in a chain with zero slack.

The late start schedule (Table 7.3) is calculated in reverse. Begin with the end time (deadline or early finish time). All activities that do not appear on the list of predecessors for unscheduled activities can be scheduled. The late finish time will be either the project end time or the minimum of the late start times for all following activities.

Table 7.3 Late start schedule

Activity	Late finish	Late start	
F Distribution	36	$36 - 0 = 36$	Releases D and E
E Advertise	36	$36 - 4 = 32$	Releases A
D Edit	36	$36 - 5 = 31$	Releases C
C Film	31	$31 - 20 = 11$	Releases B
B Cast	11	$11 - 7 = 4$	With E, Releases A
A Acquire script	$\text{MIN}(4, 32)$	$4 - 4 = 0$	

7.4 Slack

Slack (float) is the difference between the late start and early start schedules. (It doesn't matter which you use, because in both cases the difference between them is the duration.) Those activities with zero slack are critical. If they are delayed, the project completion will be delayed. There can be more than one critical path for a project, and the project network presented in Fig. 7.2 can be useful in ensuring identification of each critical path. Calculations are given in Table 7.4:

Slack in the preceding case exists for only one activity, E. More complex projects will include slack for multiple activities, such as the following scheduling of multiple projects (Table 7.5). Consider a building project consisting of two houses, each with six activities, using shared crews.

This project has the following slacks (Table 7.6):

A Gantt chart for this project is shown in Fig. 7.3, with critical path 1–2–3–9–11–12:

In this case, activities 7 (excavate house B) and 8 (concrete house B) have three days of slack. This is shared slack, in that if one runs longer than estimated, the slack for the other would be reduced. This case is **shared slack**. The Gantt chart for this project shows that activity 4 (roof house A) shares some slack with activities 6 (finish house A) and 10 (roof house B). There is no completely **independent slack** (free float) where loss of time would have no interference with other activities.

Table 7.4 Slack calculations

Activity	Early start	Early finish	Late start	Late finish	Slack	
A Acquire script	0	4	0	4	0	Critical
B Cast	4	11	4	11	0	Critical
C Film	11	31	11	31	0	Critical
D Edit	31	36	31	36	0	Critical
E Advertise	4	7	33	36	29	
F Distribute (milestone)	36	36	36	36	0	Milestone

Table 7.5 Multi-project Schedule

Activity	Duration (days)	Predecessors
1 Excavate house A	3	None
2 Concrete house A	2	1
3 Frame house A	7	2
4 Roof house A	3	3
5 Interior house A	6	3
6 Finish house A	5	4,5
7 Excavate house B	4	1
8 Concrete house B	2	2,7
9 Frame house B	8	3,8
10 Roof house B	3	4,9
11 Interior house B	8	5,9
12 Finish house B	5	6,10,11

Table 7.6 Display of slacks

Activity	Early start	Early finish	Late start	Late finish	Slack	
1 Excavate house A	0	3	0	3	0	Critical
2 Concrete house A	3	5	3	5	0	Critical
3 Frame house A	5	12	5	12	0	Critical
4 Roof house A	12	15	20	23	8	
5 Interior house A	12	18	14	20	2	
6 Finish house A	18	23	23	28	5	
7 Excavate house B	3	7	6	10	3	
8 Concrete house B	7	9	10	12	3	
9 Frame house B	12	20	12	20	0	Critical
10 Roof house B	20	23	25	28	5	
11 Interior house B	20	28	20	28	0	Critical
12 Finish house B	28	33	28	33	0	Critical

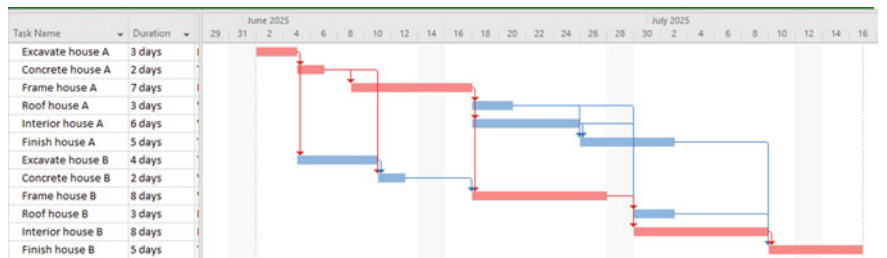


Fig. 7.3 Gantt chart for house project

Project management software includes the ability to have a number of precedence relationships. The default is generally predecessor finish = follower start. Other options include predecessor start = follower start, or predecessor start = follower start plus some lag time.

The critical path method provides a useful means of identifying the earliest a project can be completed, as well as identifying those activities that are critical. Critical activities need to be managed more closely than slack activities. This is because if any delay is experienced on a critical activity, the project completion will be delayed. On the other hand, if things are delayed for activity 6 (finish house A), it doesn't really matter until 5 spare days are wasted. However, one should be careful, because once all an activity's slack is exhausted, that activity also becomes critical.

There is a bias in projects in that activity delays accumulate, while gains from finishing early do not. This is because when an activity is late, those that must wait for it to be completed start later than scheduled. On the other hand, if an activity should be finished before it was scheduled to finish, the advantage can rarely be used, because in complicated projects different crews and materials have to be gathered and many different people need to be coordinated. The early finish time is not usually known much ahead of the activity's completion. Therefore, it is very difficult to gather all the following activities' resources together in time to start early.

Once the final planned time schedule is developed, it can be used to estimate costs, such as expenditures for human resources, materials, and overhead. Resource time requirements are a major component of estimated cost. Also important are materials used, as well as vendor-delivered products. Overhead expenses can be estimated independently of work volume. Obviously, since time is a major element in estimating budgets, one of the major reasons projects run over budget is that they take more time than expected.

Project managers usually focus on scheduling the critical chain of activities closely to make sure that they have the resources needed to proceed according to schedule. The critical chain of activities includes those activities that are critical (if managerial control can influence their duration) but is not limited to these activities. Activities with very little slack can become problems if they are delayed up to or beyond their slack. Therefore, the slack of noncritical activities should also be monitored to make sure that new critical activities are identified.

7.5 Resource Leveling

The critical path model we have considered so far has assumed unlimited resources. **Resource leveling** is the process of spreading out the early start schedule so that the maximum number of a particular resource required can be reduced. For instance, if a particular specialist is needed to accomplish more than one activity and these activities happen to be scheduled during common periods of time, something would have to give. One or the other of the activities sharing the common resource would have to be delayed (or additional resources would have to be acquired).

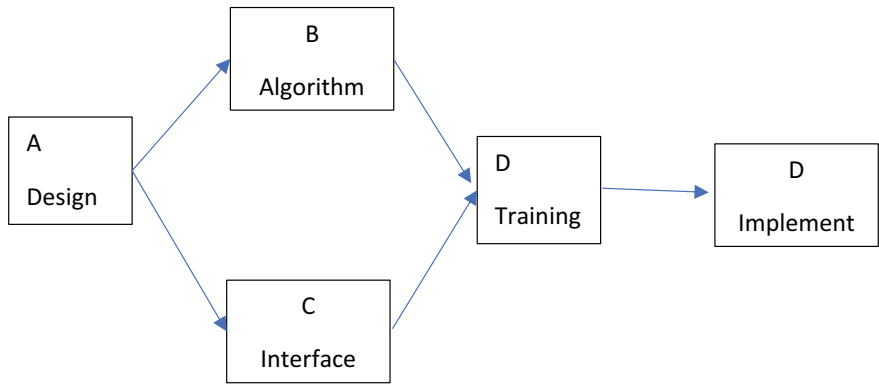


Fig. 7.4 Network for leveling project

To demonstrate, consider a project (network in Fig. 7.4) with the following data: The project data is given in Table 7.7: Figure 7.5 presents the schedule for this project.

If the same specialist was required for both activities B and C, they could not be accomplished concurrently. This means that the original critical path model is infeasible. Either B or C must be delayed. The decision as to what to delay is difficult to make if you are after the guaranteed best answer. For small projects, you might as well check all the possibilities. For instance, in this case there are only two activities to delay. If C is delayed, it cannot start until week 12, when both activity A is completed, and the specialist is through with activity B (Table 7.8):

Table 7.7 Leveling project data

Activity	Duration (weeks)	Pred	Early start	Early finish	Late start	Late finish	Slack	Resource
A Design	5	None	0	5	0	5	0	
B Algorithm	7	A	5	12	5	12	0	Specialist
C Interface	4	A	5	9	8	12	3	Specialist
D Training	2	B,C	12	14	12	14	0	
E Implement	1	D	14	15	14	15	0	

Activity	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A Design															
B Algorithm															
C Interface															
D Training															
E Implement															

Fig. 7.5 Schedule for leveling project

Table 7.8 Levelled project after C delayed

Activity	Duration (weeks)	Pred	Early start	Early finish	Late start	Late finish	Slack	Resource
A Design	5	None	0	5	0	5	0	
B Algorithm	7	A	5	12	5	12	0	Specialist
C Interface	4	B	12	16	8	12	3	Specialist
D Training	2	C	16	18	12	14	0	
E Implement	1	D	18	19	14	15	0	

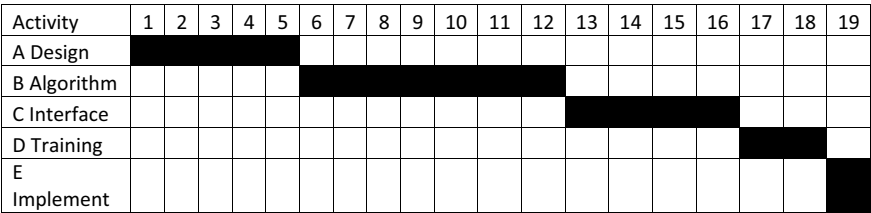


Fig. 7.6 Levelled project

Figure 7.6 gives the leveled schedule for this project. In this case, nothing is slack, because delaying any activity will delay the end of the project. We can see this, although the method of using only predecessors will not yield that result, because of the need to consider the limitation on the number of specialists.

The alternative is to delay activity C. This yields Table 7.9: This yields an identical project completion time. The total project is completed in 19 weeks instead of the 15 weeks (Fig. 7.7):

Although there is no guaranteed best way of deciding which activity to delay when sharing scarce resources, it usually works well to use a priority system. A good choice would be to schedule critical activities first, and if more than one critical activity shares the scarce resource, a second priority that is often used is to schedule the longest activity among those selected by the first priority. If multiple resources

Table 7.9 Levelled project after B delayed

Activity	Duration (weeks)	Pred	Early start	Early finish	Late start	Late finish	Slack	Resource
A Design	5	None	0	5	0	5	0	
B Algorithm	7	C	9	16	9	16	0	Specialist
C Interface	4	A	5	9	5	9	0	Specialist
D Training	2	B	16	18	16	18	0	
E Implement	1	D	18	19	18	19	0	

Activity	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
A Design																			
B Algorithm																			
C Interface																			
D Training																			
E Implement																			

Fig. 7.7 Alternative leveled project

are limited, an alternative priority could be given to scheduling the resource with the greatest number of limited resources. Project management software usually levels projects for you. They vary in what priority rules they use.

7.6 Multiproject Environments

Many organizations have evolved into organizations that manage projects as the focus of their business. This has long been true in the construction industry, where firms develop skills and equipment expertise that can be applied to a type of work, giving them competitive advantage. It has also long been true in the accounting field, where large firms develop a pool of experts in various fields that can be utilized on the problems of many firms. Information system project management has evolved from the accounting field, with many firms offering expertise that is available for hire. This is probably the most common form of outsourcing, applying expertise from consulting firms for specific problems within organizations. The matrix form of organization is especially appropriate in this environment, with pools of experts trained to a high state of development in their specialty, available on call to cope with work as it is acquired.

Both in information systems and in other fields such as construction, the vast majority of projects occur in a multiple project environment. Projects in this environment tend to be smaller than projects found in other environments. The reason for dealing with multiple projects is to share key resources, which would otherwise be underutilized. Critical resources of this type would be the bottlenecks that make sense to use as the top priority in scheduling. The scheduling of projects subject to resource smoothing was discussed earlier. In addition to the complications involved in scheduling critical resources, there are additional sources of change in multiple project environments. As new projects are obtained, and old projects encounter difficulties, the priority for use of critical resources is liable to change. For instance, a key production crew might have originally been scheduled to work on a project for the first 3 weeks in March. After one of these weeks' work was completed, an emergency requirement might arise where this crew could help. Top management has the decision of disrupting this current project to fix problems on the project with the emergency. It often makes economic sense to have one project suffer for the

greater good of the firm overall. Management is responsible for setting priorities and determining which projects will receive top priority.

7.6.1 Resource Smoothing

Resource leveling focuses on extending schedules so that particular resources are not overscheduled. **Resource smoothing** focuses on adjusting schedules to obtain a level amount of work for a given resource. This includes possible extension (or compression) of activities to make the work volume more even. It can also include identifying gaps in the work schedule that management might be able to fill by finding extra work.

Consider a multiple project operation in the road construction industry. Four resources are involved: crews for grading, for culvert work, for base, and for surfacing. For simplicity, assume that these four activities are accomplished in sequence in the order given. The firm currently has three projects, all with a deadline of being completed by the end of week 20. Project data are displayed in Table 7.10.

The Gantt chart for this schedule is shown in Fig. 7.8:

Usages for each resource are given in Table 7.11, with a blockout indicating the resource required by week. For instance, three grading crews are required in weeks 1 and 2 and two in week 3. One culvert crew is needed in week 2, three in weeks 3 through 6, two in week 7 and one in week 8.

A smooth schedule is one where a resource is working at the same level of activity each time period. These resource usage charts indicate that there are two types of smoothing that might be useful: gaps in work to do, and excess work to do. When there is no work to do, a valley (no activity) is encountered. When there is too much

Table 7.10 Project inputs for smoothing

Activity	Duration (weeks)	Predecessor	Start	Finish	Resource
1 Grading A	2	None	0	2	Grading crew
2 Culverts A	4	1	2	6	Culvert crew
3 Base A	1	2	6	7	Base crew
4 Surface A	3	3	7	10	Surface crew
5 Grading B	3	None	0	3	Grading crew
6 Culverts B	5	5	3	8	Culvert crew
7 Base B	2	6	8	10	Base crew
8 Surface B	1	7	10	11	Surface crew
9 Grading C	3	None	0	3	Grading crew
10 Culverts C	4	9	3	7	Culvert crew
11 Base C	2	10	7	9	Base crew
12 Surface C	1	11	9	10	Surface crew

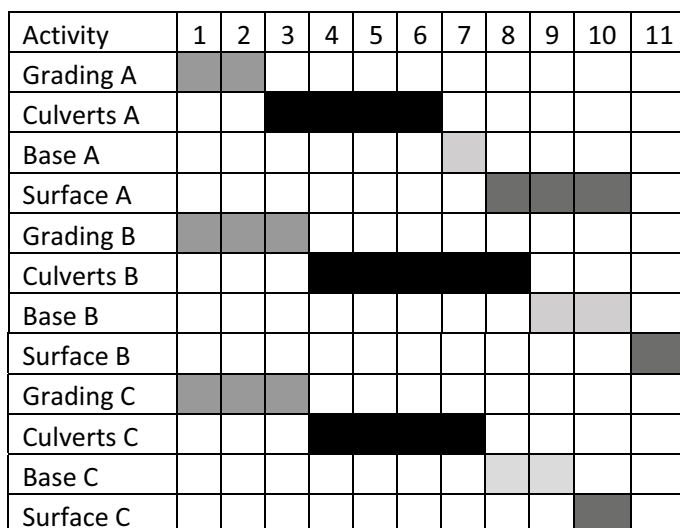


Fig. 7.8 Gantt chart for multiple projects

Table 7.11 Resource usage by crew

Crew	1	2	3	4	5	6	7	8	9	10	11
Grading	3	3	2								
Culverts			1	3	3	3	2	1			
Base							1	1	2	1	
Surface								1	1	2	1

work scheduled for a period, a peak occurs. There are valleys for the testing and installation crews. The only peaks occur for the grading and culvert crews.

The schedule with one crew of each type of work is shown in Fig. 7.9:

For all four crews, slack resources exist in that there are periods where no work is currently scheduled. This is because greater resource capacity exists than work required. Some spare resources are required to cover the inevitable uncertainty in projects. Efficiency, on the other hand, seeks to always utilize all resources at their capacity. Being careful to ensure that important project activities are not delayed for lack of resource, management may be able to increase efficiency by finding useful activities for resources during their unscheduled periods (or laying off crews when no work is to be done). One approach is investment in training and development activities. Another approach is to find more work. However, it is apparent here that there is also an imbalance across resources. If management were to obtain more work like what they have, the production crew would fall further and further behind, because their work per project is always the longest. The gaps for the testing crew, and especially for the installation crew, would get wider and wider. Sound management

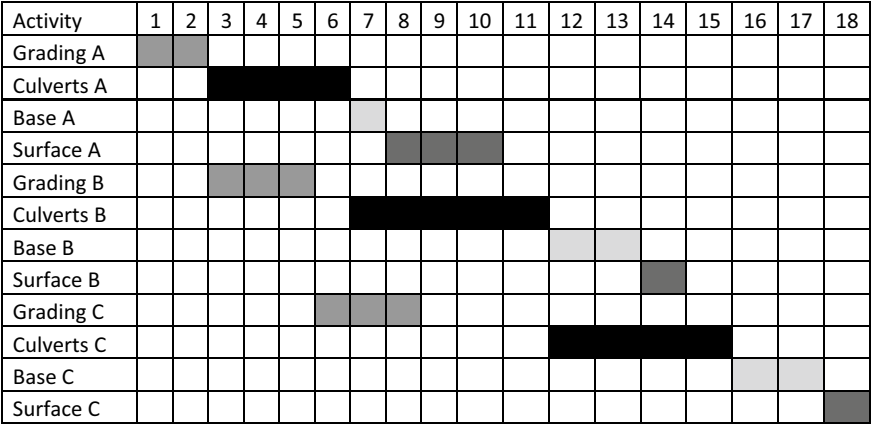


Fig. 7.9 Schedule with one crew of each type

practice here calls for balancing, through hiring additional production crews. If a great deal of similar work is obtained, more analysis crews might be called for as well. As a simple fix to this operation’s problem, we add a second culvert crew to accomplish culvert work on the second project in Fig. 7.10.

However, there is obviously more waste time that management needs to fill. Smoothing is a managerial decision. The ideal would involve balancing crew capacities so that each crew could work the same amount each time period. That is what assembly line operations do. That requires a highly predictable environment, which simply does not exist in projects. Projects will inevitably experience gaps in work,

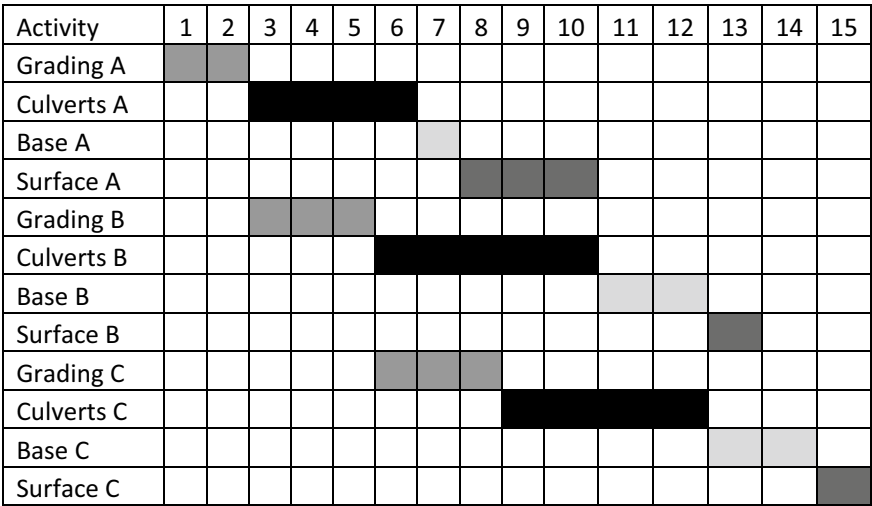


Fig. 7.10 Schedule with second culvert crew

along with some periods of critical resource shortage. The quality of project management can be measured on a scale of efficiency by the proportion of waste time encountered. However, priority should be given to ensuring that critical activities have the resources they need, even if it involves some waste. It is more important to get the job done right, on time, and within budget, than to obtain a perfectly smooth schedule.

7.7 Critical Path Criticisms

As with any model, the critical path approach makes several assumptions. Usually, the more convenient the assumptions are for mathematical solution, the less realistic the assumptions are relative to the real decision. The critical path model as demonstrated earlier is very useful in identifying the soonest one can expect to complete a project. But we have seen in previous chapters that things rarely proceed as planned and that in practice the original critical path plan usually requires significant modification as the project proceeds and more accurate activity durations are obtained. There is usually significant variance in possible durations of activities, with options available to speed selected activities. Crashing provides one means of obtaining greater understanding for these situations. One approach to uncertainty is to focus on scheduling milestones. Uncertainty in activity durations is addressed by Program Evaluation and Review Technique (PERT) models and simulation, but those get involved. Another key idea is that the critical path includes those activities that should be most closely managed. Critical activities are those that need close monitoring. But those activities with apparent slack can become critical and should not be ignored.

The critical path model assumes unlimited resources. We have discussed leveling as a means of analyzing the effects of limited resources. An additional extension of this concept is smoothing. Projects typically involve highly variable workloads, with employment starting at a very low volume, jumping to high levels in the middle of the project, and then in general declining. This overall trend masks even greater variations for the workload of specific skill sets, which can be highly erratic. Smoothing seeks to level the workload of the project, sacrificing minimum time for a more even workload. In information system projects, this is less important, as the usual condition is many projects going on at once, with more than enough work for individuals on other projects as soon as their work on this project is completed. The matrix form of organization, discussed in Chapter 3, is highly appropriate for this environment, allowing individuals to be shuffled into a particular project to perform their specialty and then moved on to the next project requiring their services.

The critical path model assumes that activities can be addressed as entities, with clear beginning and ending points. The content of complex projects changes over time. As new events unfold, project progress may require a change in direction. An obvious impact relative to information systems projects is the outcome of testing. According to the plan, testing will find that everything was successfully built as scheduled. The outcome of testing is highly uncertain. There is a possibility that testing will find that the system components do perform as designed. If they don't,

however, new activities will be required: to identify the cause of the problem, to decide how to revise the system, and quite possibly to build new system components.

7.8 Summary

The critical path method provides project managers with valuable information. The criticality of project activities can be identified, and those activities that are critical can be managed more closely. The critical path method provides an estimate of how long the project will take if everything takes no longer than estimated. The critical path method displays what can happen owing to predecessor relationships. Crashing provides a means of comparing the trade-off in cost and time in project scheduling. However, the critical path method assumes unlimited resources, which may not be realistic.

There are some things managers can do when facing resource limits. Buffers provide a way to better manage the chain of critical activities. Most project management software systems allow managers to level resources, generally following a priority system as we have outlined. While not guaranteed to provide the best solution, this ability of software packages to level resources is extremely useful.

Projects can also be leveled to stay within available resources and even smoothed to schedule resources more efficiently. Project management software is very good at accomplishing the task of leveling project activities to stay within prescribed resource levels. The solutions provided are not necessarily the best possible but are usually very good. The activity of smoothing is not as critical in project management, because efficiency is not as important as dealing with the high levels of uncertainty present in projects. Smoothing in uncertain environments is usually not effective.

The problems of managing multiple projects arise in most large organizations. The biggest difference in one large project versus many smaller projects is that in the large project, the project team can focus on attaining one set of objectives. In a multiple project environment, scarce resources need to be shared by many competing projects, and hard decisions need to be made to further overall organizational goals rather than focusing on the goals of specific projects.

Chapter 8

Probabilistic Scheduling



In a previous chapter, critical path models were discussed. The critical path method has been useful in project management planning and control. But it is widely recognized that the assumed durations of activities very often turn out to be different from those assumed in the planning stage. In this chapter, we will look at two types of project scheduling models that consider some of the uncertainty involved in project management. The Program Evaluation and Review Technique (PERT) assumes a beta distribution for project durations, which is not necessarily realistic. PERT can easily be implemented on a spreadsheet. Simulation can be used to model projects with uncertain durations on a spreadsheet. Any realistic distribution can be modeled using simulation. While more flexible than PERT, simulation analysis of projects involves more complications than PERT.

8.1 PERT

The PERT method is a modification of the critical path method, where uncertainty in activity durations can be considered. Three estimates of activity durations are required: minimum, most likely, and maximum. The most likely estimate is relatively easy. The minimum duration of an activity also might be easy to identify—the length of time the activity would take if everything went right. The third estimate, the maximum duration, is often difficult for estimators. The longest any activity can take is infinite, never finishing. However, the method won't work with that. What is meant by "maximum" is the activity duration if everything went wrong but the activity was completed regardless. When PERT is used, clear definitions of what is meant by minimum and maximum duration need to be specified. Many different variants are used.

Consider a problem of a marketing campaign consisting of six activities as shown in Table 8.1:

Table 8.1 Demonstration project input

Activity	Duration	Predecessors
A Product R&D	12 weeks	none
B Market test	6 weeks	A
C Pilot production	3 weeks	A
D Develop ad campaign	4 weeks	B
E Air advertising	3 weeks	D
F Begin distribution	1 week	C, D

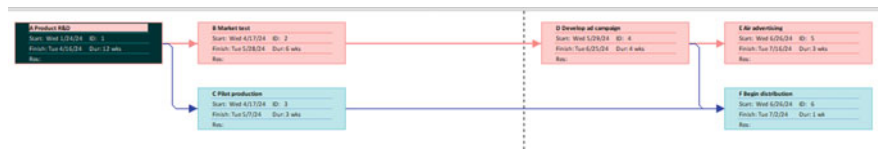


Fig. 8.1 Network for demonstration project

Figure 8.1 gives the network for this project. The critical path schedule would be as shown in Table 8.2 (asterisks are given for zero slacks, indicating critical activities):

The critical path in this case is the chain of activities A–B–D–E, a unique path, with an expected duration of 25 weeks. The PERT model is the same except that three duration estimates are used instead of one. Given these three estimates, the expected duration is given by the formula:

$$\text{expected duration} = \frac{a + 4m + b}{6}$$

where a is the minimum estimated duration, m is the most likely duration, and b is the maximum estimated duration. The variance of the duration of any activity is also a function of these estimates (calculations in Table 8.3):

Table 8.2 Critical path schedule for demonstration project

Activity	Early start	Early finish	Late finish	Late start	Slack
A	0	12	0	12	0*
B	12	18	12	18	0*
C	12	15	21	24	9
D	18	22	18	22	0*
E	22	25	22	25	0*
F	22	23	24	25	2*

*Asterisks are associated with zero slacks, indicating critical activities, as is stated.

$$\text{variance} = \left(\frac{b - a}{6} \right)^2$$

The variance of an activity with no uncertainty, such as for activity C, is 0. The expected duration is used as the duration in a critical path analysis (Table 8.4):

The variance of the project can be calculated as the sum of the variances of the chain of critical activities (Table 8.5):

This yields a project standard deviation (square root of the variance) of 1.509. Since the durations were beta distributed, the project duration is assumed to be normally distributed. The probability of completing the project on or before any particular target time can be calculated using the Z formula.

Table 8.3 Calculation of activity durations and variances

Activity	Min	Likely	Max	Expected duration	Activity variance
A Product R&D	9	12	18	12.5	81/36
B Market test	6	6	6	6	0
C Pilot production	3	3	5	3.33	4/36
D Develop ad campaign	4	4	5	4.167	1/36
E Air advertising	3	3	3	3	0
F Begin distribution	1	1	1	1	0

Table 8.4 PERT schedule for demonstration project

	Expected		Early	Early	Late	Late	
Activity	Duration	Predecessors	Start	Finish	Start	Finish	Slack
A	12.5	none	0	12.5	0	12.5	0*
B	6	A	12.5	18.5	12.5	18.5	0*
C	3.33	B	12.5	15.833	19.5	22.667	7
D	4.167	B	18.5	22.667	18.5	22.667	0*
E	3	D	22.667	25.667	22.667	25.667	0*
F	1	C,D	22.667	23.667	24.667	25.667	2

*Asterisks are associated with zero slacks, indicating critical activities, as is stated.

Table 8.5 Calculation of project variance

Activity	Maximum	Minimum	Variance
A	18	9	2.25
B	6	6	0
D	5	4	0.028
E	3	3	0
Project variance			2.278

Table 8.6 Probability calculations

Duration	Formula	Z	Probability
24	(24–25.667)/1.509	–1.105	0.135
25	(25–25.667)/1.509	–0.442	0.329
26	(26–25.667)/1.509	–0.221	0.587
27	(27–25.667)/1.509	0.883	0.811
28	(28–25.667)/1.509	1.546	0.939
29	(29–25.667)/1.509	2.209	0.986
30	(30–25.667)/1.509	2.871	0.998

$$z = \frac{x - \mu}{s}$$

To calculate the probability of finishing in x months, $z = (x - 25.667)/1.509$. The probability can be obtained by referring to a normal distribution or by using the NORM.S.DIST(z) function in Excel. Table 8.6 displays the probabilities of completing the project on or before the given duration:

In these calculations, the variance of activity C was disregarded, because it had slack. If activity C was to experience delay, while the other activities were closer to their expected durations, this could also delay the project. However, this possibility is disregarded with PERT. Here, activities B, E and F are estimated as constants, with no expected delay. They still could become critical if other activities were early for some reason.

8.2 Criticisms of PERT

PERT was developed in the 1950s for the U.S. Navy. It is theoretically attractive in that it addresses an obvious limitation of the critical path method. However, it is not widely used in practice, and when it is, the minimum durations are often casual estimates, such as one-half of the expected duration, and the maximum duration is stated as twice the expected duration. When this type of input is entered, there is obvious degradation of the quality of the analysis, with no thought given to specific activity variance. Microsoft Project uses the term PERT as a label for their network diagram. (Microsoft Project does provide support to apply PERT analysis, although not on its featured menu.)

PERT is biased toward giving overly optimistic schedule estimates owing to failure to consider time delays due to interaction of noncritical activities with critical activities. The beta distribution used in the PERT method is appropriate only in the symmetric case, with the most likely time the exact mean of the minimum and maximum times, when the most likely time is about 40 percent of the distance from the minimum to the maximum, and when the most likely time is about 60

percent of the distance from the minimum to the maximum. All other cases of the beta distribution lead to some error in the PERT duration calculation.

There is also a very real problem of obtaining multiple estimates of duration. Projects are one-time affairs, and there is not a lot of data on which duration estimates can be based. Estimators have a great deal of trouble developing the most likely duration estimate and quite often resort to quick-and-dirty estimates of the minimum and most likely estimates (such as the minimum = one-half of the most likely, and the maximum = twice the most likely).

The PERT method is also based on independence of activity durations. However, this is not true in projects, as if one activity is late, there is a tendency for management to rush the ensuing activities to compensate. This would result in a case of negative correlation between durations. There can also be similar underlying causes of lateness that might be positively correlated, such as skill shortages.

8.3 Simulation for Project Scheduling

Simulation is a very valuable tool for analyzing models involving elements described by probability distributions. Projects involve interrelated activities, many of which are probabilistic. This class of problems is very easily modeled on spreadsheets, such as Excel.

The marketing example scheduled earlier involved six activities, A through F. Assume that the durations of some of these activities, for instance A, B, E, and F, involve some uncertainty. The best way to proceed is to gather statistics on past projects (if possible) so that sound data can be used to estimate the expected durations and probability distribution for specific activities. For the purposes of demonstration, assume that the data for these four activities was found to be normally distributed (a possibility, although certainly not the only possible appropriate distribution).

The Excel CPM model is used as the basis for a simulation of this problem. We will use column A of a spreadsheet to list the six activities. Following the critical path spreadsheet model form from Chapter 6, we use column B for start times for each activity and column E for finish times. We reserve two columns (C and D) to generate simulated durations. We use the first of these columns (C) to generate a normally distributed random number and column D to convert the random numbers in column C to the appropriate outcomes reflecting simulated durations in column D.

In column B, the early start for any activities without predecessors is assigned the value of time 0. In this model, only activity A has no predecessor. Other cells in column B give maximum finish times values (column E) for all predecessors. Uncertain events are generated using random numbers in Excel. There are a number of ways to generate random numbers in Excel. The function: = **RAND()** returns a uniform random number between zero and one, which changes every time an action is executed in Excel. This uniform random number can be transformed into other distributions, but this operation requires more advanced study (see Evans and

Olson, 1998). A second way to generate random numbers within Excel is through the **Data Analysis Toolpak** on the **Tools** menu of the Excel command ribbon. From the Data Analysis Toolpak menu, select **Random Number Generation**. For normal distributions, it is even easier to generate a random number in one column (say column C) with the function = RAND() (cells C2, C4 and C6 in Table 8.7), and in another column (say column D) refer to that cell in = INT(norm.inv(mean, standard deviation), + 0.5).

Adding 0.5 in Column D is used to ensure that durations less than one time period are not included. Any minimum could be used for any activity. The formulation above yields the numbers in Table 8.8:

This provides more realistic scheduling simulations if complete time units are more appropriate.

Table 8.7 Excel spreadsheet of simulation model

Activity	Start	Random	Duration	Finish
A Product R&D	0	= RAND()	= INT(NORM.INV(C2,12.5,1.5) + 0.5)	= B2 + D2
B Market test	= E2		6	= B3 + D3
C Pilot production	= E2	= RAND()	= INT(NORM.INV(C2,3.333,0.333) + 0.5)	= B4 + D4
D Develop ad campaign	= E3	= RAND()	= INT(NORM.INV(C5,4.167,0.167) + 0.5)	= B5 + D5
E Air advertising	= E5		3	= B6 + D6
F Begin distribution	= MAX(E4,E5)		1	= B7 + D7

Table 8.8 Excel simulation output

Activity	Start	Random	Duration	Finish
A Product R&D	0	0.437	12	12
B Market test	12		6	18
C Pilot production	12	0.886	3	15
D Develop ad campaign	18	0.166	4	22
E Air advertising	22		3	25
F Begin distribution	22		1	23

8.3.1 Generating Multiple Simulation Runs

The benefit of simulation is that the experiment can be repeated many times so that patterns can be identified from the set of possible outcomes generated by the simulation. Multiple simulation runs can be obtained in a number of ways, four of which are discussed here. The hard way (1) is to generate results one by one. The easy way (2) is to use simulation add-ins, such as @RISK or CRYSTAL BALL. Within Excel, multiple simulations, each with different results, can be obtained using random numbers generated by the Data Analysis Toolpak (3), but this requires the use of VISUAL BASIC. A fourth procedure (4) works using the RAND() function, although random numbers cannot be controlled with this method.

In the short example last demonstrated, based on 100 runs, the mean project completion time was 26 months, with a minimum of 21 months and a maximum of 31 months. The distributions used in this case were all normal. The variance was 4.718, so there was more dispersion than there was in the PERT model. There may be even more variance (depending on the probability distributions used), as occasionally noncritical activities will become critical in the simulation. The standard deviation of the simulated project completion time was 2.174 weeks. Figure 8.2 displays results.

The percentile values for expected project completion time appeared to be symmetric, so we can apply the normal distribution, obtaining the following probabilities of project completion by the following times. The corresponding PERT probabilities are given for comparison in Table 8.9. The simulation has much more dispersion, owing to the higher variances of inputs used.

Table 8.9 data can be plotted in Fig. 8.3

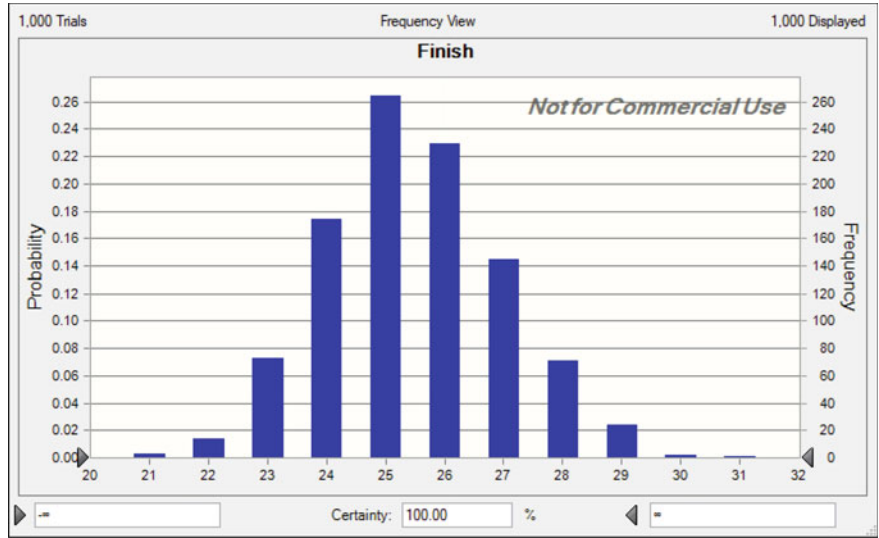


Fig. 8.2 Histogram of simulated project completion times

Table 8.9 PERT comparisons—cumulative probabilities

Months	Z	PERT probability	Simulation probability
< 21	$(21-25.667)/1.509 = -3.093$	0.001	0.003
< 22	$(22-25.667)/1.509 = -2.430$	0.008	0.017
< 23	$(23-25.667)/1.509 = -1.767$	0.038	0.090
< 24	$(24-25.667)/1.509 = -1.105$	0.135	0.265
< 25	$(25-25.667)/1.509 = -0.442$	0.329	0.264
< 26	$(26-25.667)/1.509 = 0.221$	0.587	0.757
< 27	$(27-25.667)/1.509 = 0.883$	0.811	0.903
< 28	$(28-25.667)/1.509 = 1.546$	0.939	0.974
< 29	$(29-25.667)/1.509 = 2.209$	0.986	0.997
< 30	$(30-25.667)/1.509 = 2.871$	0.998	0.999
< 31	$(31-25.667)/1.509 = 3.534$	1.000	1.000

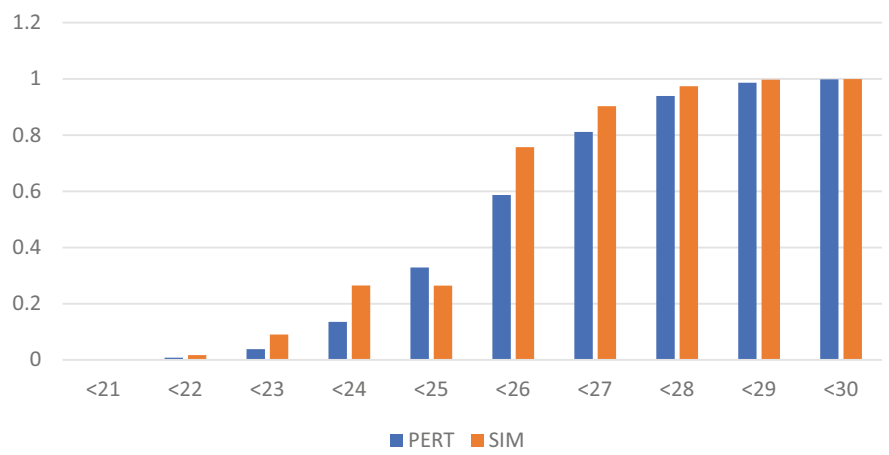


Fig. 8.3 Comparison of cumulative probabilities

Figure 8.3 shows similar results between PERT and Monte Carlo simulation. Simulation is a more complete model, and would also allow changing duration distributions, as well as allowing slack activities to become critical.

PERT addresses the widely recognized uncertainty involved in project management activities but makes a rigid assumption about the distribution of durations, and the calculation of probability of completion by a specified time disregards noncritical activities. Simulation provides a flexible means to evaluate the probability of projects being completed by specific times. Any distribution of duration can be assumed. The distribution used should be based on empirical data if possible. All activity paths are considered in the simple spreadsheet network. For instance, observed data may not

be symmetric. The triangular distribution might provide a better fit to such data than does the normal distribution.

8.4 Summary

Project management techniques have proven very useful in the planning and control of interrelated activities, but the durations of projects is notoriously variable. CPM and PERT are very useful in analyzing the interrelationships among activities, but CPM assumes constant durations, and while PERT allows a specific type of duration distribution, this particular distribution is not always appropriate, and PERT includes other inaccuracies.

Simulation of project management problems is much more flexible. Whatever distributions of expected times are appropriate can be entered into a simulation model. Simulation analysis is very flexible as well. Analysis in much greater detail is possible with simulation than is possible with PERT, although simulation requires more in the way of statistical analysis to interpret output. The mean values of simulation output provide the same information as CPM, and a more complete probabilistic output is obtained with simulation than is available from PERT.

Chapter 9

Critical Chain Scheduling



Critical chain project management (CCPM) has become a research area, applying the ideas developed by Goldratt.¹ The gist is that care should be applied to bottleneck activities (critical activities) and buffers applied to obtain assurance of completion of those critical activities on time.

9.1 Buffers

The primary means to ensure that critical activities are completed on time is to use buffers. A buffer is time that is included in the schedule to protect against unanticipated delays and to allow early starts. Buffers are different from slack. Slack is spare time. Buffers are time blocks that are not expected to be used for work time (the same as slack) but are dedicated to cover most likely contingencies and are closely watched so that if they are not needed, subsequent activities can proceed at the earliest time possible. **Project buffers** are used after the final task of a project to protect project completion time from delays. **Feeding buffers** are placed at each point where a noncritical activity is related to a critical path activity. Feeding buffers protect the critical activities from tasks that precede them and allow for early starts of critical activities. **Resource buffers** are placed before resources that are scheduled to work on critical activities to ensure that resources will be available and that their shortage will not delay critical activities. Resource buffers can be implemented by either issuing warning notices to those managing the resource ahead of the required time or scheduling a time to mobilize the resource as a predecessor to the critical activity using it. In multiple project environments, **strategic resource buffers** can be used to ensure that key resources are available for critical activities.

9.2 Critical Chain Process

We draw on Ellis² to describe the implementation of CCPM. The idea is to increase deficiency by minimizing multitasking and procrastination to ensure tasks are ready to start, focus on cooperative teamwork, and generate a common focus on critical tasks. The method uses what they call “aggressive duration estimates.” (They assume that estimators provide conservative duration estimates, at roughly the 90th or 95th percentile of expected, which leads to waste.) The idea is that the critical chain of activities will have components that are sometimes early, sometimes late, and, rather than use the late assumption for each activity, use the most likely and add a project buffer at the end to represent project variance.

This is problematic. As someone who spent a decade estimating construction projects, and studying delays thoroughly, I would dispute that you can assume this inflation on a regular basis. Duration estimates are biased, but the more experience the organization has with an activity, the less variance. That’s in construction. In information systems projects, I have had a very good systems engineer tell me that when he is ever asked how long a programming activity will take, he responds, “As long as it takes to do it right.” Thus, the CCPM method starts with a gross assumption that makes it look like it saves a great deal of time but that I would argue leads to very dangerous scheduling. Furthermore, the assumption that there will be compensating early finishes to balance some of the late finishes is also problematic, as correlation often appears in projects.

Once a critical path network is developed, with estimated durations and resource requirements provided, the critical path is identified. Then CCPM involves the following steps:

Step 1: Remove resource constraints

All tasks using the same resource are staggered to avoid overtaxing constrained resources.

Step 2: Reduce task duration by changing certainty from 90 to 50%

Since the 95th percentile in a normal curve is 1.96 standard deviations from the mean, divide the original duration estimates by 2.

Step 3: Add feeder buffers

Ensure noncritical activities don’t delay the critical path by adding a buffer to interfaces between them. Common buffers are 50% of the non-critical activity feeding into the critical activity.

Step 4: Add resource buffers

Ensure constrained resources are available. Typical buffers are 50% of the constrained resource duration.

Step 5: User the relay-race mentality

When noncritical tasks are scheduled, delay them to start as late as possible (considering feeder buffers).

Step 6: Add a project buffer

At the end of a critical chain, add a buffer (50% is popular) to reflect project variance.

If Step 6 is done, on average the CCPM duration will be about 75% of the traditional critical path (durations are divided by 2; then the overall critical path duration is multiplied by 1.5). This is supposed to show improved efficiency. This requires the gross assumptions discussed earlier—not all projects have the same variance. (Although CCPM proponents would rightly argue that you can adjust the project buffer accordingly, dividing the durations by two is undeniably gross). The argument is that there are behavioral biases of estimators leading to duration inflation that is counterproductive (and there is admittedly some truth to that). The student syndrome refers to a tendency among students to put off working on assignments until the last minute. Parkinson’s Law is that the time taken to complete a task will tend to equal the allowed time. There is also the project feature of predecessor relationships whereby finishing early has no value (if you finish early, you just have to wait for the others).

To demonstrate the use of buffers, consider a road construction project using a million-dollar milling machine for three projects (A, B & C). The schedule consists of the activities given in Table 9.1.

At this stage, using the modes the critical path is CP-CM-CF (planned duration 40 weeks). However, there may be ways to accomplish specific tasks at the cost of some extra effort or expenditure.

The second step in critical chain scheduling is to remove resource constraints. The milling machine is overbooked in weeks 5 through 8, needed for both project A and B. Milling for B will be delayed until weeks 9 through 12. Removing this overscheduling results in the schedule shown in Table 9.2.

Step 3 is to reduce durations by 50%. Results are shown in Table 9.3.

Here the critical path is CP-CM-CF. Feeding buffers can be used as pseudo activities to make sure that critical activities are not delayed. Feeding buffers might be 50% of estimated duration of the feeding activity, which in Table 9.3 is activity BM feeding into activity CM. Feeding buffers are applied at the end of the preceding activity, demonstrated in Table 9.4.

Table 9.1 Project activities

Code	Activity	Min	Mode	Max	Predecessors	ES	EF
AP	Preliminary A	2 wks	4 wks	7 wks	None	0	4
AM	Mill A	3 wk	4 wks	6 wk	AP	4	8
AF	Finish A	5 wk	8 wks	14 wk	AM	8	16
BP	Preliminary B	3 wk	4 wks	6 wk	None	0	4
BM	Mill B	3 wk	4 wks	6 wk	BP	4	8
BF	Finish B	8 wk	12 wks	18 wk	BF	8	20
CP	Preliminary C	14 wk	20 wks	28 wk	None	0	20
CM	Mill C	3 wk	4 wks	6 wk	CP	20	24
CF	Finish C	8 wk	16 wks	26 wk	CM	24	40

Table 9.2 Project activities after removing resource constraints

Code	Activity	Duration	Predecessors	ES	EF	Slack
AP	Preliminary A	4 weeks	None	0	4	8
AM	Mill A	4 weeks	AP	4	8	8
AF	Finish A	8 weeks	AM	8	16	24
BP	Preliminary B	4 weeks	None	0	4	8
Res	Resource constraint	4 weeks	BP	4	8	8
BM	Mill B	4 weeks	Res	8	12	8
BF	Finish B	12 weeks	BM	12	24	16
CP	Preliminary C	20 weeks	None	0	20	0
CM	Mill C	4 weeks	CP	20	24	0
CF	Finish C	16 weeks	CM	24	40	0

Table 9.3 Schedule after duration reduction

Code	Activity	Duration	Predecessors	ES	EF	Slack
AP	Preliminary A	2 weeks	None	0	2	4
AM	Mill A	2 weeks	AP	2	4	4
AF	Finish A	4 weeks	AM	4	8	12
BP	Preliminary B	2 weeks	None	0	2	4
Res	Resource constraint	2 weeks	BP	2	4	4
BM	Mill B	2 weeks	Res	4	6	4
BF	Finish B	6 weeks	BM	6	12	8
CP	Preliminary C	10 weeks	None	0	10	0
CM	Mill C	2 weeks	CP	10	12	0
CF	Finish C	8 weeks	CM	12	20	0

Table 9.4 Model after feeding buffer

Code	Activity	Duration	Predecessors	ES	EF	Slack
AP	Preliminary A	2 weeks	None	0	2	4
AM	Mill A	2 weeks	AP	2	4	4
AF	Finish A	4 weeks	AM	4	8	12
BP	Preliminary B	2 weeks	None	0	2	4
Res	Resource constraint	2 weeks	BP	2	4	4
BM	Mill B	2 weeks	Res	4	6	4
FB	Feeding buffer	1 week	BM	6	7	7
BF	Finish B	6 weeks	BM	7	13	7
CP	Preliminary C	10 wks	None	0	10	0
CM	Mill C	2 weeks	CP	10	12	0
CF	Finish C	8 weeks	CM	12	20	0

It is important that if the buffer is not needed, subsequent activities should proceed to work early. This buffer, like all other buffers, should not be used unless necessary in light of sound management principles. Resource buffers could be used if a key resource, such as the milling machine, was scheduled on multiple activities. They are usually 50% of the activity that uses the resource's duration. Resource buffers are applied before the resource use, shown in Table 9.5.

The next step is to apply the relay race concept, delaying feeding buffers activities with slacks. This yields Table 9.6.

The final step of the critical chain method is to add a project buffer equal to 50% of expected project duration, here amounting to 10 weeks. Estimated duration has decreased from 40 to 30 weeks.

The Program Evaluation Review Technique (PERT) schedule would have had a project duration of 40.833, a bit more than this schedule, which is what critical chain advocates argue you should expect—a shorter schedule with just as much if not more control. From the data in Table 9.1, we calculate a project variance for critical path CP-CM-CF of 14.6944, yielding a standard deviation of 3.833. Using this PERT data, the probability of finishing in 30 weeks would be:

$z = (40.833 - 30)/3.833 = 2.695$	Probability $\{z = -2.695\} = 0.002$
-----------------------------------	--------------------------------------

We could also use the minimums, in line with the idea that close managerial control might speed up activities. The PERT minima yields a project completion of 25 weeks.

$z = (40.833 - 25)/3.833 = 4.131$	Probability $\{z = 4.131\} = 0.000$
-----------------------------------	-------------------------------------

Table 9.5 Model after resource buffers

Code	Activity	Duration	Predecessors	ES	EF	Slack
AP	Preliminary A	2 weeks	None	0	2	2
AM	Mill A	2 weeks	AP	2	4	2
AF	Finish A	4 weeks	AM	4	8	12
BP	Preliminary B	2 weeks	None	0	2	2
Res	Resource constraint	2 weeks	BP	2	4	2
RB	Resource buffer	1 week	AM	4	5	2
BM	Mill B	2 weeks	Res	5	7	2
FB	Feeding buffer	1 week	BM	7	8	2
BF	Finish B	6 weeks	BM	8	14	6
CP	Preliminary C	10 weeks	None	0	10	0
RB	Resource buffer	1 week	BM	9	10	0
CM	Mill C	2 weeks	CP	10	12	0
CF	Finish C	8 weeks	CM	12	20	0

Using the mode would yield a project completion of 40 weeks.

$z = (40.833 - 40)/3.833 = 0.087$	Probability $\{z = 0.087\} = 0.413$
-----------------------------------	-------------------------------------

These three calculations show how sensitive the probability is to the data. In this case, the critical chain schedule would be very ambitious. But the idea is that project management can focus on control to reduce durations. If they could effectively control the project, they would have a pretty good probability of finishing on time. If they let the project follow its course, they could be in trouble. The practicality of that idea depends a great deal on the ability to control variances. We compare that with PERT, which makes a great many assumptions of its own and in practice probably tends to inflate schedules more than needed.

9.3 Fever Charts

The purported value of CCPM is in project control, using fever charts to monitor progress by focusing on the remaining project buffer. The key to fever charts is to compare progress against two limits. A safe region might be using less than 80% of the project buffer available compared with critical path complete. A danger region might be use of over 110% of available project buffer compared with critical path complete. In the safe region, buffer penetration is low, and there is no perceived need to interfere with project progress. In the danger region, action is called for

Table 9.6 Model after relay race concept

Code	Activity	Duration	Predecessors	ES	EF	Slack
AP	Preliminary A	2 weeks	None	0	2	2
AM	Mill A	2 weeks	AP	2	4	2
AF	Finish A	4 weeks	AM	4	8	12
BP	Preliminary B	2 weeks	None	0	2	2
Res	Resource constraint	2 weeks	BP	2	4	2
RB	Resource buffer	1 week	AM	4	5	2
BM	Mill B	2 weeks	Res	5	7	2
FB	Feeding buffer	1 week	BM	9	10	2
BF	Finish B	6 weeks	BM	8	14	6
CP	Preliminary C	10 weeks	None	0	10	0
RB	Resource buffer	1 week	BM	9	10	0
CM	Mill C	2 weeks	CP	10	12	0
CF	Finish C	8 weeks	CM	12	20	0

(motivation, crashing, or contacting project owner). In between, the project manager should consider contingent action plans if more delays are encountered.

A key calculation in a fever chart is to compare critical chain completed (equivalent to earned weeks) with time passed. The fever chart focuses on buffer penetration, which is defined as time passed minus critical chain completed. Here the critical chain is CP-CM-CF. The focus is to compare buffer penetration into three areas—if buffer penetration is below an OK line, which is sometimes set at 80% of the planned buffer use of the critical chain path, all is well. If buffer penetration is above a Danger line, sometimes set at 110% of the planned buffer use of the critical chain path, drastic action is called for. Between these limits, project management is encouraged to be watchful and proactive. Assume the most likely estimated durations in our example (Table 9.1). The project using the modes took 40 weeks, 10 weeks over the CCPM estimate. Slack activities don't impact the fever chart. (They could disrupt project completion, however). Proportions in Table 9.7 are the proportion of the critical path earned based on CCPM estimated durations.

In Table 9.7, the mode durations from Table 9.1 are used. Here, the buffer generated by the critical chain schedule assuming sped-up durations is steadily used up, exhausted in week 20 when the project was 50% complete. This is what would happen if no corrective action were taken to bring things back on schedule (through motivation or crashing or whatever means exist to speed progress in the lagging activity). Figure 9.1 displays these results:

Figure 9.1 shows that the project was close to the danger buffer limit throughout the project. Table 9.8 shows results using the MIN values from Table 9.1.

These results are plotted in Fig. 9.2.

In this case, while the project extended beyond the estimated 20 weeks, it was well within the buffer of 30 weeks, and never got in the OK zone let alone the danger zone. Conversely, Fig. 9.3 demonstrates the results of a project with some variance:

Projects often start with optimistic estimates of progress. Here all looks well through week 7. In weeks 7–11 there is some slippage, but below the OK line. In week 12 there is some warning of losing control. The 10 weeks of buffer are used up by week 21. Slippage gets worse through week 25.

9.4 Summary

The critical chain idea is good enough in that there is a bias toward inflation of duration estimates. However, the solution the CCPM comes up with often ends up putting the padding back. And it is a gross assumption that project estimates are double the expected. The more experience and the less risk, the lower this biased duration inflation would be. Dividing durations by two in that case would be very dangerous.

The idea of the fever chart could be a useful means of monitoring project progress. However, there is a well-known tendency for projects to appear to be fine until the

Table 9.7 Fever chart elements using mode

Week	Activity	Critical path progress	Buffer used	OK	Danger
1	CP	0.5	0.5	0.4	0.55
2	CP	1	1	0.8	1.1
3	CP	1.5	1.5	1.2	1.65
4	CP	2	2	1.6	2.2
5	CP	2.5	2.5	2	2.75
6	CP	3	3	2.4	3.3
7	CP	3.5	3.5	2.8	3.85
8	CP	4	4	3.2	4.4
9	CP	4.5	4.5	3.6	4.95
10	CP	5	5	4	5.5
11	CP	5.5	5.5	4.4	6.05
12	CP	6	6	4.8	6.6
13	CP	6.5	6.5	5.2	7.15
14	CP	7	7	5.6	7.7
15	CP	7.5	7.5	6	8.25
16	CP	8	8	6.4	8.8
17	CP	8.5	8.5	6.8	9.35
18	CP	9	9	7.2	9.9
19	CP	9.5	9.5	7.6	10.45
20	CP	10	10	8	11
21	CM	10.5	10	8	11
22	CM	11	10	8	11
23	CM	11.5	10	8	11
24	CM	12	10	8	11
25	CF	12.5	10	8	11
26	CF	13	10	8	11
27	CF	13.5	10	8	11
28	CF	14	10	8	11
29	CF	14.5	10	8	11
30	CF	15	10	8	11
31	CF	15.5	10	8	11
32	CF	16	10	8	11
33	CF	16.5	10	8	11
34	CF	17	10	8	11
35	CF	17.5	10	8	11
36	CF	18	10	8	11
37	CF	18.5	10	8	11

(continued)

Table 9.7 (continued)

Week	Activity	Critical path progress	Buffer used	OK	Danger
38	CF	19	10	8	11
39	CF	19.5	10	8	11
40	CF	20	10	8	11

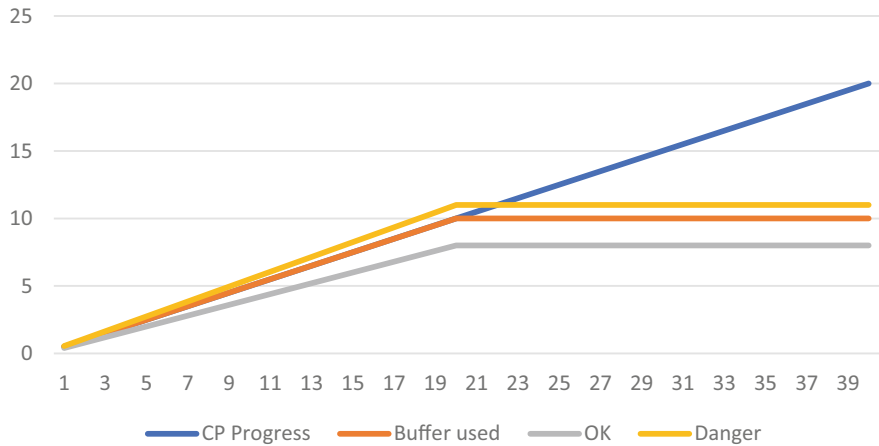


Fig. 9.1 Fever chart for modes

bitter end, when unexpected delays are encountered. In that case, the fever chart would appear to be unable to identify trouble until too late.

Table 9.8 Fever chart elements using MIN values

Week	Activity	Critical path progress	Buffer used	OK	Danger
1	CP	0.714	0.286	0.4	0.55
2	CP	1.428	0.572	0.8	1.1
3	CP	2.142	0.858	1.2	1.65
4	CP	2.856	1.144	1.6	2.2
5	CP	3.5	1.5	2	2.75
6	CP	4.214	1.786	2.4	3.3
7	CP	4.928	2.072	2.8	3.85
8	CP	5.642	2.358	3.2	4.4
9	CP	6.356	2.644	3.6	4.95
10	CP	7.07	2.93	4	5.5
11	CP	7.785	3.215	4.4	6.05
12	CP	8.5	3.5	4.8	6.6
13	CP	9.215	3.785	5.2	7.15
14	CP	10	4	5.6	7.7
15	CM	10.67	4.33	6	8.25
16	CM	11.33	4.67	6.4	8.8
17	CM	12	5	6.8	9.35
18	CF	13	5	7.2	9.9
19	CF	14	5	7.6	10.45
20	CF	15	5	8	11
21	CF	16	5	8	11
22	CF	17	5	8	11
23	CF	18	5	8	11
24	CF	19	5	8	11
25	CF	20	5	8	11

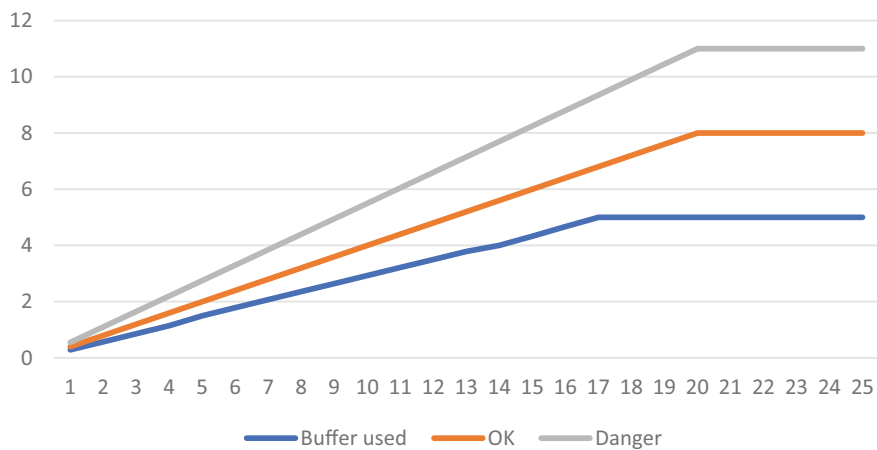


Fig. 9.2 Fever Chart for MIN durations

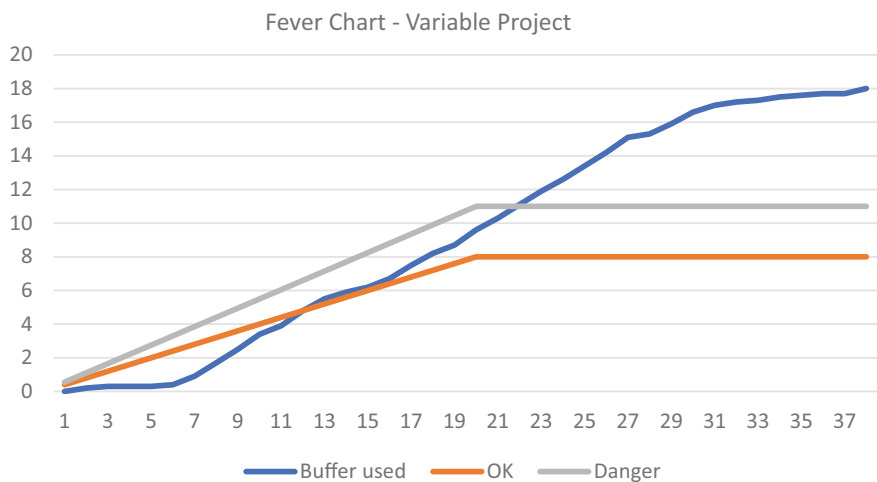


Fig. 9.3 Fever chart for MAX durations

Notes

1. Goldratt (1997).
2. Ellis (2016).

References

- Ellis G (2016) Critical Chain Project Management (CCPM). In: Project management in product development: leadership skills and management techniques to develop great projects. Butterworth-Heinemann, Oxford, England
- Goldratt EM (1997) Critical chain. North River Press, Great Barrington, MA

Chapter 10

Agile Project Development



A response to the poor performance record of information systems with respect to time, budget, and functionality in the software industry was the development of collaborative techniques to develop software much faster and closer to user requirements. These techniques are known as agile methods, and include a number of versions (scrum, extreme programming, lean techniques). However, the common bases for these techniques include working closely with users throughout the project, iteratively develop software versions in the spirit of prototyping, demonstrating these prototypes with users to identify required changes in direction, and avoiding heavy documentation in favor of getting software developed faster. While agile techniques might seem the reverse of the Capability Maturity Model approach, Level 5 software producers have been found to use agile techniques when faced with stiff schedules and budgets for important projects.

This chapter provides a description of agile approaches to project development. It discusses related systems to include scrum and lean techniques. It also presents crashing scheduling techniques.

10.1 PMI View of Agile

The Project Management Institute (PMI) publishes its Project Management Body of Knowledge (PMBOK) regularly, seeking to standardize project management concepts and practice. The PMI views a project as a temporary group of activities designed to produce a unique product, service, or result. PMBOK provides rules, guidelines, and characteristics for project management, viewing project management as a process divided into the groups of:

- Initiation—define and authorize the project
- Planning—define project scope, and plan needed actions to reach the goal
- Execution—train people to follow the project management plan

- Monitoring and control—during project evolution, identify needed corrective actions
- Closing—formal acceptance by owner and project conclusion

These process groups are supported by PMBOK knowledge areas:

1. Project integration management—activities needed to coordinate a group of processes and their activities
2. Project scope management—activities guaranteeing that the project covers all planned elements
3. Project time management—activities guaranteeing project conclusion in planned time
4. Project cost management—plan, estimation, budgeting, and control of costs to accomplish the project
5. Project quality management—determination of responsibilities, objectives, and quality policies
6. Project human resource management—management of the project team
7. Project communication management—generation, collection, distribution, storage, and recovery of data
8. Project risk management—identification, monitoring, analysis, and proposed solutions to manage risks
9. Project procurement management—purchase and supply of products and services required

The PMBOK seeks to provide a general structure applicable to all projects. In practice, this can be found to be bureaucratic and rigid. Agile approaches were intended to overcome some of these limitations, although it must be noted that formal approaches do have benefits in well-established project environments where change is less common.

10.2 Agile Definition

Agile is a project management ideology providing a lightweight alternative to the waterfall approach of systems analysis and design. Agile was intended to be more responsive, relying on close working relationships among teams dealing with specific development components.

Agile methods have proven very effective when applied to small projects. They also work (with care) on larger projects, although effort is needed to get teams to work well together. The best case is when a small, self-organized and motivated team located in the same location works with a small number of on-site customers. There are even cases when teams have worked in a geographically separated environment, although synchronization of efforts is required.

Differences between agile and traditional (waterfall model) systems development are demonstrated in Table 10.1.

Table 10.1 Comparison of agile with traditional project development¹

Factor	Traditional model	Agile
Management style	Process (command and control)	People (leadership/collaboration)
Assignments	Individual	Team
Communication	Formal (hierarchical)	Self-organizing (open)
Development model	Lifecycle	Evolutionary iterative
Organizational structure	Mechanistic (matrix, project)	Organic (participative team)
Team location	Often distributed	Collocated
Documentation	Substantial—before project start	Less—interactive input during project
Resource utilization	Optimize plan	Ad hoc
Customer involvement	Requirement analysis	Continuous
Cost estimation	Initial	Guess in each sprint
Quality control	At end of phases	Continuous
Contractual issues	Clear initial definition	Estimates evolve

Agile project management practices include:

- Focus on product vision
- Use of simple project planning and communication tools and processes

The traditional approach uses Gantt charts or work breakdown structure as opposed to less formal tools like spreadsheets or visual panels, pictures, or drawings.

- Use of iterative planning

The traditional approach uses more detailed project planning.

- Frequent monitoring of planning and updating processes

The traditional approach applies detailed planning with revision while agile firms develop by iteration.

- Activities developed (and monitoring and updating) by self-managed teams

Traditional firms update the project plan at the end of each phase. Agile firms update weekly.

The economy in recent years has had a major impact on the software industry. Because of the increased importance of bringing software products to market faster, revision of traditional methods has been apparent. Agile methods are techniques of software development designed to deliver these products faster, with more functionality necessary to users.

There are many different techniques that have been used in this effort to attain agile programming. These include collaborative techniques (pair programming, having customers work on site as team members). What this amounts to is working very

closely with customers to ensure that the software products developed are what customers need. For instance, a common agile practice is to have customers write acceptance tests.

This approach emphasizes:

- Focus on individuals and their interactions over focus on processes and tools.
- Delivering working software is more important than comprehensive documentation.
- Customer collaboration is sought rather than contract negotiation.
- Response to change is preferred to the following plans.

This manifesto clearly diverges from the principles of the Capability Maturity Model. However, the agile approach maximizes cooperation with users that has been found to be so critical to information system project success. The principles behind the agile movement are:

- Highest priority is given to satisfying the customer through early and continuous delivery of valuable software.
- Requirement changes are welcomed, even late in the development process. Agile processes harness change with the aim of improving customer competitive advantage.
- Working software is delivered frequently, from a couple of weeks to a couple of months, preferably to shorter timescales.
- Cooperative work among businesspeople and developers is needed on a daily basis.
- Projects are built around motivated individuals, who operate in a positive environment with all the support they need, and who are trusted to get the job done.
- Face-to-face conversation is relied upon as the most efficient and effective method of communication.
- The primary measure of progress is working software.
- Agile processes promote sustainable development.
- Technical excellence and good design are sought continuously, enhancing agility.
- Simplicity is essential, seeking to maximize the amount of work not done.
- Self-organizing teams develop the best architecture, requirements, and designs.
- Teams reflect on ways to become more effective at regular intervals, and tune and adjust behavior accordingly.

Agile emphasizes team performance. Thus, attention should be given to team composition and building teams that work well together. Therefore, conflict resolution is important. The method emphasizes working together and collocation, diverging from a recent tendency in information technology of distributed teams.

Stakeholder engagement is also important, with focus on the total quality management concept of delighting customers. This requires a thorough understanding of stakeholder needs to build a shared vision across the development team. Creativity needs to be fostered, along with strong communication with stakeholders by applying critical interpersonal skills.

Agile calls for progressive elaborations of project development. Problems, risks, threats, and issues need to be uncovered and understood to develop sound solutions. Decisions should be made at the last responsible moment to consider all important factors.

The outcome of agile project management includes the project and team charter, backlog preparation and refinement, and activities to include stand-up, retrospective, and demonstration reviews. Performance needs to be measured to enable troubleshooting. Agile provides a means to intensively develop products and is useful when creativity needs to be emphasized and time is a factor. Traditional approaches work better for larger-scale projects.

10.3 Scrum

The scrum approach is one form of agile development approach, focusing on performing work in short iterations, usually from a week to a month. During these periods (time-boxes), sprints occur where a cross-functional team performs all tasks (such as design, building, and testing) needed to complete a working feature. Scrum teams usually involve 6 to 10 members providing cross-functionality. One team member is the product owner representing the stakeholders. There also is a Scrum-Master, a coach, facilitator, and remover of impediments. Each sprint involves meetings, to include planning (identification of issues and their prioritization), stand-ups (daily meetings where team members describe what they have accomplished and plan on doing next), reviews, and retrospectives.

The relative advantages of agile and traditional software development approaches (waterfall) have been compared.² The traditional approach was perceived as ineffective in software project management, as project releases were often late as well as failing to meet client requirements. Part of the problem is that clients have minimal interaction with the project team, at which time the clients often do not have a clear idea of what they want. Agile software development follows a project plan, but without great initial detail. Plans are designed in stages iteratively. The focus is on completing projects faster and at a lower cost.

The scrum methodology is structured around the components of **roles, artifacts, and processes**:

Roles—the three key roles are the scrum master, the product owner, and the scrum team. The scrum master provides loose team leadership and interacts with the environment. The product owner sets the scope of the project and prioritizes product requirements, constantly refined as the project progresses. The scrum team is self-organized and builds the product in sprints (iterations).

10.3.1 Artifacts

The product backlog consists of prioritized project requirements and provides a road map. After the product backlog is completed, the scrum team breaks it into sections of work, creating the sprint backlog. Progress is constantly monitored, and time estimates are updated daily. These estimates can be plotted, creating a burn-down chart, showing daily estimates of time remaining. User stories are stored as sprint backlog items. These stories consist of an ID, name, importance, initial estimate, demo procedure, and notes. User stories are sorted into smaller tasks for teams to complete.

10.3.2 Processes

1. Kick-off—project launch
2. Sprint planning meeting—determine sprint backlog and objectives—all attend
3. Daily scrum meeting—Scrum master and team spend about 15 min to track progress and identify issues and delays
4. Sprint review meeting—held after a sprint to demonstrate functionalities—all attend
5. Retrospective meeting—after sprint review meetings, scrum master and team identify problems experienced and evaluate backlog, as well as develop ways to improve future sprints
6. Interruptions—lead to requirements changes

Agile has been proposed for smaller firms. Scrum has been proposed as workable for large firms as well. Scrum has project risk management features, with product owner meetings providing a forum for risk consideration in backlog development, while daily team communication, scrum meetings, and sprint planning give means to identify and mitigate risks.

10.3.3 Kanban

The kanban approach has been applied in lean manufacturing for decades (Toyota factories in the 1940s and thereafter), credited with improving quality by immediate detection and correction of problems, with residual benefits of reducing clutter by controlling work in process (WIP) and inventory by ordering materials to arrive just in time. Kanban is another agile methodology that has been applied to software development projects (and by inference can be applied to other projects. In kanban, roles are not prescribed. Signals of work process are used to limit WIP.

Workflow visualization—work is divided into specific tasks, visualized in columns on a board, indicating its state in the workflow

Limiting WIP—explicit limits to the number of tasks are defined for each workflow state

Measuring lead time—defined as the time from task creation to completion—measured to optimize workflow and to make it as fast and predictable as possible

Table 10.2 shows a comparison of scrum and kanban agile methodologies:

Scrum splits work into small tasks, accelerating completion, and major backlog items, making it suitable for software projects. Kanban splits work into even smaller items to ensure a continuous work process, not using the concept of sprints. Both methodologies work on items by prioritizing, allowing product release immediately upon completion. This makes the kanban approach highly suitable for customer service environments.

Agile practices provide a means to rapidly develop projects, providing a platform conducive to creativity. It works if teams work closely together in a spirit of collective ownership. Team leaders need to develop and maintain a close collaborative working environment. The process includes daily meetings with a demonstration of features emphasized, and documentation delayed until the product takes firm form. Agile needs full stakeholder participation and operates in the spirit of just in time.

Table 10.2 Comparison—Scrum and Kanban

	Scrum	Kanban
Iterations	Sprints	None
Product release	At end of each sprint, at owner discretion	Constant development
Teams	Cross functional	Specialized
Roles	Product owner, Scrum master, team	None prescribed
Change management	Changes at the beginning of new sprints	Prompt response to any change
Meetings	Planned	None required
WIP	Limited per sprint	Limited per workflow state
Product backlog	Listed prioritized items	Board cards
Visualization	Board, product backlog, sprint backlog	Board to visualize the process
Bottleneck management	Addressed immediately	Avoided

10.4 Project Crashing

Project crashing involves application of logic to consider alternative ways to accomplish a task at different costs.

Another possibility is that extra resources can be acquired to complete critical activities quicker. This creates a problem for management, trading off quicker completion time with higher cost. If the savings from faster completion time are known, then the problem is simply one of minimizing cost. Often, however, it is a matter of risk in that you know some activities are subject to delay, and earlier planned finishing times are far safer than those plans that push things to some deadline.

To demonstrate crashing, consider the problem of installing a robotic system as part of a business process reengineering activity for a small manufacturing firm. The activities are an initial study of the existing system, followed by receiving bids from three vendors with time and dollar estimates. Once a system has been selected, there will be installation in parallel with training development as well as software development. Training can be expedited for one month by hiring a consulting firm for \$500,000. Software can be expedited from 3 to 2 months at an additional cost of \$100,000. Bid selection, training, and software will be followed by a pilot operation activity, concluding events until full operation. Figure 10.1 gives a network for this project. The new system is expected to save the firm \$200,000 per month.

The bid options are:

McKenzie's bid is for \$6,100,000, with installation time of 5 months.

The Speedo bid for \$6,200,000 has installation time of 4 months.

The Dynamo bid is for \$6,500,000 with installation time of 3 months.

The critical path calculations for the McKenzie option with normal training are shown in Table 10.3.

This option would cost \$6,100,000 for installation and start production in 13 months, at a cost of \$2,600,000 in lost production, for a total of \$8,700,000.

Options available to speed installation to 12 months are:

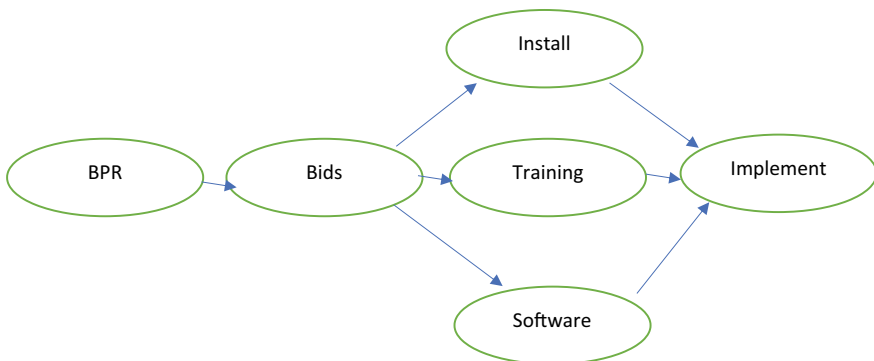


Fig. 10.1 Network for robotics project

Table 10.3 Slack calculations

Activity	Duration (months)	Pred	Early start	Early finish	Late start	Late finish	Slack
A BPR	5	None	0	5	0	5	0
B Bids	2	A	5	7	5	7	0
C Install	5	B	7	12	7	12	0
D Training	3	B	7	10	9	12	2
E Software	4	B	7	11	8	12	1
F Implement	1	C,D, E	12	13	12	13	0

- 1) Accept the Speedo bid at cost of \$6,200,000
- 2) Hire the training contractor at a cost of \$500,000, cutting training time to 2 months
- 3) Purchase an off-the-shelf software for \$100,000 in addition to the time your software people would normally take, cutting software duration to 3 months

Option (1) would save \$200,000 in idle production, at additional cost of \$100,000

Option (2) would not save any time as it was slack 2 months

Option (3) would not save any time as it was slack 1 month

Thus, to cut from 13 to 12 months, accept the Speedo bid, resulting in the schedule shown in Table 10.4.

Another month of idle production would be saved by cutting project duration to 11 months. Options are:

- 1) Accept the Dynamo bid at a cost of \$6,500,000
- 2) Hire the training contractor at a cost of \$500,000
- 3) Purchase an off-the-shelf software for \$300,000 in addition to the time your software people would normally take, cutting software duration to 3 months

Here there are two parallel critical paths: A-B-C-F and A-B-E-F. To reduce to 11 months would require crashing both, at additional cost of \$300,000 for the robotics system and \$100,000 in additional software costs, together \$400,000 which exceeds the benefit of \$200,000.

Table 10.4 CPM using the Speedo option

Activity	Duration (months)	Pred	Early start	Early finish	Late start	Late finish	Slack
A BPR	5	None	0	5	0	5	0
B Bids	2	A	5	7	5	7	0
C Install	4	B	7	11	7	11	0
D Training	3	B	7	10	8	11	1
E Software	4	B	7	11	7	11	0
F Implement	1	C,D, E	11	12	11	12	0

Thus the optimal solution would be to adopt the Speedo option, and not hire the training contractor nor purchase the software.

10.5 Summary

Often projects encounter optional means to accomplish activities. Each probably has a different cost, so project planning involves identification of a plan to accomplish the project at maximum revenue (or for government projects, minimum cost). Crashing is identifying how the project can be accomplished faster at a cost. If a project falls behind, crashing can be applied to evaluate alternatives available. Usually, there is a cost–time trade-off involved. Activities might be sped up through overtime, subcontracting, or hiring more personnel. Brooks' Law, of course, pointed out that hiring more in information systems contexts may actually slow the project, making that approach suspect in that environment. On the other hand, in construction, more shovels can usually dig more ditches faster.

Notes

1. Sundararajan et al. (2013)
2. Tanner and Mackinnon (2015).

References

- Sundararajan S, Bhasi M, Vijayaraghavan PK (2013) Case study on risk management practice in large offshore-outsourced agile software projects. *IET Software* 8(6):245–257
- Tanner M, Mackinnon A (2015) Sources of interruptions experienced during a scrum sprint. *Electron J Inf Syst Eval* 18(1):3–18

Chapter 11

Implementation and Control



Project control, like risk analysis and estimation, needs to be applied from the beginning of the project to the project's end. **Control** is concerned with keeping the project on target with respect to its objectives. Assessment is the means of monitoring project progress to identify problems requiring control action. The project, to be a success, should conform to the project plan as much as possible. Project management must adapt to new circumstances while simultaneously seeking to meet cost, time, and quality targets. This requires close coordination between the project manager and his or her superiors, whether they are part of the same organization or not.

11.1 Risk Management

We have seen that there are many potential problems that need to be anticipated in project management. There are many risks in software development projects. The actions to alleviate these risks are means of control. Actions need to be taken throughout the project and include continual monitoring to ensure that problems do not get out of control.

11.1.1 Personnel Shortfalls

In the area of personnel management, one obvious approach is to obtain quality people. When operating in a matrix organization, people are obtained from the functional areas of the organization. Boehm recommended meeting with these functional managers to preschedule good people by name and document these agreements. If

people are obtained either from a matrix organization or from outside the organization, references should be checked to improve the probability that they will fit their planned roles.

The second personnel technique is team building. Proactive efforts should be undertaken to see that project management and team members share objectives. Participative planning and objective setting activities are useful to attain this aim, as well as group consensus techniques. Brainstorming, the Nominal Group Technique, and Delphi methods are tools that can lead to group consensus by sharing ideas on the issue under consideration.

11.1.2 Controlling Dynamic Requirements

Project requirements are sometimes changed extensively. Some change needs to be accommodated, as there are inevitable environmental changes that must be adjusted for. We have stressed user involvement, but once the scope of the project has been set, additional change (what systems people call scope creep) can be very detrimental to productivity. Some projects get out of control in the sense that requirements are constantly adjusted. One response to gain control over such a situation is to have a **high change threshold**, that is, not changing plans until it is clear that this is necessary. Risks are also reduced by deferring the development of those project components that involve high probabilities of change. Another means of dealing with inevitable changes consists of determining major directions of anticipated changes in requirements and developing the system, so that only minor revisions to the system are required (only one module would need to be replaced rather than revising the entire system).

11.1.3 Controlling Externally Provided Project Components

Combining system components obtained from multiple sources always creates risks of compatibility. These risks can be reduced by the activities of coordination and inspection. **Reference checking** involves contacting existing users of the component in question to verify features such as ease of use, ease of change, and other critical risk factors. **Preaward audits** in the form of checklists and other techniques can be used to reduce the risk that outside vendors are incapable of delivering the component they are to provide. Other means of reducing the risk of failure in externally furnished components include use of award-fee contracts, soliciting competitive designs and prototypes, and involving outside providers in the project team.

11.1.4 Real-Time Performance Risk

The ability of the developed system to perform under actual working conditions is always uncertain until the system is implemented. There are several techniques available to reduce the risk of inadequate performance. In the context of project management, **benchmarking** involves placing the system under a representative workload and analyzing the system's ability to cope with this level of activity. Prototyping is another means of testing the critical performance features of a system. Instrumentation and tuning measure system performance in terms of bottlenecks. When bottlenecks are encountered, workload can be adjusted by changing task sequence, priorities, data distribution, and other workload characteristics to improve system performance. **Technical analysis** examines the ability of the system to function in environments such as distributed processing. The system's fault tolerance, security, and accuracy are also tested. Techniques supporting technical analysis include fault-tree analysis, and failure modes and effects analysis.

11.1.5 Contingent Development Methods

Software systems should deliver functions needed by users. **Incremental development** allows organizations to test the functionality of system parts without wasting money on the entire project should it prove inadequate. **Prototyping** is a useful means of verifying the functionality of systems to users. Prototyping is also good for assessing the fault tolerance of systems. As the functionality of systems components is evaluated, those that are found lacking can be eliminated by **requirements scrubbing**. Cost-benefit methods provide one basis for deciding whether to keep a system feature or not.

Mission analysis is the process of evaluating the contribution of a system component to accomplish an organizational objective. Mission analysis requires examination of the user's mission and the role of information system support in leading to successful accomplishment of those missions. One of the key factors in information system project success has been identified as a clear statement of project objectives. Mission analysis would be a way of implementing this key factor. Tools to study the user's mission include identification of user tasks, data flows, and control procedures, as well as cost-benefit analysis.

11.1.6 Unrealistic Estimates

Many projects suffer from difficulties in obtaining accurate estimates of cost and time in information systems projects. One approach is to design projects to the available budget. The level of delivered functionality would be a variable. Software system

components could be prioritized into mandatory, desired, and optional groups. The final design would have to include mandatory items, should include desired items, and would include optional items only if adequate time and budget were available.

11.2 The Control Process

There are some fundamental differences between planning and control. Planning focuses on setting goals and directions. Control is concerned with guiding project effort toward attainment of these goals. Planning involves allocating resources to elements of the project hierarchy. Control seeks to ensure the effective use of these resources. Planning requires anticipation of problems. Control requires correction of problems. Planning is a motivational effort, while control is more a matter of rewarding achievement.

Control requires understanding what is going on with a project. Project managers need to visit operations and see for themselves what is going on. Many actual problems do not show up in reports until it is too late for effective corrective action to be taken. Furthermore, a sense of the morale of members of the project team is better gained by actual visits.

Visits, in turn, do not reveal details about performance that reports can. Reports of project progress should be as accurate as possible. Reports are used in a control sense by providing concrete measures of actual versus planned accomplishments of technical activities. Both time and cost reports are crucial to give the project manager a clear picture of project performance. Major variances need to be understood by the project manager. There could be reasons beyond the control of anyone connected with the project. On the other hand, they could be the result of things that the project management team could correct. These variances also trigger a need for overall management to decide whether the project should be continued or dropped.

There are three general phases of the control process. **Performance standards** are set at the beginning of the project. These provide technical specifications, budgeted costs, schedules, and resource requirements. Source documents include user specifications and the project plan. Comparisons are made between actual performance and the plan during operations (**assessment**). Major project control elements include projected completion date and total estimated project cost. When severe variances are experienced, the adoption of **corrective action** needs to be considered.

Control Process

Performance Standards
Assessment
Corrective Action

Task	Description	SE	HA	SA	TR	User
A1	Define work process	A				R
A2	Identify required information and sources	R				C
A31	Basic software	R		C		
A32	Data access required	R		C		C
A33	Vendor software required	C	I	R	I	I
A4	Identify required hardware components	C	R			
B1	Obtain quotations from hardware vendors			R		
B21	Identify potential software products			R		
B22	Obtain test copies for user to examine			R		
B23	User comparison and approval		C	C	I	R
C1	Order components	R	I	I	I	
C2	Test components for technical functionality	R	C	C		
C3	Assemble system	R	C	C		
C4	Train users on software				R	C
C5	User test of system	C	I	I	I	R

Fig. 11.1 Example of a responsibility chart

There are many tools to control project development. We review three representatives here, in the area of personnel management, accounting, and work authorization.

11.2.1 Responsibility Assignment

A project team is formed to bring people with a variety of skills together to accomplish the project's objectives. A project responsibility chart is a simple device commonly used to clearly state who is responsible for each project activity (Fig. 11.1).

Note that the RACI (Responsible, Accountable, Consulted, Informed) charts discussed in Chapter 5 are closely related to the responsibility chart given in Fig. 11.1. Both forms can be used to clearly delineate the responsibilities of all parties involved. Also valuable is the information related to who must be consulted, informed, or is available for advice.

11.2.2 Budget

The traditional means of implementing cost control is **variance analysis**. Accurate measurement requires accurate identification of the quantities of work involved. Some costs are nonlinear, making it more difficult to estimate costs. This is especially true in information system projects, where progress is uncertain until testing is complete.

Cost control works most effectively when it is tied to work packages. **Work packages** include work descriptions, time-phased budgets, work plans and schedules, resource requirements, and assignments of responsibility. To make sense of cost reports, managers need to keep up with changes generated by schedule delay, differences in planned and actual times, and changes in the scope of work. The **earned value concept** is very useful to project management in this process. Earned value is the measure of the budgeted value of work that has been accomplished. Earned value indicates how much of the budget was planned to have been spent in order to do the amount of work done so far. This concept operates on the **standard time unit**, which measures work accomplished in terms of what a normal, prudent worker with average luck would accomplish in that time unit. Standard hours are traditionally used in manufacturing. Other time units, such as day or week, can be used as well. A standard week is probably more appropriate for information systems work. This can be useful in measuring the efficiency of a work unit. It also focuses on activity completion. Calculations are shown in Table 11.1.

In this example, work packages A and B are completed, so the earned value is easy to calculate. The measures for unfinished activities are more problematic, because it is difficult to accurately estimate the percentage of work completed. We have seen that in information systems projects, the final testing and debugging phases can include a great deal of unanticipated work. However, estimates of earned values can be calculated. In this case, credit is taken for half of the effort required on activity C and 75% of the effort required on activity D.

Relative efficiency can be calculated by dividing earned hours by actual hours. For instance, if the actual effort expended on work package A was 25, it would have been rated as $20/25 = 0.80$ efficient. If work package B took 8 units of effort, the efficiency rating would be $10/8 = 1.25$, or 25% more efficient than expected. If activities C and D had actual expenditures equal to their earned value, the efficiency of this project component would be the total earned value (51) divided by total expended effort (54) $= 51/54 = 0.94$, or 94% of planned, on average. However, these measures need to be considered considering conditions beyond the control of the project team. If late deliveries of materials or software cause work package A to run over its budget, the inefficiency could not, in all fairness, be attributed to poor performance on the part of those responsible for doing the work of work package A. Project management needs to coordinate closely with the project team to understand the reasons for work lagging in time or running over in cost.

Table 11.1 Earned value calculations

Work package	Budgeted	% Complete	Earned value	Actual
A	20	100	20	25
B	10	100	10	8
C	12	50	6	6
D	20	75	15	15
Totals			51	54

A more complete example with updates over time is presented. Table 11.2 gives a list of five project activities, with predecessor relationships, durations, and budget.

The schedule for this project has the following early start schedule given in Table 11.3.

Assume that 5 weeks into the project, status is as given in Table 11.4.
Project efficiency is 150 earned over 160 actual, or 0.9375.

In Table 11.4, Actual is input obtained from accounting. Completion percentages are obtained from field personnel. Earned for each activity is Full budget times percent complete. The Earned and Actual for the project are simply the sum of those elements over all activities. Efficiency is Earned over Actual (and can be calculated not only for the full project, but also for each activity). System design is only 83% efficient and Develop interfaces 80%. These activities are having problems. Conversely, Code seems to be doing very well, at 120% efficiency. Overall, the project is over budget by \$10K, with an efficiency of 93.75%.

Table 11.2 Demonstration Input

Label	Activity	Predecessor	Duration (weeks)	Budget (K)
A	System design	None	3	50K
B	Code	A	5	150K
C	Develop interfaces	A	4	80K
D	Train	C	2	30K
E	Implement	B,D	1	40K

Table 11.3 Early start schedule

Activity	ES	EF
System design	0	Week 3
Code	Week 3	Week 8
Develop interfaces	Week 3	Week 7
Train	Week 7	Week 9
Implement	Week 9	Week 10

Table 11.4 Status 5 weeks in

Activity	Full budget	Complete	Earned	Actual
System design	50	100%	50	60
Code	150	40%	60	50
Develop interfaces	80	50%	40	50
Train	30	0%	0	0
Implement	40	0%	0	0
			150	160

Progress is usually monitored at least weekly, but we will demonstrate with longer intervals. Table 11.5 gives project status 10 weeks into the project:

In this case, efficiency slipped a bit (300 earned over 330 actual gives efficiency of 0.909). System design was completed at the 5- week stage, and so is the same. Code slipped, with efficiency now dropping to 93.75%. Develop interfaces caught up and finished at 100% efficiency. Train is running 10K over budget at this stage, two-thirds complete. The project efficiency overall dropped to 90.9%.

Table 11.6 gives data as of project completion.

After week 10, Train stayed 10K over budget. Efficiency for this activity rose because the numerator and denominator rose by the same amount. Implement came in 10K under budget, giving it the only efficiency in the positive range. The project efficiency overall rose to 94.59%.

The point is that monitoring efficiency over time helps project control by identifying where problems are occurring. Management needs to find out why and seek to rectify negative situations. Project efficiencies also can be used for evaluation of project management personnel.

People tend to do the easiest work first, especially if it is important to appear as if they are making progress. In project environments, this often leads to higher levels of progress reported than is actually the case. Projects are almost always reported to be on schedule until they are about 90 or 95% complete. Some activities finish as scheduled. But many activities start to lag the planned completion date in the latter stages of their work. One reason for this in information systems projects is that problems are often not identified until the testing stage. Progress reports in projects

Table 11.5 Status 10 weeks in

Activity	Full budget	Complete	Earned	Actual
System design	50	100%	50	60
Code	150	100%	150	160
Develop interfaces	80	100%	80	80
Train	30	67%	20	30
Implement	40	0%	0	0
			300	330

Table 11.6 Completed project calculations

Activity	Full budget	Complete (%)	Earned	Actual	Activity efficiency
System design	50	100	50	60	0.8333
Code	150	100	150	160	0.9375
Develop interfaces	80	100	80	80	1.0000
Train	30	100	30	40	0.7500
Implement	40	100	40	30	1.3333
Efficiency			350	370	0.9459

should be considered suspect until actual completion. That is the principal point of the milestone concept.

The primary role of the project information system is to monitor project progress. Data is collected from a variety of sources, including materials invoices, timecards, change notices, test results, and attitude surveys. Reports convey this information to key managers. All levels of management need to be provided with the detailed information they need to identify problems and take corrective action. These reports need to identify variances. They also need to be published in a timely fashion.

11.2.3 Work Authorization

Authorization is a primary means of controlling projects. Upper management authorizes the project manager to proceed. The project manager in turn authorizes departments within the project management team to accomplish various elements of work through work orders. The authorization chain works its way down through the organization. Ensuring that expenditure and work activity do not proceed without authorization is a primary means of maintaining control over the project.

The data collection process works pretty much in reverse of the authorization flow. Work sections report their progress to their managers, who report up the hierarchy until, ultimately, the project manager receives summary reports. These summary reports need to focus on problems and to identify accomplishments to date and current cost performance. The project manager needs to keep upper management informed. Upper management usually uses project information to merely keep informed of activities, unless things get so bad that canceling the project appears appropriate.

Because of the uncertain nature of information systems projects, change is to be expected. Management control through work authorization is a way to manage this change. Reasons for change include:

- Change in project scope and specification
- Changes in design
- Changes to improve rate of return
- Changes to adopt improvements

The owner may desire change in project scope and specification owing to new circumstances. These can arise because of new regulations or changes in the market. Changes in design can occur if the original plan will not work or, possibly, if new technology may now be available that is clearly better than what was known when the project was adopted. These can arise because of errors, omissions, afterthoughts, or revised needs. Such changes can offer opportunities for improvement but may also open opportunities for major foul-ups. Changes to improve the rate of return are a financial factor, where the owner has better things to do with the money required by the project.

Changes are troublesome for project teams. The more the project is completed, the more difficult it is to adopt changes. Changes can impact project scope, cost, and

time, which in turn can lead to major sources of conflict. A change review process is a very good way to consider changes as rationally as possible. Changes should be avoided unless there are compelling reasons.

Project termination may be appropriate under a variety of circumstances. Sometimes, the owner may determine that it is better to stop than to continue. This can be due to changes in the market, cost overruns, or depleted resources. When termination before completion occurs, there are a few things that still need to be done. Final closeout activities need to be accomplished to ensure that the owner receives whatever he or she is entitled to and the project team gets their appropriate payment. There is also a need to coordinate with functional managers of the organization to return those individuals assigned to the project on a temporary basis. Other members of the project team may need to be reassigned, possibly to other projects.

11.2.4 Project Evaluation

Project evaluation is concerned with monitoring progress on a project. The ultimate output of project evaluation is the decision to continue with the project or to quit. There also is a need for after-action reports, providing a means to learn from the experiences of a project.

Formative evaluations refer to those that are used to control the project. Because projects involve so many interrelated activities, and are undertaken with limited resources, there can be a lot of apparent reasons why a particular activity or work package is not going well. Furthermore, it is very easy for management to think there is a problem when there really isn't.

Meetings are very important in project control. Regular meetings in such an environment are well worth the time taken. There is a need within projects to maintain high levels of communication and coordination. Regularity leads to more thorough preparation on the part of participants. Two commonly used types of meetings used to manage projects are to coordinate activities (usually unstructured) and to reach decisions (usually more structured).

After-action evaluations involve first classifying completed projects into categories of success and failure. Management needs to understand why successful projects worked as well as they did. The reason could be especially favorable external conditions or, possibly, especially effective implementation by the project team. The key to success is to understand what is true for a given project.

It is also important for organizations that are continuously involved in projects of a given type to gather information about time and cost performance for specific work packages. These work packages should be measured in terms that occur across projects so that the organization obtains knowledge about what doing a particular bit of work takes. This gives the organization a tremendous advantage over time relative to their competitors who either lack experience or fail to keep similar records.

11.3 Schedule Delay

We have contended that projects are often (in fact usually) late, due to the complex interaction of activities with probabilistic durations. This has been found to be true in many repetitive project environments to include construction as well as software system development projects.

In the construction industry, delay types, or allocation of blame (owner versus contractor, or acts of nature), is often important. Delay types are also categorized as excusable (compensable or noncompensable) and nonexcusable.

There are several techniques comparing as-planned and as-built schedules reviewed by Alkass et al. (1996).¹

1. Global impact
2. Net impact
3. Adjusted as-built critical path method (CPM)
4. “But-for” or collapsing
5. Snapshot
6. Time impact

The global impact technique plots all delays and disruptions on a bar chart, with start and finish dates for each event determined. Total project delay is calculated as the sum of durations of all delaying events. The issue with this approach is that the effect of concurrent delays are disregarded, and delay types are not differentiated as every delay has an equal impact, resulting in an overstatement of entitlement as liability.

The net impact technique displays only the net effect of all claimed delays on a bar chart of the as-built schedule. Time extension is the difference between the as-planned and the as-built completion dates. This method also does not differentiate the type of delay and the true effect of a particular delay on overall project completion is difficult to determine.

The adjusted as-built technique uses a CPM format for the as-built schedule, displaying delays as activities linked to work activities. Critical paths are identified using the as-planned schedule, and again using the as-built schedule. The net effect on project completion date is again used. Again, however, delay types are not considered.

The “but-for” technique is also called a collapsing technique, using the CPM format. One party takes the as-planned schedule and inserts all delays that they accept responsibility for. The updated schedule yields a revised project completion date, which is compared with the as-built schedule, implying that this party is responsible for this much delay. The as-built schedule is calculated from both owner and builder perspectives.

The snapshot technique gives a means to determine the amount of delay in a project, when it occurred, and its causes. Total project duration is divided into time periods (snapshots), often coinciding with major project milestones. The project completion of this extended schedule is compared with the as-planned schedule, determining delay. Then the causes of delay are assessed. This approach is systematic and objective but involves significant effort.

The time impact technique is similar to the snapshot technique, examining effects of delays at different project times. As opposed to the snapshot technique, the time impact technique concentrates on specific delays, not time periods. This method also takes significant effort and requires analysis to apportion entitlement.

The “but-for” approach seems the most straightforward. Usually, a contractor has a penalty to pay for being late. The “but-for” contractor schedule is a way to show a fair and reasonable finish time given no contractor fault. It is possible that a management firm is the agent for the owner, in which case responsibility needs to be allocated across the two entities, and the “but-for” schedules for both contractor and manager can be compared and serve as input to the negotiations to assess relative penalty.

11.3.1 Demonstration Schedule

The following case is widely used in civil engineering publications. The project consists of ten activities (A through J) with the following durations and precedence relationships (Table 11.7).

This yields the planned schedule shown in Figs. 11.2 and 11.3.

Delays are categorized as excusable noncompensable (EN)—nature, excusable compensable (EC)—manager’s responsibility, and nonexcusable (NE)—contractor’s responsibility. In the example, delays by activity and category are given in Table 11.8.

These delays result in the as-built schedule shown in Fig. 11.4.

This as-built schedule is 18 days later than the planned finish time of 23 days. We can create the “but-for” schedule to show what would have happened but for delays due to management by taking out the “M” delays, obtaining the “but-for” schedule from the perspective of the contractor (Fig. 11.5).

Table 11.7 Kraiem and Diekman CPM data

Activity	Planned duration (days)	Actual duration(days)	Predecessors
A	7	11	–
B	5	10	–
C	7	12	A
D	9	9	B
E	6	15	B
F	4	6	C
G	3	5	D
H	9	11	E
I	5	12	F
J	3	5	H

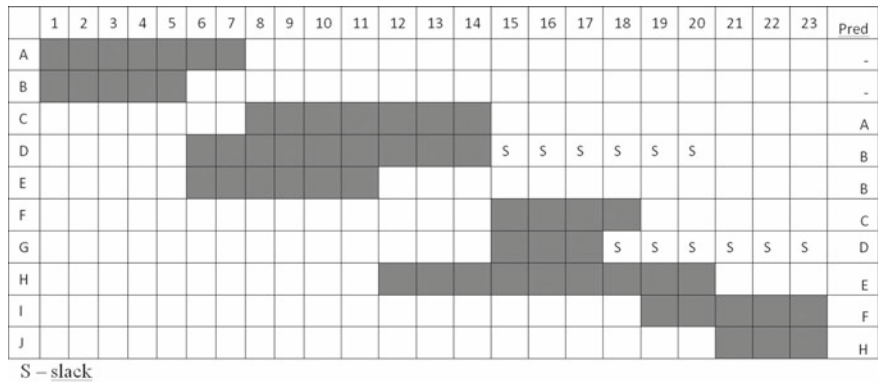


Fig. 11.2 Planned schedule

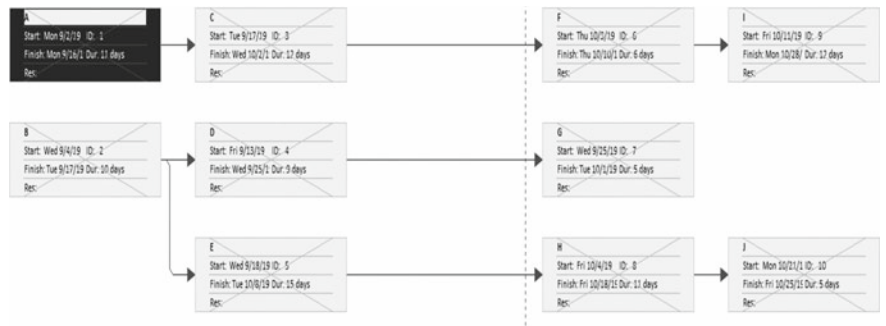


Fig. 11.3 Network diagram of planned schedule

Table 11.8 Demonstration of project delays

Activity	EN delay—excusable	EC delay—manager	NE delay—contractor
A	1 day		3 days
B	3 days	1 day	1 day
C		2 days	3 days
D			
E	5 days	3 days	1 day
F		2 days	
G		1 day	1 day
H	1 day	1 day	
I	2 days	2 days	3 days
J	2 days		

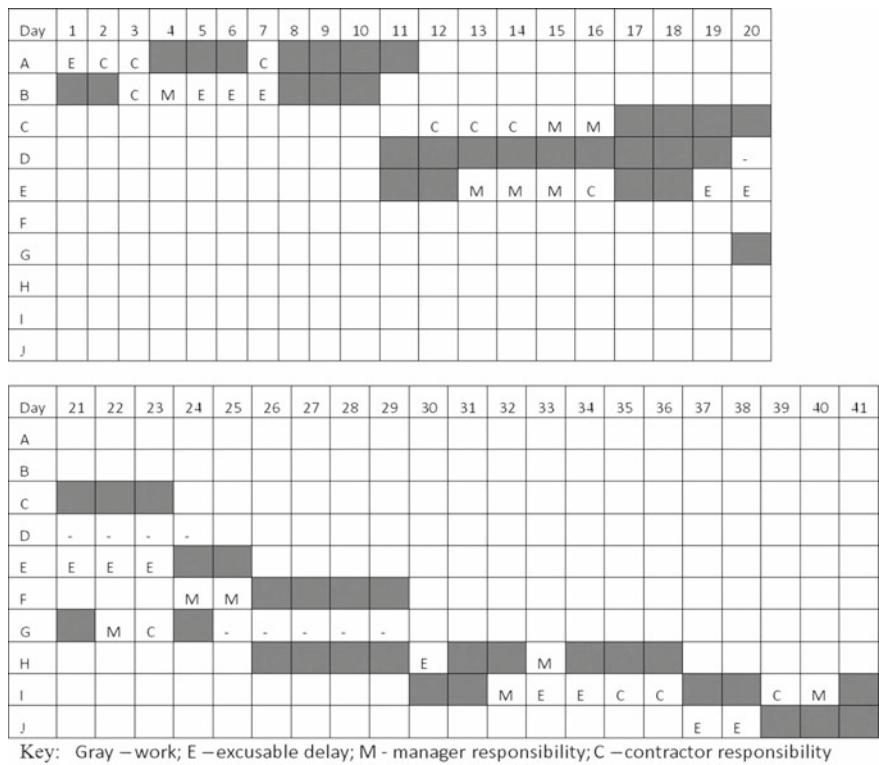


Fig. 11.4 As-built schedule

Figure 11.5 shows that but for the delays that were the responsibility of management, the contractor would have finished the project in 36 days. The argument would thus be that the contractor is responsible for 13 days of penalty rather than 18. If there is joint responsibility to be considered, the “but for” schedule from the perspective of the management firm is shown in Fig. 11.6

Here we see that the delays due to contractor responsibility taken out result in project completion time of 39 days. This is 16 days late. There is an overlap of 13 days, with the management firm responsible for the extra 3 days.

Such an exercise usually involves exercise of negotiation skills, arguing over who is responsible for what. This exercise generally starts with the project participants, but often then extends to the involvement of lawyers, and if that is not resolved, courtrooms.

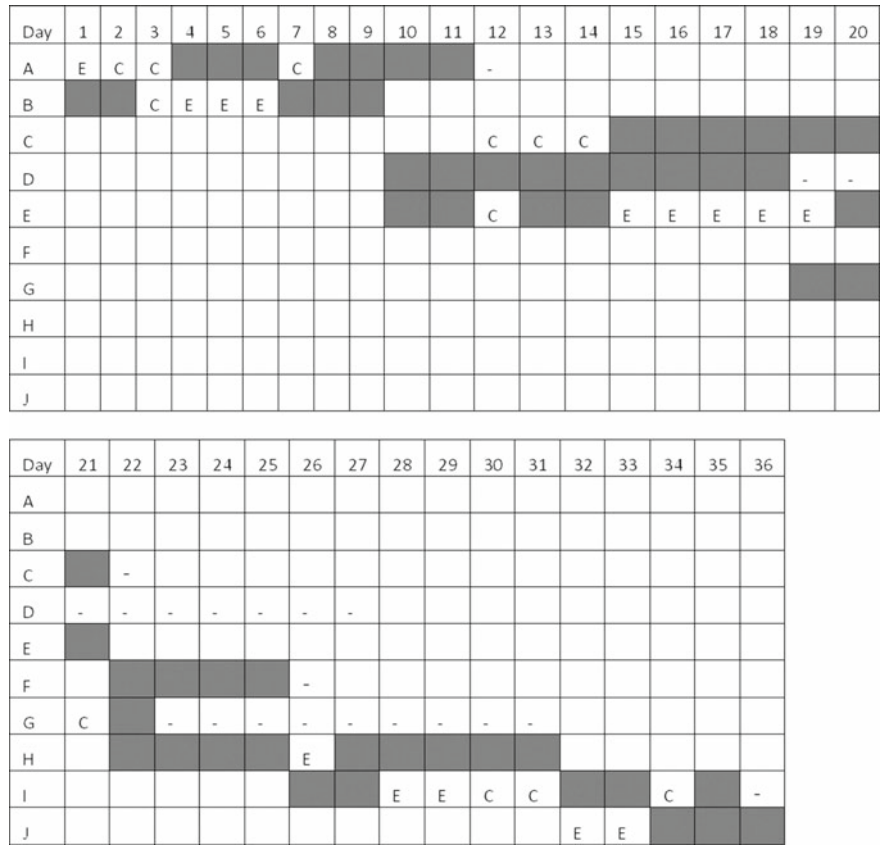


Fig. 11.5 But-for contractor perspective

11.4 Summary

Project control is a means to guide project efforts toward achievement of project goals. A primary means of control is to compare actual work with planned accomplishments. This requires accurate data collection and efficient dissemination of project reports.

There are many reasons that projects fail. Consideration of the failures of others is needed to increase the probability of success of a specific project. High levels of uncertainty and change require flexible project management. Experience and preparation help increase the odds of project success.

Project risk management needs to be applied throughout the life cycle of the project. Consideration of risk was important when the project was initially considered for adoption and was implemented by estimating what could go wrong in developing the project. In the organization stages of the project, there are several things project management can do to ensure good people are obtained, that estimates are realistic,

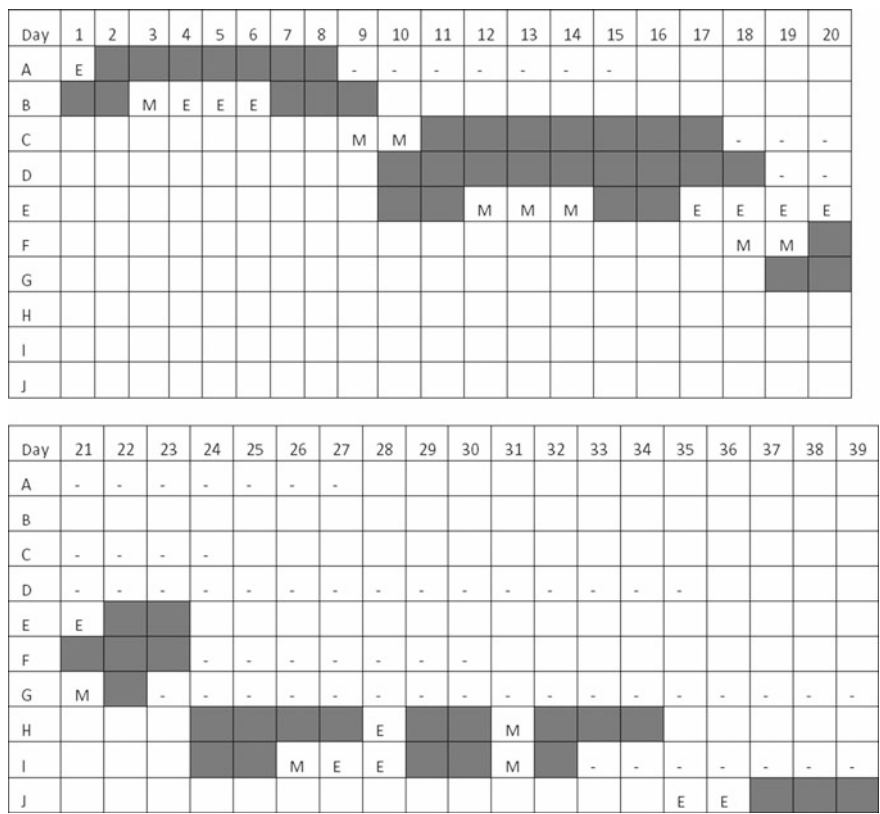


Fig. 11.6 But-for manager perspective

that contingency options are available should problems arise, and that project components delivered by those outside the project organization are received in a timely fashion and perform to specifications. During the building of the project, original time and cost budgets need to be compared with actual performance to determine whether problems are arising. It is important not only to understand that problems exist, but to plan for ways of dealing with the problems that are identified.

Project control should clearly identify responsibilities for work packages and use these work packages as the basis for cost accounting. The authorization process ensures that management retains control over the project. There is a need to constantly update projected performances in order that top management has a complete picture and can make decisions to continue the project based on sound, objective information.

Project evaluation provides organizations that deal with a continuous stream of projects the ability to develop a database of knowledge by work package. This gives such organizations a tremendous edge over competitors that are unable to do the same. Furthermore, careful analysis of post action success or failure enables organizations to better understand their work environment.

Schedule delay is usually present in projects. Contracts are usually written to cover contingencies. The material in this chapter demonstrates some methods that might be utilized to reconcile delay responsibility in an equitable manner. But ultimately, the analysis usually will be material upon which lawyers can base their arguments.

Note

1. Alkass et al. ([1996](#)).

Reference

Alkass S, Mazerolle M, Harris F (1996) Construction delay analysis techniques. *Constr Manag Econ* 14:375–394

Glossary

Agile software development –	Methodology emphasizing speed, with a focused team working intensely to bring out a software project to meet an ambitious deadline
Agile –	Project management approach streamlining project execution to obtain quicker project completion
Artifacts –	Documents describing product backlog
As-built –	Schedule that actually occurred
Benchmarking –	test system under representative workload
Buffer –	spare time added to schedule to improve probability of on-time finish
Business case –	Financial analysis of a project proposal
Business case –	Financial analysis of a proposed information systems software project
But-for –	Schedule that would have occurred but for delays for which one party is responsible
Capability maturity model (CMM) –	Carnegie Mellon Institute system of software development functionality standards widely used to assess organizational software development process capability
Capability maturity model –	Carnegie Mellon rating system for organizational software development maturity
Charter –	Formal document authorizing organizational support for a project
Checklist –	A means of evaluating initial project proposals in terms of a list of expected characteristics
Checkpoint reviews –	reviews of project progress at the conclusion of each phase

COCOMO –	a nonlinear regression means of estimating information systems projects
Compensable –	Delay for which one party to the contract is liable to the other party
Control points –	points in time where specific project progress can be updated
Control subsystem –	Means to manage project operations
Coordinator –	See Expediter
Corresponding failure –	Failure of a system to meet its design objectives
Cost–benefit analysis –	Economic analysis pricing all factors, dividing benefits by investment
Crashing –	speed project at some additional cost
Crashing –	speeding project completion time by shortening one or more activity's durations at some additional cost
Critical path –	chain of project activities with zero slack
Critical path method –	process to schedule a project given activity list, their durations, and predecessor relationships
Data warehousing –	Large-scale database technology to organize and store vast quantities of data, enabling data mining analysis
Decision support systems –	Computer systems used to evaluate alternative decision choices
Detailed task list –	list of unique work packages in a project
Discounted cash flow –	Conversion of cash flows into their worth as of some base period, usually the present
Early start schedule –	schedule where all activities are accomplished as soon as possible given estimated durations
Earned value –	measure of budget earned by actual work accomplished
Enterprise resource planning –	Systems designed to provide all organizational MIS needs through integrated, optimized computer software
Excusable –	Delay for which the entity (owner or contractor) is not liable
Expectation failure –	Failure of a project to meet its stakeholder's expectations
Expediter (or coordinator) –	Individuals who monitor problems in an organization and work on alleviating them rapidly
Feasibility study –	Analysis prior to adoption of a project intended to determine whether the system is doable, within a prescribed budget
Feeding buffer –	buffer added to improve probability of non-critical activity delaying the critical path

Fever chart –	graphic monitoring buffer usage
Float –	synonym for slack
Formal specification –	Documentation of project mission and scope
Function point analysis –	a means of estimating information systems projects based on factor complexity
Functional manager –	Manager of a permanent organization
Functional organization –	Organization by type of work group members do
Gantt chart –	visual graphic of activities by time
Hybrid –	Mix between functional and matrix organizational forms
Implementation phase –	Phase of an information systems project where the finished product is put into operation
Independent slack –	slack for an activity not shared with other activities
Individual characteristics –	Result of different personalities, of individuals with different demographic backgrounds as well as education and experience
Informatics –	Business analytics in a contemporary environment, potentially involving access to big data analyzed by machine-learning software
Intangible factors –	Items of value that are difficult to quantify in monetary terms
Interaction failure –	Condition of a system not being used by its intended users
Interdependence –	People working together, team effort necessary to make projects work
Internal rate of return –	The marginal value of capital that yields a net present value of zero for a stream of cash flow
Internet commerce –	Doing business over the Internet
Interpersonal conflict –	Difficulties encountered by people in getting along with others without friction
Kanban –	Agile lean manufacturing approach
Late start schedule –	schedule where all activities are accomplished as late as possible without delaying project completion time
Liaison –	Individual integrating two or more groups through personal communication
Lines of code (LOC) –	a rough means of estimating information systems projects based on how many lines of computer code are expected
Make or buy decision –	Selecting the alternative among making an information systems element with in-house resources, or purchasing the element from a vendor
Management control –	Use of information systems to control an organization

Matrix organization –	Grid-like structure of dual reporting relationships combining functional and project organizational features
Milestones –	events indicating completion of a project block of activities
Mission analysis –	evaluate component contribution to organizational objectives
Multiple criteria analysis –	Analysis considering not only one measure of value (such as profit), but the value trade-offs over multiple measurement scales
Net present value –	The worth, in today's terms, of a stream of cash flow over time, assuming a given marginal cost-of-capital
Network –	graphic display of project activities showing predecessor relationships
Nonexcusable –	Delay for which fiscal liability is present
Optimistic estimate –	duration estimate used in PERT, conventionally what you would expect not to be exceeded 5 percent of the time
Optimization models –	Models of decisions allowing the identification of the decision that is optimal with respect to selected criterion (or criteria)
Organization chart –	hierarchical display of project reporting relationships
Organization –	Structure to coordinate efforts of many to accomplish shared goals
Outsourcing –	Hiring others to do at least part of one's information system
Payback –	Financial analysis estimating the time required to recover investment
Pessimistic estimate –	duration estimate used in PERT, conventionally what you would expect not to be exceeded 95 percent of the time
Pilot operation approach–	Implementation of a system on a small scale prior to cutting the old system
Process failure –	Failure of a project to meet time and budget requirements
Process –	Activity undertaken to accomplish something the organization needs done
Programmers –	Information systems job title responsible for building systems
Project buffer –	buffer added to overall project
Project champion –	Cheerleader who keeps motivation for the project going

Project entities –	Types of people involved in the project
Project evaluation and review technique (PERT) –	probabilistic form of critical path modeling
Project failure –	Standards for project goals that are not met (budget, time, quality, as well as stakeholder expectations)
Project information subsystem –	Means to monitor accomplishment of project objectives
Project Management Body of Knowledge (PMBOK) –	PMI set of standards comprehensively covering project management processes
Project management system –	Technical core, control subsystem, and project information system
Project manager –	Coordinator of the project, responsible for project success
Project manager –	Individual assigned the responsibility of organizing and managing a project
Project organization –	Organization by project task or groups of tasks
Project Profile –	A display of expected project performance on important criteria, allowing a comparison of alternative projects
Project scope –	Statement giving project aims and constraints
Project team –	Group of people charged with accomplishing the project through the coordinated efforts of diverse skills
Project team –	Staff undertaking the project
Prototyping –	model development to test
Prototyping –	Testing of a system by developing a small-scale model, so that it can be evaluated
Quality control –	Means taken to assure that designed product quality is present
RACI –	responsible, accountable, consult and inform responsibilities for project elements
Rapid prototyping –	Use of prototypes with user feedback to quickly identify system weaknesses
Reference checking –	assess initial systems to eliminate unnecessary or cost ineffective components
Relay race concept –	delay noncritical tasks to start as late as possible
Request for proposal –	Formal document outlining project requirements to prospective vendors, asking for their product and associated price
Resource buffer –	buffer added to improve probability that a critical resource will be available when needed
Resource leveling –	adjustment of a schedule to stay within resource constraints

Resource smoothing –	adjustment of a project schedule to reduce variation in resource usage
Responsibility assignment –	identification of who is responsible for project activities
Responsibility matrix –	grid listing who is responsible for aspects of each project element
Return on investment (ROI) –	Rate of financial return on investment, defined as net profit divided by investment
Schedule delay –	excusable vs. nonexcusable; compensable vs. noncompensable
Scope –	Detailed description of the project and/or product, part of the project charter
Screening –	A method of implementing a checklist for project evaluation by listing the worst acceptable performance on a list of criteria
Scrum –	Project management approach focusing on short iterations of work to gain quicker project completion
Shared slack –	slack for a chain of slack activities
Simulation –	numerical analysis using probability distributions
Slack –	difference between early start and late start schedules – spare time for an activity or chain of activities
SMART –	Simple multi-attribute rating theory, a means of objectively comparing the values of projects considering multiple criteria
Specification phase –	phase of an information systems project where specifications are generated based on requirements analysis
Sponsor –	Client paying for the project
Stakeholders –	Clients—those for whom the project is being undertaken
Standard time unit –	work accomplished in a time unit under normal conditions
Statement of work –	document containing project description, constraints, schedule and budget requirements, and responsibilities
Statement of work –	Formal document specifying standards for a computer software project
Status review meetings –	meetings where project management reviews project progress
Strategic resource buffer –	resource buffer when used in a multiple project environment

Systems analysis –	Function of identifying what an information systems project is intended to do
Systems analysts –	Information systems job title responsible for identifying systems needs and designing the software product
Systems design –	Result of computer software project design intended to accomplish elements identified in systems analysis
Systems development approach –	Rational way of systematically evaluating information systems projects
Tangible benefits –	Benefits that are measurable; in cost–benefit terms, measurable specifically in monetary terms
Task force –	Temporary grouping of people of diverse skills with the intent of accomplishing a specific job
Technical core –	Expertise and equipment providing the means to accomplish the project
Time value of money –	The current worth of future flows of cash flow
Total slack –	sum of an activity's independent and shared slacks
Transaction processing –	Use of computer systems to perform repetitive clerical tasks for an organization
User training –	Training provided to users to demonstrate what the system will do, and how the users can apply the new system
Validation –	Process of ensuring that the software performs as designed
Variance analysis –	accounting measure identifying difference between planned and actual performance
Waterfall model –	Systems development cycle consisting of serial activities
Work breakdown structure (WBS) –	hierarchical chart of project tasks and subtasks
Work in process (WIP) –	Partially completed inventory—in the context of project management, the number of tasks defined for a project

Index

A

Agile, 33, 34, 109–115, 137
Agile software development, 33, 113
Artifacts, 113, 114, 137
As-built, 129, 130, 132, 137

B

Benchmarking, 121, 137
Buffer, 56, 86, 97–99, 102, 103, 137
Business case, 16, 31, 46, 59, 137
But-for, 129, 130, 133, 134, 137

C

Capability Maturity Model (CMM), 30, 41, 42, 109, 112, 137
Charter, 31, 113, 137
Checklist, 61, 62, 120, 137
Checkpoint reviews, 55, 137
Compensable, 129, 138
Constructive cost model (COCOMO), 35, 39, 41, 57, 138
Control points, 50, 55, 138
Control subsystem, 6, 138
Coordinator. *See* Expediter
Corresponding failure, 8, 9, 138
Cost–benefit analysis, 59–61, 64–67, 70
Crashing, 85, 86, 103, 109, 116–118, 138
Critical path, 57, 73, 75, 76, 78, 79, 85, 88, 89, 97–99, 102, 116, 129, 138
Critical path method, 73, 78, 86, 87, 90, 138

D

Data warehousing, 138

Decision support systems, 138
Detailed task list, 51, 52, 138
Discounted cash flow, 65, 66, 138

E

Early start schedule, 73–76, 78, 125, 138
Earned value, 124, 138
Enterprise resource planning (ERP), 13, 138
Excusable, 129, 138
Expectation failure, 9, 138
Expediter (or coordinator), 27, 138

F

Feasibility study, 44, 45, 138
Feeding buffer, 97, 99, 101, 138
Fever chart, 102–107, 139
Float, 76, 139
Formal specification, 31, 139
Functional manager, 24, 26, 45, 119, 128, 139
Functional organization, 18, 19, 22, 24, 139
Function point analysis, 36, 139

G

Gantt chart, 43, 73, 74, 76, 77, 82, 83, 111, 139

H

Hybrid, 26, 34, 139

I

Implementation phase, 139
 Independent slack, 76, 139
 Individual characteristics, 15, 19, 139
 Informatics, 139
 Intangible factors, 59, 67, 70, 71, 139
 Interaction failure, 9, 139
 Interdependence, 15, 139
 Internal rate of return (IRR), 61, 66, 139
 Internet commerce, 139
 Interpersonal conflict, 15, 16, 19, 139

K

Kanban, 114, 115, 139

L

Late start schedule, 75, 76, 139
 Liaison, 27, 139
 Lines of code (LOC), 35, 36, 39, 49, 57, 139

M

Make or buy decision, 139
 Management control, 56, 127, 139
 Matrix organization, 10, 15, 19, 24, 26, 119, 120, 140
 Milestone, 6, 31, 55, 57, 85, 127, 129, 140
 Mission analysis, 121, 140
 Multiple criteria analysis, 71, 140

N

Net present value, 60, 61, 64–70, 140
 Network, 10, 14, 21, 46, 53, 75, 79, 88, 116, 131, 140
 Nonexcusable, 129, 140

O

Optimistic estimate, 103, 140
 Optimization models, 140
 Organization, 4, 140
 Organization chart, 22, 53, 54, 140
 Outsourcing, 45, 56, 81, 140

P

Payback, 61, 65, 71, 140
 Pessimistic estimate, 140
 Pilot operation approach, 11, 140
 Process, 113, 140
 Process failure, 9, 140

Program Evaluation and Review Technique (PERT), 85, 87–91, 93–95, 101, 141
 Programmers, 3, 12, 14, 39, 46, 52, 140
 Project buffer, 97–99, 102, 140
 Project champion, 7, 10, 12, 140
 Project entities, 7, 141
 Project failure, 7–9, 11, 12, 17, 141
 Project Management Body of Knowledge (PMBOK), 24, 109, 110, 141
 Project management system, 6, 7, 57, 141
 Project manager, 5–7, 10, 12, 13, 17–21, 24–27, 31, 45, 46, 78, 86, 103, 119, 122, 127, 141
 Project organization, 21, 23–27, 50, 53, 134, 141
 Project profile, 61, 63, 141
 Project scope, 16, 45, 109, 127, 141
 Project team, 3, 6, 7, 9, 10, 12, 13, 15, 17, 20, 24, 27, 45, 53, 86, 113, 120, 122–124, 127, 128, 141
 Prototyping, 31–33, 46, 56, 109, 121, 141

Q

Quality control, 10, 47, 141

R

RACI (Responsible, Accountable, Consult, and Inform), 54, 123, 141
 Rapid prototyping, 32, 141
 Reference checking, 120, 141
 Relay race concept, 101, 102, 141
 Request for proposal, 45
 Resource buffer, 97, 101, 141
 Resource leveling, 78, 82, 141
 Resource smoothing, 81, 82, 142
 Responsibility assignment, 123, 142
 Responsibility matrix, 53, 142
 Return on investment (ROI), 62, 65, 142

S

Schedule delay, 124, 135, 142
 Scope, 16, 23, 29, 31, 34, 49, 124, 142
 Screening, 61–63, 70, 142
 Scrum, 109, 113–115, 142
 Shared slack, 76, 142
 Simple multi-attribute rating technique (SMART), 69–71, 142
 Simulation, 85, 87, 91–95, 142
 Slack, 55, 73, 75, 76, 97, 117, 142
 Specification phase, 44, 142
 Sponsor, 7, 31, 46, 142

Stakeholders, [7](#), [9](#), [18](#), [31](#), [112](#), [113](#), [142](#)
Standard time unit, [124](#), [142](#)
Statement of work, [31](#), [44](#), [50](#), [142](#)
Status review meetings, [55](#), [56](#), [142](#)
Strategic resource buffer, [97](#), [142](#)
Systems analysis, [6](#), [49](#), [110](#), [143](#)
Systems analysts, [3](#), [9](#), [14](#), [143](#)
Systems design, [12](#), [143](#)
Systems development approach, [30](#), [32](#), [34](#),
[43](#), [44](#), [143](#)

T

Tangible benefits, [60](#), [67](#), [70](#), [143](#)
Task force, [23](#), [25](#), [27](#), [143](#)
Technical core, [6](#), [143](#)
Time value of money, [43](#), [61](#), [65](#), [143](#)
Total slack, [143](#)

Transaction processing, [143](#)

U

User training, [6](#), [8](#), [143](#)

V

Validation, [31](#), [143](#)
Variance analysis, [123](#), [143](#)

W

Waterfall model, [14](#), [31](#), [32](#), [143](#)
Work breakdown structure (WBS), [51](#), [55](#),
[57](#), [111](#), [143](#)
Work in process (WIP), [114](#), [143](#)